Riverbed IQ User Assistance

Copyright and Legal Statements

Copyright 2024, Riverbed Technology.

Built at 08:46 PM, 02 October 2024

Some content, such as terminological definitions for common network objects and behaviors, may use text taken from the ITIL V4 Foundation Glossary by AXELOS:
Copyright © AXELOS Limited 2012. All rights reserved. Material is reproduced with the permission of AXELOS

Table of Contents

1

Riverbed IQ User Assistance 1

Copyright and Legal Statements **2**

Table of Contents **3**

Welcome **24**

First Steps With Riverbed IQ **25**

Set up Riverbed IQ: 25

Add users to Riverbed IQ: 25

Operate Riverbed IQ: 25

User Management Page **26**

Configuring Riverbed Edge and Data Sources **27**

Configuring Riverbed Edge on an AWS VM **28**

Configuring Riverbed Edge on an Azure VM **31**

Configuring Riverbed Edge on an ESXi VM **34**

Firewall Rules **37**

Troubleshooting Riverbed Edge Status **39**

A Riverbed Edge instance shows red status on the Data Sources page 39

There's no discernible evidence of data being pushed to Azure blob storage 39

iotedge check Output Example 40

Adding a Data Source **42**

Configuring Riverbed AppResponse **44**

Configuring Riverbed NetIM **48**

Configuring Riverbed NetProfiler **50**

Troubleshooting Data Source Connections **57**

IQ has not received data during the expected time frame from this source 57

For a NetProfiler data source: 57

Data is not flowing 57

Clock synchronization 58

Missing configuration: host data 58

Missing configuration: network interface data 64

For an AppResponse data source: 65

Data is not flowing 66

Clock synchronization 66

Missing configuration: Host data 66

For a NetIM data source: 68

Check device polling configuration 68

Check the Riverbed IQ Connector 69

Authentication failure 69

NetProfiler data source 70

AppResponse data source 72

NetIM data source 72

The data source has not provided any recent status update 73

Preferred Data Sources **75**

Important Considerations 75

Changing the Preferred Data Source 76

System Overview **77**

Customer Environment 78

Riverbed IQ 78

Connecting a Customer Environment to Riverbed IQ SaaS **80**

Incident Generation and Runbook Execution **83**

Introduction 83

Data Sources: Key Measurements 84

Ingest & Analytics: Indicators 85

Overview 85

Anomaly Detection Algorithms 85

Static Threshold (ThresholdDetector) 85

Time-series Baseline (BaselineDetector) 86

Dynamic Threshold (DynamicThreshold) 86

State Change (ChangeDetector) 87

Continuously Increasing (AlwaysIncreasingDetector) 87

Current Key Measurements and Associated Anomaly Detection Algorithms 88

Correlation: Detections 91

Incidents 91

Overview 91

Basic Anatomy of an Incident 92

Incident Lifecycle 100

Automation (LogIQ Engine) 102

Managing Incidents **105**

Riverbed IQ Pages **106**

Impact Dashboard **107**

Incidents Page **109**

Automation Management Page **111**

Runbooks Pages **113**

Filtering the List of Runbooks 113

Working On an Existing Runbook 114

Creating a New Runbook 114

Runbook List Columns 114

Runbook Editor/Subflow Editor **117**

Tool Bar 117

Runbook Node Palette 119

Canvas 120

Properties Blade 120

Runbook Analyses Page **121**

Runbook Scheduling page **123**

Creating a Schedule 124

Usage Considerations 124

Integrations Library Page **125**

Installing an Integration 125

Modifying an Integration After Installation 126

Analytics & Threshold Configuration page **128**

Editing a Static Threshold Value 135

User Management Page **136**

Data Sources Page **137**

License and Usage Page **139**

Third Party Authentication Page **140**

API Access Page **141**

Monitoring Health Page **143**

Overall Status 143

Performance Charts 144

Incident Search **145**

Using the Text Panel 145

Support Page **146**

Built-In Runbooks **147**

Runbook Nodes **148**

Runbook Node Categories **151**

Triggers 151

Data Queries 151

Functions 151

Logic 151

Impacts 151

Integrations 151

Visualizations 151

Variables 151

Miscellaneous 152

Interface Issue **153**

Properties 153

Data Query Reference Time 153

Device Issue **154**

Properties 154

Data Query Reference Time 154

Application Issue **155**

Properties 155

Data Query Reference Time 155

Location Issue **156**

Properties 156

Data Query Reference Time 156

Webhook **157**

Properties 157

Data Query Reference Time 157

Input Node **158**

Starting Entity 158

Additional Parameter Values Received From Caller 158

Applications Query **159**

Properties 159

Filters 159

Limit 159

Metrics 159

Properties 159

Devices Query **160**

Properties 160

Filters 160

Limit 160

Metrics 160

Properties 160

Device Status **161**

Properties 161

Filters 161

Limit 161

Metrics 161

Properties 161

Devices CPU Utilization Query **162**

Properties 162

Filters 162

Limit 162

Metrics 162

Properties 162

Devices Memory Utilization Query **163**

Properties 163

Filters 163

Limit 163

Metrics 163
Properties 163
Network Device Disk Utilization Query 164
Properties 164
Filters 164
Limit 164
Metrics 164
Properties 164
Devices Availability Query 165
Properties 165
Filters 165
Limit 165
Metrics 165
Properties 165
Devices Up Time Query 166
Properties 166
Filters 166
Limit 166
Metrics 166
Properties 166
Network Device Configuration Changes Query 167
Properties 167
Filters 167

Limit 167
Metrics 167
Properties 167
Device Performance 168
Properties 168
Filters 168
Limit 168
Metrics 168
Properties 168
Hosts Query 169
Properties 169
Filters 169
Limit 169
Metrics 169
Properties 169
Client Hosts Query 170
Properties 170
Filters 170
Limit 170
Metrics 170
Properties 170
Server Hosts Query 171
Properties 171

Filters 171
Limit 171
Metrics 171
Properties 171
Client-Server Pairs Query 172
Properties 172
Filters 172
Limit 172
Metrics 172
Properties 172
Locations Query 173
Filters 173
Limit 173
Metrics 173
Metrics 174
Client Locations Query 175
Properties 175
Filters 175
Limit 175
Metrics 175
Properties 175
Server Locations Query 176
Properties 176

Filters 176
Limit 176
Metrics 176
Properties 176
Network Interface Traffic Query 177
Properties 177
Filters 177
Limit 177
Metrics 177
Properties 177
Interface Status Query 178
Properties 178
Filters 178
Limit 178
Metrics 178
Properties 178
Network Interface Error Query 179
Properties 179
Filters 179
Limit 179
Metrics 179
Properties 179
Interface Performance 180

Properties 180
Filters 180
Metrics 180
Properties 180
Protocols **181**
Properties 181
Filters 181
Limit 181
Metrics 181
Properties 181
DSCPs **182**
Properties 182
Filters 182
Limit 182
Metrics 182
Properties 182
Client-Server Location Pairs **183**
Properties 183
Filters 183
Limit 183
Metrics 183
Properties 183
Client-Server Pairs and Protoport **184**

Properties 184
Filters 184
Limit 184
Metrics 184
Properties 184
User Device Node 185
Properties 185
Filters 185
Limit 185
Metrics 186
Aggregator Node 187
Properties 187
Grouping 188
Metrics 188
Transform 189
Output Data Definition 189
Transform Template 190
Decision Branch 191
Set Incident Priority 192
Properties 192
Tag As Affected Applications 193
Properties 193
Tag As Affected Locations 194

Properties 194
Tag As Affected Users **195**
Properties 195
HTTP Request **196**
Authentication 196
Endpoint 196
HTTP Headers 196
HTTP Payload 196
Retry Until Ready 197
Context 197
Table **198**
Pie Chart **199**
Bar Chart **200**
Time Series Chart **201**
Cards **202**
Gauges **203**
Connection Graph **204**
Text **205**
Properties 205
Title 205
Row 205
Notes 205
Set Primitive Variable **206**

Properties 206
Variables to Set 206
Template to Assign Value to Variables 206
Set Structured Variable 207
Properties 207
Variables to Set 207
Comment 208
Label 209
Filters 210
Metric Collection 213
Limits 214
Dialog and Wizard Descriptions 215
Add an Riverbed Edge Wizard 216
Add an Authentication Profile Wizard 217
Add an Automation Wizard 219
Explorer Overview 221
Custom Properties 223
Custom Properties Page 223
Defining a Custom Property 223
Importing and Exporting Custom Properties 224
Searching By Custom Property 224
Global Search 224
Incident Search 224

Custom Properties in Runbooks 224
Custom Properties in Automations 224
Introduction to Runbooks 226
Incident Automations 228
High-level Workflow 228
Creating and Editing Custom Runbooks Using the Runbook Editor 229
Trigger Inputs For Testing Runbooks 231
Automation and Trigger Overview 232
Subflow Overview 233
Subflow Use Case Examples 234
Built-In Subflows 234
Including a Subflow In a Runbook 235
Editing an Existing Subflow 235
Editing Subflow Versions 235
Exporting and Importing Subflow Definitions 236
Deleting a Subflow 236
Updating a Subflow to a New Version 236
Updating Built-In Subflows to a New Version 236
Updating Custom Subflows to a New Version 237
Updating Integrations to a New Subflow Version 237
Vantage Points 238
Variables Overview 239
Primitive and Structured Variables 239

Syntax 239

Variable Definition Editor 241

Add Variable 241

User-Defined Variables 241

Built-In Variables 241

Working With Liquid Templates 242

Template Variables 242

Liquid Template Editor 245

Liquid Basics 248

Tips 248

Built-In Variables 250

Webhooks in Riverbed IQ 252

Webhook Inputs 253

Using JSON Variables With Webhooks 254

Example 1 254

Example 2 258

On-Demand Runbooks 264

Executing On-Demand Runbooks 267

On-Demand Runbook Example 268

Incident Lifecycle Events and Runbooks 274

Simple Runbook Examples 276

Creating and Editing Custom Runbooks Using the Runbook Editor 281

Adding and Assembling Runbook Nodes 283

Working With Decision Branches **284**

Importing and Exporting Runbook Definitions **285**

Copying a Runbook **286**

Testing a Runbook and Previewing Its Output **287**

Deleting a Runbook **288**

Label **289**

Filters **290**

Limits **293**

Metric Collection **294**

Data Ocean Metrics **295**

Riverbed Edge **296**

Data Source **298**

Indicator **299**

Incident **300**

Runbook **301**

Trigger **302**

Support Page **303**

Frequently Asked Questions **304**

General 305

What is Riverbed IQ? 305

What is the release schedule? How frequently will Riverbed IQ deliver new capabilities? 305

Which Riverbed Network Performance Monitoring (NPM) products are supported with Riverbed IQ? 305

Does Riverbed IQ support third-party integration? 306

Does Riverbed IQ learn from the customer environment? 306

Data Ocean 307

What are the implications of moving the data from our large on-premise data sources to Riverbed IQ SaaS? 307

What data is streamed from available Riverbed Network Performance Monitoring (NPM) products into the Riverbed IQ Analytics Pipeline? 308

What data is available to Riverbed IQ for automated investigation? 310

Is it possible to customize the data processed by Riverbed IQ? 310

Automation and Runbooks 312

How many runbooks are included, and can customers create their own? 312

What skills are needed to build a runbook? 314

Security 315

Does Riverbed IQ allow a user to self-service a password reset? 315

What security standards does your SaaS meet? 315

What protocol is used for communication to the Edge device and from the Edge to the SaaS? Is it TLS 1.3? Secure Restful API? Other? 316

What's New In This Version of Riverbed IQ? 317

October 3, 2024 317

Application Roll-Up 317

August 30, 2024 317

Subflow Versioning 317

July 18, 2024 318

Scheduled Runbooks 318
Analytics & Threshold Configuration 318
June 27, 2024 318
Integrations Library 318
April 26, 2024 319
On-Demand Runbooks 319
Custom Decision Branch Labels 319
Preferred Data Sources 319
April 16, 2024 319
Custom UI Filter Configuration 319
Runbook Editor Usability 319
Runbook Test Usability 320
Explorer Detail Enhancements 320
OAuth 2.0 Support for Custom Parameters 320
AppResponse Data Source Vantage Points 320
User Assistance Table of Contents 320
February 7, 2024 321
Custom Properties 321
Subflows 321
October 27, 2023 321
Custom Triggers 321
Async Support in HTTP Request Node 321
August 24, 2023 322

Incident Lifecycle Events 322
Webhook Trigger 322
August 18, 2023 322
July 6, 2023 322
May 24, 2023 323
Runtime Variables 323
Multi-Device Down Analysis Runbook 323
Monitoring Health 323

Welcome

Welcome to Riverbed IQ!

Riverbed IQ enables you to automate responses to incidents in your network. It recognizes incidents that originate from entities in your network, and enables you to automate responses to those incidents by using Runbooks that query the data ocean acquired from those entities to execute context-specific reports and mitigation.

- **Get started:** See First Steps With Riverbed IQ for guidance on how to begin using Riverbed IQ to monitor your network.
- **System overview:** The System Overview page introduces you to Riverbed IQ's basic components.
- **Contact Riverbed support:** Refer to the Support Page for Riverbed Technology customer support contact information.

First Steps With Riverbed IQ

Set up Riverbed IQ:

Configure Riverbed Edge and add one or more data sources.

Add users to Riverbed IQ:

Open the User Management page to specify authorized domains and invite users who should have access to Riverbed IQ.

Operate Riverbed IQ:

Open the Impact Dashboard to learn which occurrences on your network are affecting your vital resources (users, locations, and applications) at the current time. These occurrences are called incidents, and can be investigated using Riverbed IQ's Impact Dashboard, Incidents page, and runbooks.

User Management Page

The User Management page enables you to administer user accounts, roles, and domains for Riverbed IQ.

Click **User Management** in the Navigation pane to display the User Management page.

Click **Invite User** to send Email to a specified account, asking the owner to register with Riverbed IQ as a user. The Email account must be a member of a previously authorized domain. You can also assign a role to the specified account: Admin, Write, or Read.

Click **Manage Authorized Domains** to add or delete domains that your organization trusts for authorizing users and sending and receiving Email. Only Email accounts that belong to authorized domains can be used to invite users to access Riverbed IQ.

Configuring Riverbed Edge and Data Sources

Riverbed IQ's data sources are network monitoring applications such as Riverbed Technology's Riverbed NetIM and Riverbed NetProfiler, which forward observability data to Riverbed IQ for correlation and analysis via Riverbed Edge, which handles data source connections for IQ. Each data source must be configured explicitly to communicate with Edge.

To configure Riverbed Edge, refer to the relevant instructions for your VM type:

- Configuring Riverbed Edge on an AWS VM
- Configuring Riverbed Edge on an Azure VM
- Configuring Riverbed Edge on an ESXi VM

Configuring Riverbed Edge on an AWS VM

Riverbed IQ's data sources are network monitoring applications such as Riverbed Technology's Riverbed NetIM and Riverbed NetProfiler, which forward observability data to Riverbed IQ for correlation and analysis via Riverbed Edge, which handles data source connections for IQ. Each data source must be configured explicitly to communicate with Edge.

Follow these steps to configure an Riverbed Edge instance and a data source:

1. Create new non-admin user accounts on your Riverbed Technology data sources. For NetProfiler, use the role *Operator*. For NetIM, use the role *ALLUVIO_IQ*. For AppResponse, use a non-admin role with permissions for Feature Settings and Definitions set to Full Control.
2. In the Riverbed IQ web UI, open the Data Sources page and click **Add an Edge** to display the Add an Edge wizard.
3. Specify the Riverbed Edge instance name and virtual machine type, and click **Next**.
4. If necessary, specify the connection type for the Edge:
    - If the Edge does not have access to the Internet, specify a proxy to use and enable **Use Websockets**.
    - If the Edge has only port 443 open to the Internet, enable **Use Websockets**.
    - If neither condition applies, simply click **Next**.
5. Inspect the summary of the Edge definition and click **Submit** to make it manageable by IQ. IQ shows you a brief summary of the next steps you need to execute.
   Note: Riverbed IQ employs a shared ownership model in which Riverbed supplies an Edge image for deployment on top of a customer-owned and

managed VM.

a. Download the Edge cloud-init file using the provided external link URL.

b. Go to the external, official Ubuntu VM download page to retrieve a supported version to set up the customer-owned and managed VM that will host Riverbed Edge.

c. Deploy and prepare your VM for Riverbed Edge.

d. Use the Edge cloud-init file to configure your target VM environment for Edge.

6. Select the Edge in the Data Sources page and click on the **...** menu. Choose **Obtain VM Image** to open an external web page for AWS Marketplace for the Ubuntu VM download page to retrieve a supported version in order to set up the customer-owned and managed VM that will host Riverbed Edge.

7. Select the Edge and click on the **...** menu. Choose **Download cloud-init** to obtain the necessary zip file. Deploy a VM using the provided OVA file and the contents of the cloud-init zip file (specifically, the user-data file).

   a. The recommended instance type requires the following resources: RAM: 16GB, vCPU: 4 cores, and Storage 300GB. (For example, t2.xlarge.)

   b. After configuration, click **Launch through EC2** in AWS. AWS will prompt you about “Selecting an existing key pair or create a new key pair”. Here, select “Proceed without a key pair” to complete the deployment process.

   c. While launching the instance, under Advanced Details, paste the content of the user-data file (included in the cloud-init zip file) into the User Data field.
   Upon first boot, the initial configuration will be set. (*This may take a few minutes; wait for at least five minutes before attempting log in to the VM.*)

8. If you use a firewall with a domain whitelist or a list of blocked ports, make sure to modify it to whitelist DNS names and open ports for the deployed Edge as indicated in Firewall Rules.
9. Check the Riverbed Edge instance's health:
    - Verify that the *Status* for the Edge is green on the Data Sources page. This page does not refresh automatically, so refresh it explicitly if you are watching for a status change.
    - Contact Riverbed Technology in case of any errors or issues
10. Select the Edge in the Data Sources page and click **...**, then click **Connect a Data Source** to display the Connect a Data Source wizard.
11. Choose the data source type and specify the IP address or hostname of the data source, as well as the protocol (HTTPS is the default) and port by which to communicate, then click **Submit**.
12. Click the **...** menu for the data source and choose **Set Credentials** to display the Set Credentials dialog. Type the username and password for accessing the data source, and click **Apply**. The credentials are stored with the Edge.
13. Again, for the data source, choose **Edit** from the **...** menu. Specify the Edge VM IP address and enable the data source. With enabling, the Edge is connected to the data source and ready to push data to Riverbed IQ.
14. Select the Edge in the Data Sources page and click the arrowhead to its left; this will show the status of connected data sources. The web UI should show green status for the Edge and connected data sources.
15. **Note:** See Troubleshooting Riverbed Edge Status for information about identifying and correcting problems with connecting to Riverbed Edge

Configuring Riverbed Edge on an Azure VM

Riverbed IQ's data sources are network monitoring applications such as Riverbed Technology's Riverbed NetIM and Riverbed NetProfiler, which forward observability data to Riverbed IQ for correlation and analysis via Riverbed Edge, which handles data source connections for IQ. Each data source must be configured explicitly to communicate with Edge.

Follow these steps to configure an Riverbed Edge instance and a data source:

1. Create new non-admin user accounts on your Riverbed Technology data sources. For NetProfiler, use the role *Operator*. For NetIM, use the role *ALLUVIO_IQ*. For AppResponse, use a non-admin role with permissions for Feature Settings and Definitions set to Full Control.
2. In the Riverbed IQ web UI, open the Data Sources page and click **Add an Edge** to display the Add an Edge wizard.
3. Specify the Riverbed Edge instance name and virtual machine type, and click **Next**.
4. If necessary, specify the connection type for the Edge:
    - If the Edge does not have access to the Internet, specify a proxy to use and enable **Use Websockets**.
    - If the Edge has only port 443 open to the Internet, enable **Use Websockets**.
    - If neither condition applies, simply click **Next**.
5. Inspect the summary of the Edge definition and click **Submit** to make it manageable by IQ. IQ shows you a brief summary of the next steps you need to execute.
   Note: Riverbed IQ employs a shared ownership model in which Riverbed supplies an Edge image for deployment on top of a customer-owned and

managed VM.

   a. Download the Edge cloud-init file using the provided external link URL.

   b. Go to the external, official Ubuntu VM download page to retrieve a supported version to set up the customer-owned and managed VM that will host Riverbed Edge.

   c. Deploy and prepare your VM for Riverbed Edge.

   d. Use the Edge cloud-init file to configure your target VM environment for Edge.

6. Select the Edge in the Data Sources page and click on the **...** menu. Choose **Obtain VM Image** to open the official Ubuntu VM download page to retrieve a supported version in order to set up the customer-owned and managed VM that will host Riverbed Edge.

7. Select the Edge and click on the **...** menu. Choose **Download cloud-init** to obtain the necessary zip file. Deploy a VM using the provided OVA file and the contents of the cloud-init zip file (specifically, the user-data file).

   a. The recommended instance type requires the following resources: RAM: 16GB, vCPU: 4 cores, and Storage 300GB.

   b. While configuring the instance, under Advanced Details, paste the content of the user-data file (included in the cloud-init zip file) into the Custom Data field.

   c. After configuration, click Create.
   Upon first boot, the initial configuration will be set. (*This may take a few minutes; wait for at least five minutes before attempting log in to the VM.*)

8. If you use a firewall with a domain whitelist or a list of blocked ports, make sure to modify it to whitelist DNS names and open ports for the deployed Edge as indicated in Firewall Rules.

9. Check the Riverbed Edge instance's health:

- Verify that the *Status* for the Edge is green on the Data Sources page. This page does not refresh automatically, so refresh it explicitly if you are watching for a status change.
- Contact Riverbed Technology in case of any errors or issues

10. Select the Edge in the Data Sources page and click **...**, then click **Connect a Data Source** to display the Connect a Data Source wizard.
11. Choose the data source type and specify the IP address or hostname of the data source, as well as the protocol (HTTPS is the default) and port by which to communicate, then click **Submit**.
12. Click the **...** menu for the data source and choose **Set Credentials** to display the Set Credentials dialog. Type the username and password for accessing the data source, and click **Apply**. The credentials are stored with the Edge.
13. Again, for the data source, choose **Edit** from the **...** menu. Specify the Edge VM IP address and enable the data source. With enabling, the Edge is connected to the data source and ready to push data to Riverbed IQ.
14. Select the Edge in the Data Sources page and click the arrowhead to its left; this will show the status of connected data sources. The web UI should show green status for the Edge and connected data sources.

Note: See Troubleshooting Riverbed Edge Status for information about identifying and correcting problems with connecting to Riverbed Edge

Configuring Riverbed Edge on an ESXi VM

Riverbed IQ's data sources are network monitoring applications such as Riverbed Technology's Riverbed NetIM and Riverbed NetProfiler, which forward observability data to Riverbed IQ for correlation and analysis via Riverbed Edge, which handles data source connections for IQ. Each data source must be configured explicitly to communicate with Edge.

Follow these steps to configure an Riverbed Edge instance and a data source:

1. Create new non-admin user accounts on your Riverbed Technology data sources. For NetProfiler, use the role *Operator*. For NetIM, use the role *ALLUVIO_IQ*. For AppResponse, use a non-admin role with permissions for Feature Settings and Definitions set to Full Control.
2. In the Riverbed IQ web UI, open the Data Sources page and click **Add an Edge** to display the Add an Edge wizard.
3. Specify the Riverbed Edge instance name and virtual machine type, and click **Next**.
4. If necessary, specify the connection type for the Edge:
    - If the Edge does not have access to the Internet, specify a proxy to use and enable **Use Websockets**.
    - If the Edge has only port 443 open to the Internet, enable **Use Websockets**.
    - If neither condition applies, simply click **Next**.
5. Inspect the summary of the Edge definition and click **Submit** to make it manageable by IQ. IQ shows you a brief summary of the next steps you need to execute.
   Note: Riverbed IQ employs a shared ownership model in which Riverbed supplies an Edge image for deployment on top of a customer-owned and

managed VM.

a. Download the Edge ISO Image using the provided external link URL.

b. Go to the official Ubuntu VM download page to retrieve a supported version in order to set up the customer-owned and managed VM that will host Riverbed Edge.

c. Deploy and prepare your VM for Riverbed Edge

d. Load the Edge ISO image file on top of your target VM environment.

6. Select the Edge in the Data Sources page and click on the **...** menu. Choose **Obtain VM Image** to open the official Ubuntu VM download page to retrieve a supported version in order to set up the customer-owned and managed VM that will host Riverbed Edge

7. Select the Edge and click on the **...** menu. Choose **Download ISO** to obtain the necessary ISO file. Deploy a VM using the provided OVA file and the ISO image you acquired.

   a. Recommended specification: (RAM: 16GB, vCPU: 4 cores, and Storage 300GB)

   b. Deploy OVA using provided OVA file - DO NOT power on automatically.

   c. Mount the ISO (seed.iso) created earlier

      On VMware, this can be added using CD/DVD drive > iso image

   d. Power ON the VM

      Upon first boot, the initial configuration will be set. (*This may take a few minutes; wait for at least five minutes before attempting log in to the VM.*)

8. If you use a firewall with a domain whitelist or a list of blocked ports, make sure to modify it to whitelist DNS names and open ports for the deployed Edge as indicated in Firewall Rules.

9. Check the Riverbed Edge instance's health:
    - Verify that the *Status* for the Edge is green on the Data Sources page. This page does not refresh automatically, so refresh it explicitly if you are watching for a status change.
    - Contact Riverbed Technology in case of any errors or issues
10. Select the Edge in the Data Sources page and click **...**, then click **Connect a Data Source** to display the Connect a Data Source wizard.
11. Choose the data source type and specify the IP address or hostname of the data source, as well as the protocol (HTTPS is the default) and port by which to communicate, then click **Submit**.
12. Click the **...** menu for the data source and choose **Set Credentials** to display the Set Credentials dialog. Type the username and password for accessing the data source, and click **Apply**. The credentials are stored with the Edge.
13. Again, for the data source, choose **Edit** from the **...** menu. Specify the Edge VM IP address and enable the data source. With enabling, the Edge is connected to the data source and ready to push data to Riverbed IQ.
14. Select the Edge in the Data Sources page and click the arrowhead to its left; this will show the status of connected data sources. The web UI should show green status for the Edge and connected data sources.

Note: See Troubleshooting Riverbed Edge Status for information about identifying and correcting problems with connecting to Riverbed Edge

Firewall Rules

Riverbed Edge requires the following inbound and outbound network access. It does not require any firewall inbound access from the public Internet to your corporate network.

Note: We discourage whitelisting any of the individual IP addresses for the DNS targets below, as Microsoft Azure can change the IP addresses behind those records without prior notice! Azure publishes a list of all currently used IP addresses that can be narrowed down further by region (Riverbed IQ's region is "useast") and service type (IoT Hub, Container Registry): Azure IP Ranges and Service Tags – Public Cloud.

Service/Protocol	From	To	DNS Targets	Port	Description
HTTPS	Riverbed Edge (LAN)	NetProfiler (LAN) AppResponse (LAN)		443	
HTTPS	Riverbed Edge (LAN)	NetIM (LAN)		8543	
HTTP	NetProfiler (LAN) NetIM (LAN)	Riverbed Edge (LAN)		11002	
HTTPS	Riverbed Edge (LAN)	Internet (WAN)	mcr.microsoft.com packages.microsoft.com *.azurecr.io	443	

			*.azure-devices.net *.blob.-core.windows.net *.docker.io *.ser-vicebus.windows.net		
AMQP over TLS	Riverbed Edge (LAN)	Internet (WAN)	*.ser-vicebus.windows.net	5671 5672 Not requir-ed if an HTTP proxy is used.	(default config)
AMQP over Websockets	Riverbed Edge (LAN)	Internet (WAN)		443	Used only if you have con-figured the con-fig.toml of iot edge to use upstream protocol AmqpWs.

Refer to this step in Configuring Riverbed IQ and Data Sources to see how this information is used during the Riverbed Edge configuration process.

Troubleshooting Riverbed Edge Status

In the event that you experience problems with connecting your Riverbed Edge instance as described in Configuring Riverbed Edge and Data Sources:

- A Riverbed Edge instance shows red status on the Data Sources page.
- There's no discernible evidence of data being pushed to Azure blob storage.

A Riverbed Edge instance shows red status on the Data Sources page

- Make certain that you have whitelisted DNS names and opened required ports as detailed in Firewall Rules. Having these set incorrectly is the most common cause of Riverbed Edge not coming up completely and correctly.
- ssh to Riverbed Edge using the default username and password.
    - Run the following commands and collect the corresponding outputs:
        - *iotedge check* (see an example of this later on this page)
        - *cloud-init analyze boot*
    - Collect the following files and outputs:
        - /var/log/cloud-init.log
        - /var/log/cloud-init-output.log
        - /etc/aziot/config.toml

There's no discernible evidence of data being pushed to Azure blob storage

Riverbed Edge has partial connectivity: the Data Sources page shows its status as green, but no data is being collected from data sources.

- Make certain that you have whitelisted DNS names and opened required ports as detailed in Firewall Rules. Having these set incorrectly is the most common cause of Riverbed Edge not coming up completely and correctly.
- Run *iotedge check* and **docker ps** and collect the corresponding outputs. (See an example of *iotedge check* output below). Most likely, one of the DNS names is not whitelisted or one of the ports is not open.
    - Verify that port 11002 is open from the point product. This port lets point products push data to the Azure blob.
    - Verify that ports 443 (NetProfiler/AppResponse) and 8543(NetIM) are open for on-demand calls from the Riverbed IQ service.

iotedge check Output Example

An example of typical *iotedge check* output follows:

```
Configuration checks (aziot-identity-service)
---------------------------------------------
√ keyd configuration is well-formed - OK
√ certd configuration is well-formed - OK
√ tpmd configuration is well-formed - OK
√ identityd configuration is well-formed - OK
√ daemon configurations up-to-date with config.toml - OK
√ identityd config toml file specifies a valid hostname - OK
‼ aziot-identity-service package is up-to-date - Warning
    Installed aziot-identity-service package has version 1.2.5 but 1.3.0 is the
latest stable version available.
    Please see https://aka.ms/aziot-update-runtime for update instructions.
‼ host time is close to reference time - Warning
    Time on the device is out of sync with the NTP server. This may cause prob-
lems connecting to IoT Hub.
    Please ensure time on device is accurate, for example by installing an NTP
daemon.
√ preloaded certificates are valid - OK
√ keyd is running - OK
√ certd is running - OK
√ identityd is running - OK
√ read all preloaded certificates from the Certificates Service - OK
√ read all preloaded key pairs from the Keys Service - OK
√ ensure all preloaded certificates match preloaded private keys with the same ID
- OK

Connectivity checks (aziot-identity-service)
--------------------------------------------
√ host can connect to and perform TLS handshake with iothub AMQP port - OK
√ host can connect to and perform TLS handshake with iothub HTTPS / WebSockets
port - OK
√ host can connect to and perform TLS handshake with iothub MQTT port - OK
```

```
Configuration checks
--------------------
√ aziot-edged configuration is well-formed - OK
√ configuration up-to-date with config.toml - OK
√ container engine is installed and functional - OK
√ configuration has correct URIs for daemon mgmt endpoint - OK
‼ aziot-edge package is up-to-date - Warning
    Installed IoT Edge daemon has version 1.2.7 but 1.3.0 is the latest stable
version available.
    Please see https://aka.ms/iotedge-update-runtime for update instructions.
√ container time is close to host time - OK
‼ DNS server - Warning
    Container engine is not configured with DNS server setting, which may impact
connectivity to IoT Hub.
    Please see https://aka.ms/iotedge-prod-checklist-dns for best practices.
    You can ignore this warning if you are setting DNS server per module in the
Edge deployment.
√ production readiness: logs policy - OK
‼ production readiness: Edge Agent's storage directory is persisted on the host
filesystem - Warning
    The edgeAgent module is not configured to persist its /tmp/edgeAgent dir-
ectory on the host filesystem.
    Data might be lost if the module is deleted or updated.
    Please see https://aka.ms/iotedge-storage-host for best practices.
‼ production readiness: Edge Hub's storage directory is persisted on the host
filesystem - Warning
    The edgeHub module is not configured to persist its /tmp/edgeHub directory on
the host filesystem.
    Data might be lost if the module is deleted or updated.
    Please see https://aka.ms/iotedge-storage-host for best practices.
√ Agent image is valid and can be pulled from upstream - OK
√ proxy settings are consistent in aziot-edged, aziot-identityd, moby daemon and
config.toml - OK

Connectivity checks
-------------------
√ container on the default network can connect to upstream  AMQP port - OK
√ container on the default network can connect to upstream HTTPS / WebSockets
port - OK
√ container on the default network can connect to upstream MQTT port - OK
√ container on the IoT Edge module network can connect to upstream AMQP port - OK
√ container on the IoT Edge module network can connect to upstream HTTPS /
WebSockets port - OK
√ container on the IoT Edge module network can connect to upstream MQTT port - OK
30 check(s) succeeded.
6 check(s) raised warnings. Re-run with --verbose for more details.
```

Adding a Data Source

Before you add a data source to Riverbed Edge, make sure it is configured appropriately; each data source type requires specific configuration in its own UI to be able to integrate with Riverbed IQ. Refer to these topics as needed:

- Configuring AppResponse
- Configuring NetIM
- Configuring NetProfiler
- Configuring Aternity SaaS

The generic procedure for adding a data source to a Riverbed Edge is:

1. In the Data Sources page, select the Edge and click **...** and click **Connect a Data Source** to display the Connect a Data Source dialog.
2. Choose the Source Type: **AppResponse**, **Aternity**, **NetIM**, or **NetProfiler**.
3. Type the data source's Hostname or IP Address.
4. Specify the data source's Protocol and Port.
5. Type a name for the data source.
6. Click **Submit** to complete, or **Close** to quit without making any change. The data source is listed under its associated Riverbed Edge on the Data Sources page.
7. Add the user account credentials for the data source to the Edge. Click the **...** menu for the data source and choose **Set Credentials** to display the Set Credentials dialog. Type the username and password for accessing the data source, and click **Apply**.
8. On the Data Sources page, select the data source under its Edge, click **...** and click **Edit**. Edit the data source and enable it, and specify the Edge IP address or hostname. Click **Submit** to complete the configuration.

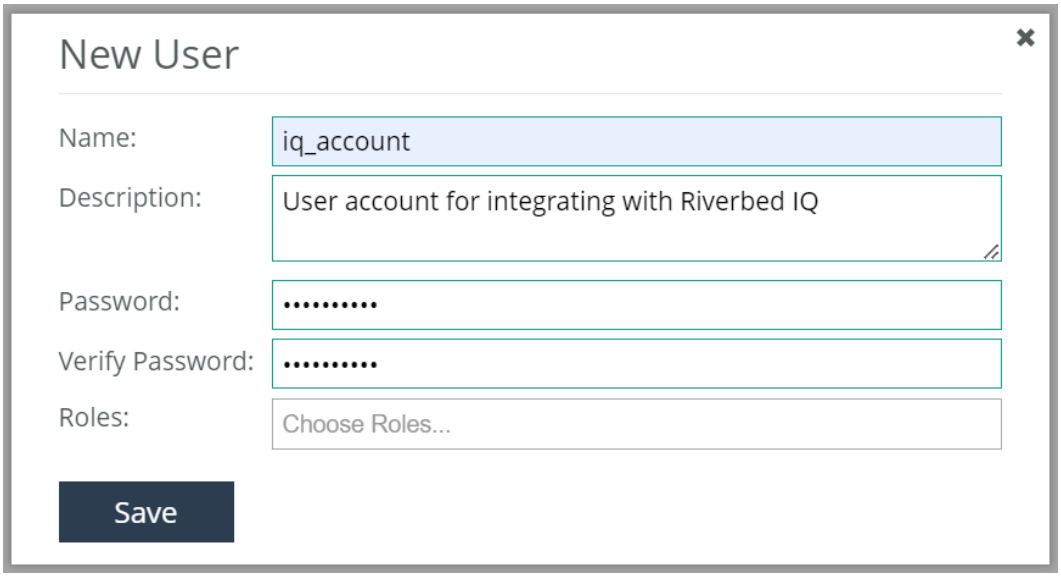
If you see error messages or observe other problems related to your data source connections, refer to Troubleshooting Data Source Connections for guidance.

Configuring Riverbed AppResponse

This topic describes how to configure Riverbed AppResponse to communicate with Riverbed Edge. Refer to Configuring Riverbed Edge and Data Sources for instructions for configuring and deploying Riverbed Edge.

Prior to configuring Riverbed IQ to connect to an AppResponse data source, you need to configure the AppResponse system. For each AppResponse system, use the AppResponse web UI:

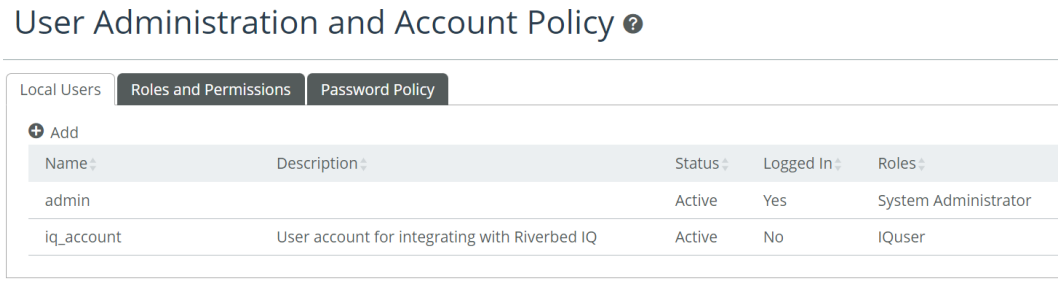
1. Click **Administration** > **User Administration** and create a new local user. Specify whatever username you want, but consider making it descriptive so there's never any question about its purpose. This example shows "iq_account" as the user name.



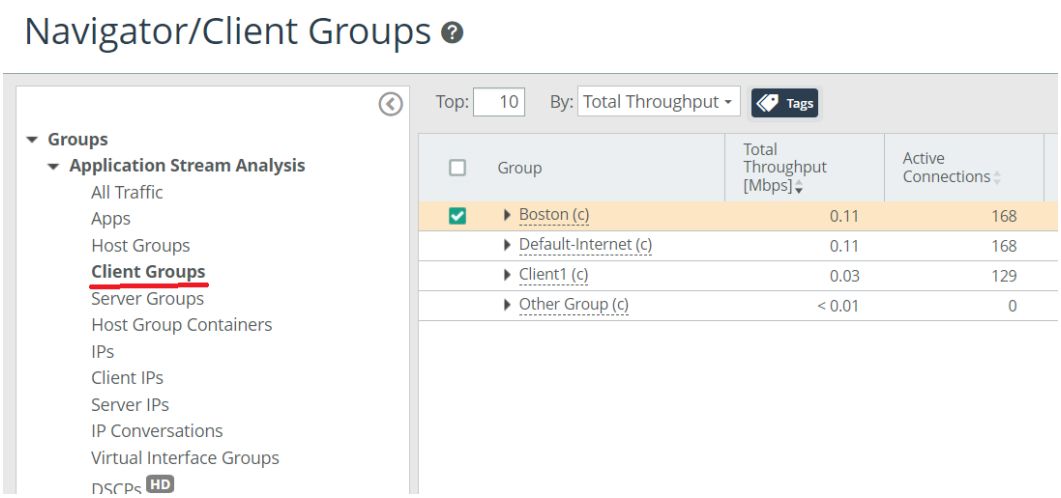
2. Open the Roles and Permissions tab and click **Add** to create a role to define the permissions needed for integrating with Riverbed IQ. The permissions for Feature Settings and Definitions must be set to **Full Control**. Other permissions are not neccessary, and can be left set to **No Access**.

Permission	No Access	View Only	Full Control
System configuration Configuration settings in Administration menu	◉	○	○
User Management User account, roles and password policy	◉	○	○
Feature Settings and Definitions Feature Settings sub-menu and definitions	○	○	◉
Access Packets / Live Views Packet export/download, apply PA+ live views	◉	○	○
Configure Trace Clips Trace clip configuration	◉	○	○
Access Other Users' Objects Read/Edit other users' private Insights, reports, schedules etc.	◉	○	○

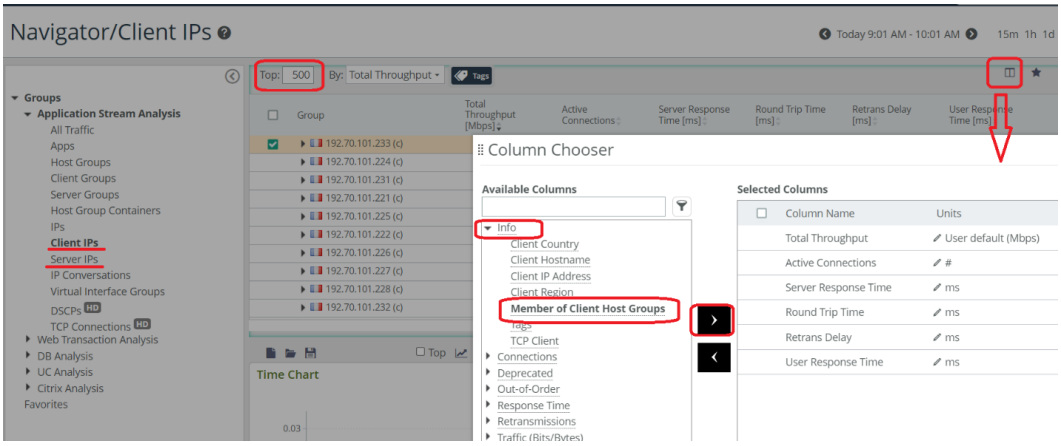
3. Assign that role to the user account you just created ("iq_account" in this example).



4. You will need to create the user account on Riverbed Edge, also. [The account needs to be setup on the Riverbed Edge, as is done for NetProfiler.]
5. A successful integration with Riverbed IQ requires Host Group definitions to be configured and working. If there are ungrouped hosts in your network, Riverbed IQ is not monitoring the ungrouped traffic, and you may be missing important incidents.
6. In the AppResponse UI, go to Definitions > Host Group and review the host groups. IQ monitors how applications are performing for each configured client host group. To ensure the definition is complete, go to Navigator > Client Groups and ensure that the groups are reporting traffic data.



7. To verify that the groups are complete, you can also go to Navigator > Client IPs and make sure all IPs are reported within a group. Click on Navigator > Client IPs, then change the top to 500 (500 is the maximum you can look at), then click on the column chooser icon on the top right, and, under the Info section, select the Member of Client Host Groups column and add it:



8. You can then click on the Member of Client Host Groups column header to sort on it and verify that there are no hosts that do not belong to groups, e.g., in this example, there are two ungrouped IPv6 hosts:

Group	Total Throughput [Mbps]	Member of Client Host Groups	Active Connections
fe80::20c:29ff:feb7:9ae9 (c)	< 0.01		0
fe80::20c:29ff:fee1:2ec3 (c)	< 0.01		0
192.70.1.234 (c)	< 0.01	Default-Internet, Boston	4
192.70.1.240 (c)	< 0.01	Default-Internet, Boston	3
192.70.1.237 (c)	< 0.01	Default-Internet, Boston	2
192.70.1.239 (c)	< 0.01	Default-Internet, Boston	3
192.70.1.231 (c)	< 0.01	Default-Internet, Boston	2
192.70.1.229 (c)	< 0.01	Default-Internet, Boston	3
192.70.1.232 (c)	< 0.01	Default-Internet, Boston	2
192.70.1.238 (c)	< 0.01	Default-Internet, Boston	3

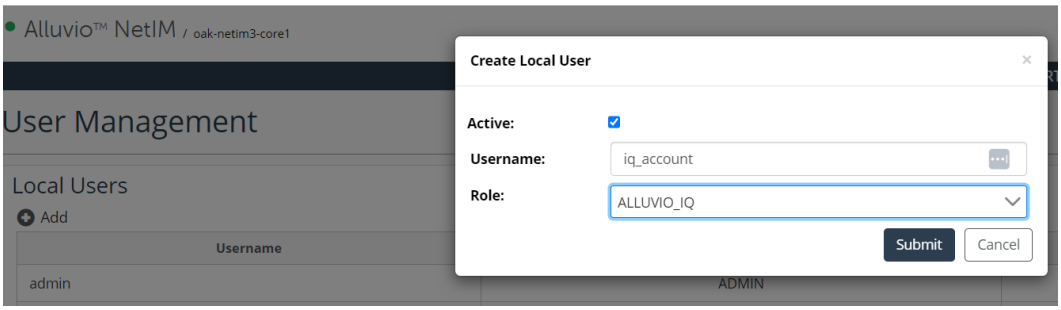
9. Go to Definitions > Host Groups and include those hosts in group definitions.
10. Repeat the same process for Navigator > Server IPs.
11. Once the host group definitions are changed, allow 5 to 10 minutes for the AppResponse definition to take effect, and verify in the Navigator that the IPs are now showing in groups.
12. In Riverbed IQ, create and configure each AppResponse as a data source.

Configuring Riverbed NetIM

This topic describes how to configure Riverbed NetIM to communicate with Riverbed Edge. Refer to Configuring Riverbed Edge and Data Sources for instructions for configuring and deploying Riverbed Edge.

Prior to configuring Riverbed IQ to connect to a NetIM data source, you need to configure the NetIM system. For each NetIM system, use the NetIM web UI:

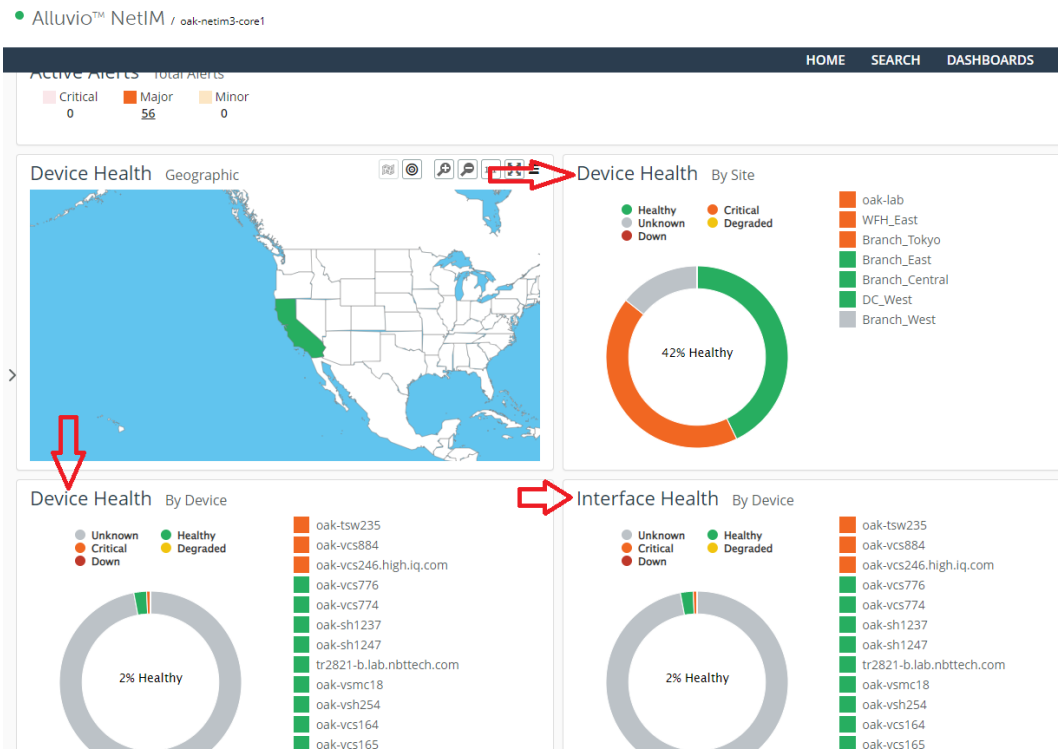
1. Create a new local user that has a role of ALLUVIO_IQ:



   NetIM release 2.8.0 ships with default password aging policy of 90 days. If the password is expired, change it and follow this step to add the credentials of the data source.

   Follow this support KB article:Extending User Account Password Expiration Period In NetIM 2.8 (https://supportkb.riverbed.com/support/index?page=content&id=S38111).

2. Log in to the NetIM UI and ensure that the Device Health By Site, Device Health By Device, and Interface Health widgets are populated. If not, go to the Configure menu and configure devices for polling.



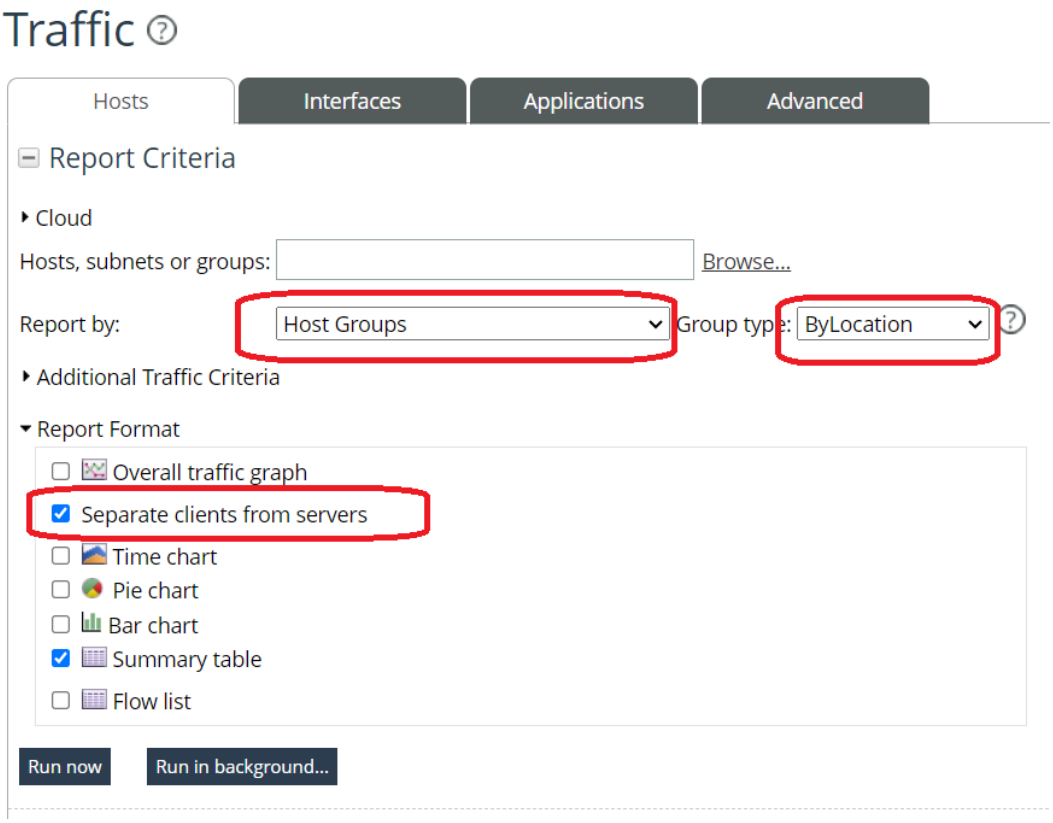
3. In Riverbed IQ, create and configure each NetIM as a data source.

Configuring Riverbed NetProfiler

This topic describes how to configure Riverbed NetProfiler to communicate with Riverbed Edge. Refer to Configuring Riverbed Edge and Data Sources for instructions for configuring and deploying Riverbed Edge.

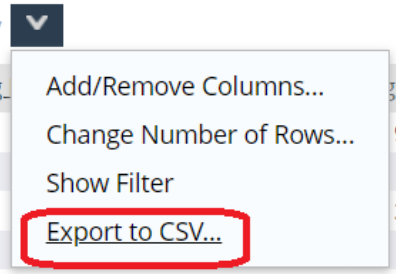
1. Go to the Riverbed Technology support site to install the latest version of NetProfiler: GA 10.24.2.
2. Create a new local user that has a role of operator.
3. Configure Host Groups on NetProfiler. A successful integration with Riverbed IQ requires the host group definitions to be configured and working.

   In the NetProfiler UI, go to Definitions > Host Group Types and review the ByLocation host groups. Riverbed IQ is monitoring how applications are performing for each configured ByLocation group. To ensure the definition is complete, go to Reports > Traffic and run a host groups report:

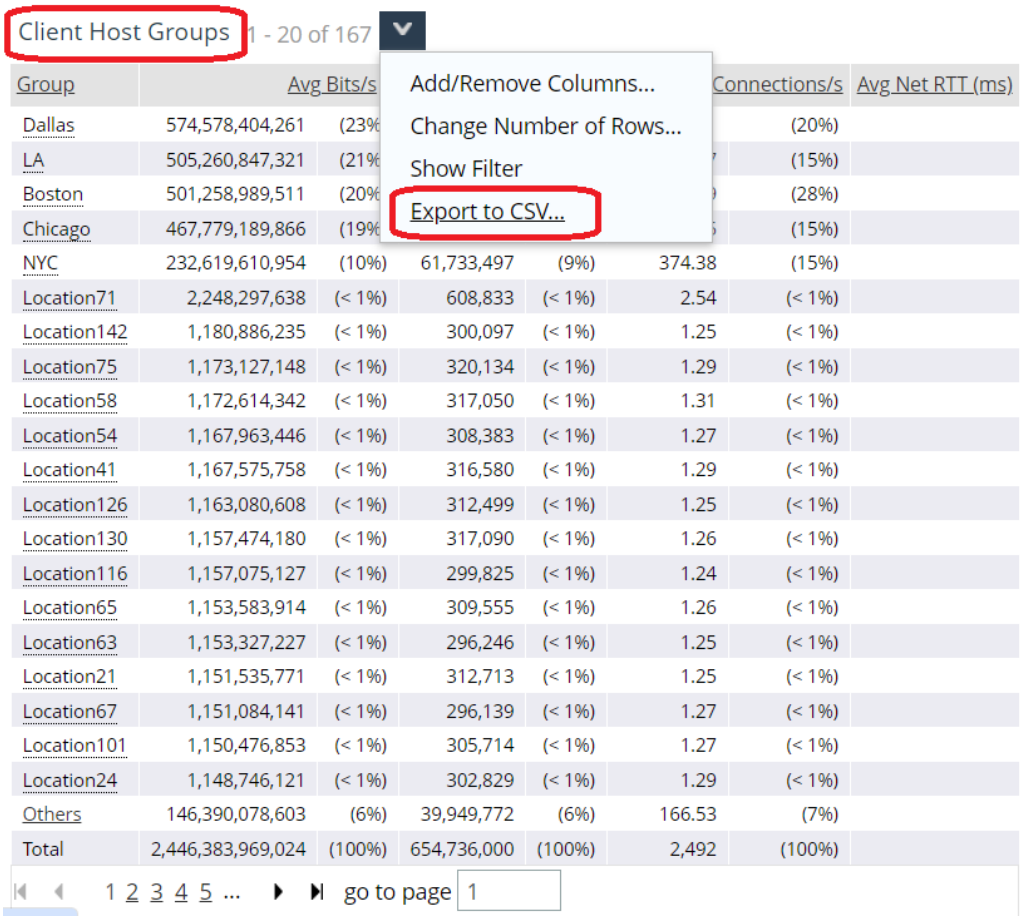


Check that the server and client groups are returned, and the results are as expected. To do so, export the Server Host groups table and the Client Host groups table to CSV, and check the resulting files.

Server Host Groups 1 - 17 of 17



Group	Avg_				Active Connections/s	
Philadelphia	1,186,944,560,880				991.42	(40%)
Miami	846,739,591,448				1,119	(45%)
SF	406,305,768,173				371.26	(15%)
Boston	4,977,872,723				6.05	(< 1%)
Dallas	380,676,281	(< 1%)	96,253	(< 1%)	< 1	(< 0.01%)
Chicago	213,801,603	(< 0.01%)	47,440	(< 0.01%)	< 1	(< 0.01%)
NYC	207,776,279	(< 0.01%)	49,318	(< 0.01%)	< 1	(< 0.01%)
LA	169,108,589	(< 0.01%)	51,851	(< 0.01%)	< 1	(< 0.01%)
Location3	57,738,386	(< 0.01%)	14,713	(< 0.01%)	< 1	(< 0.01%)
DC_West	13,004,576	(< 0.01%)	1,126	(< 0.01%)	< 1	(< 0.01%)
Fresno	8,133,662	(< 0.01%)	1,017	(< 0.01%)	< 1	(< 0.01%)
Durham	6,852,197	(< 0.01%)	16,831	(< 0.01%)	< 1	(< 0.01%)
Location99	2,821,540	(< 0.01%)	2,009	(< 0.01%)	< 1	(< 0.01%)
Branch_Central	2,108,056	(< 0.01%)	1,150	(< 0.01%)	< 1	(< 1%)
Branch_Tokyo	3,139	(< 0.01%)	1.95	(< 0.01%)	< 1	(< 0.01%)
Branch_West	156.80	(< 0.01%)	< 1	(< 0.01%)	< 1	(< 0.01%)
WFH_East	139.48	(< 0.01%)	< 1	(< 0.01%)	< 1	(< 0.01%)
Others	354,151,195	(< 1%)	42,445	(< 0.01%)	3.15	(< 1%)
Total	2,446,383,969,024	(100%)	654,736,000	(100%)	2,492	(100%)

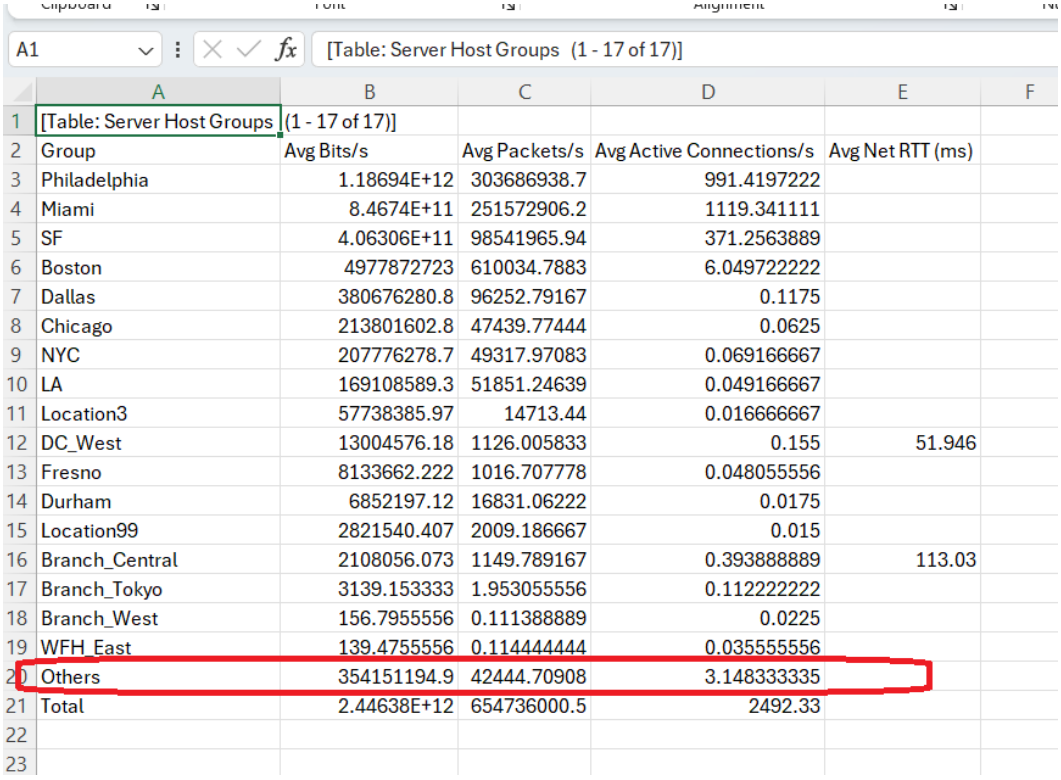


Group	Avg Bits/s					Connections/s	Avg Net RTT (ms)
Dallas	574,578,404,261	(23%				(20%)	
LA	505,260,847,321	(21%				(15%)	
Boston	501,258,989,511	(20%				(28%)	
Chicago	467,779,189,866	(19%				(15%)	
NYC	232,619,610,954	(10%)	61,733,497	(9%)	374.38	(15%)	
Location71	2,248,297,638	(< 1%)	608,833	(< 1%)	2.54	(< 1%)	
Location142	1,180,886,235	(< 1%)	300,097	(< 1%)	1.25	(< 1%)	
Location75	1,173,127,148	(< 1%)	320,134	(< 1%)	1.29	(< 1%)	
Location58	1,172,614,342	(< 1%)	317,050	(< 1%)	1.31	(< 1%)	
Location54	1,167,963,446	(< 1%)	308,383	(< 1%)	1.27	(< 1%)	
Location41	1,167,575,758	(< 1%)	316,580	(< 1%)	1.29	(< 1%)	
Location126	1,163,080,608	(< 1%)	312,499	(< 1%)	1.25	(< 1%)	
Location130	1,157,474,180	(< 1%)	317,090	(< 1%)	1.26	(< 1%)	
Location116	1,157,075,127	(< 1%)	299,825	(< 1%)	1.24	(< 1%)	
Location65	1,153,583,914	(< 1%)	309,555	(< 1%)	1.26	(< 1%)	
Location63	1,153,327,227	(< 1%)	296,246	(< 1%)	1.25	(< 1%)	
Location21	1,151,535,771	(< 1%)	312,713	(< 1%)	1.25	(< 1%)	
Location67	1,151,084,141	(< 1%)	296,139	(< 1%)	1.27	(< 1%)	
Location101	1,150,476,853	(< 1%)	305,714	(< 1%)	1.27	(< 1%)	
Location24	1,148,746,121	(< 1%)	302,829	(< 1%)	1.29	(< 1%)	
Others	146,390,078,603	(6%)	39,949,772	(6%)	166.53	(7%)	
Total	2,446,383,969,024	(100%)	654,736,000	(100%)	2,492	(100%)	

1 2 3 4 5 ... go to page 1

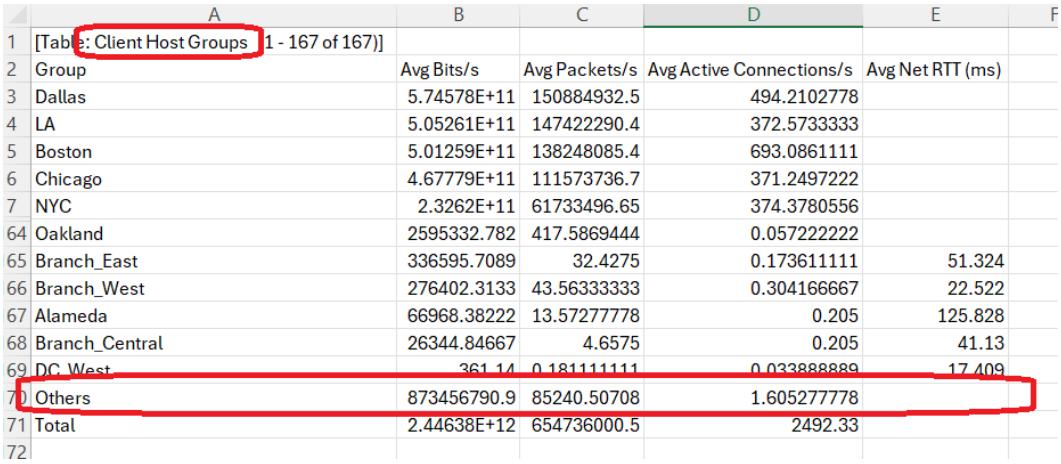
The CSV lists all host groups and their traffic. All groups summed up together add up to 100% of the traffic; if there are ungrouped hosts in your network, that traffic will show up in the Others row. The Others row represents ungrouped data; Riverbed IQ is not monitoring the ungrouped traffic and you may be missing important incidents. If the Others row in the CSV is present and has a large value, go to Definitions > Host Group Types and edit the ByLocation definitions. Then,re-run the report, export to CSV, and ensure that the Others group traffic is reduced.

An example CSV export for Server host groups:



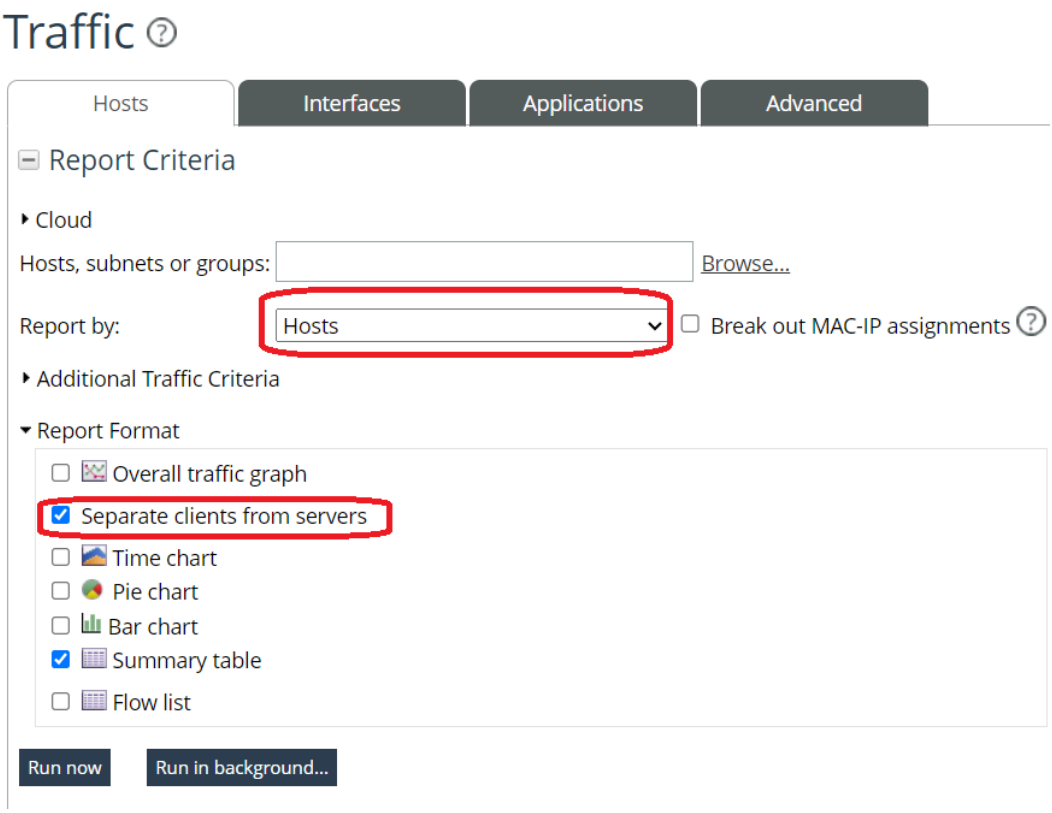
	A	B	C	D	E	F
1	[Table: Server Host Groups	(1 - 17 of 17)]				
2	Group	Avg Bits/s	Avg Packets/s	Avg Active Connections/s	Avg Net RTT (ms)	
3	Philadelphia	1.18694E+12	303686938.7	991.4197222		
4	Miami	8.4674E+11	251572906.2	1119.341111		
5	SF	4.06306E+11	98541965.94	371.2563889		
6	Boston	4977872723	610034.7883	6.049722222		
7	Dallas	380676280.8	96252.79167	0.1175		
8	Chicago	213801602.8	47439.77444	0.0625		
9	NYC	207776278.7	49317.97083	0.069166667		
10	LA	169108589.3	51851.24639	0.049166667		
11	Location3	57738385.97	14713.44	0.016666667		
12	DC_West	13004576.18	1126.005833	0.155	51.946	
13	Fresno	8133662.222	1016.707778	0.048055556		
14	Durham	6852197.12	16831.06222	0.0175		
15	Location99	2821540.407	2009.186667	0.015		
16	Branch_Central	2108056.073	1149.789167	0.393888889	113.03	
17	Branch_Tokyo	3139.153333	1.953055556	0.112222222		
18	Branch_West	156.7955556	0.111388889	0.0225		
19	WFH_East	139.4755556	0.114444444	0.035555556		
20	Others	354151194.9	42444.70908	3.148333335		
21	Total	2.44638E+12	654736000.5	2492.33		
22						
23						

An example CSV export for Client host groups:

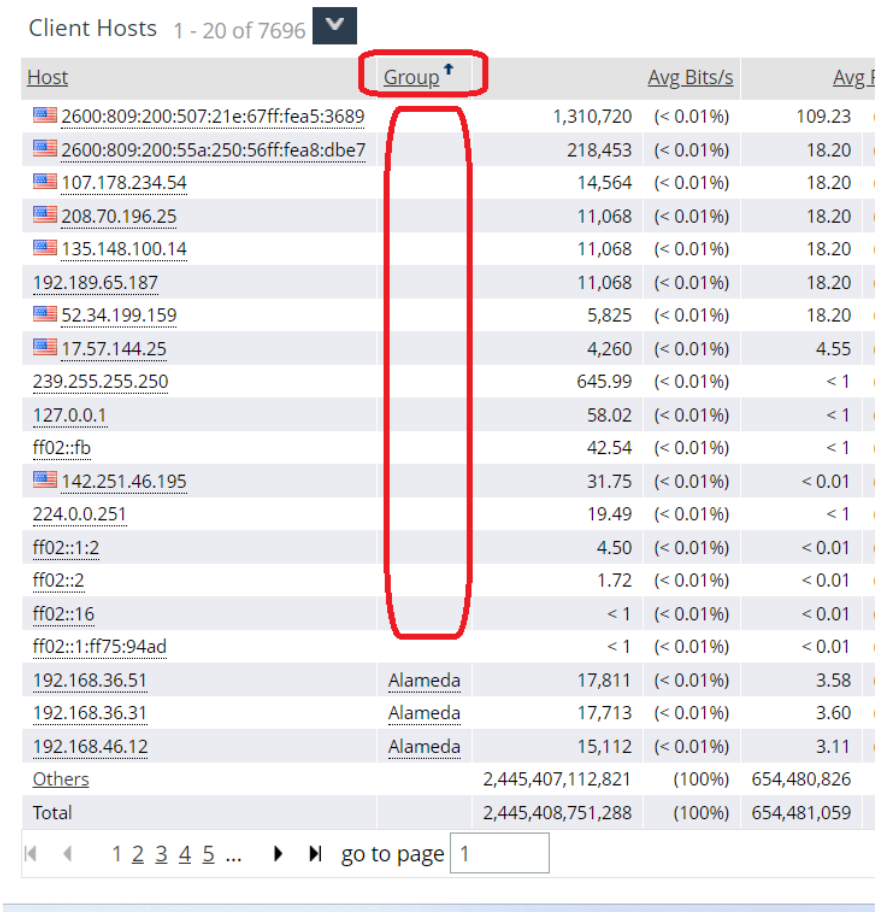


	A	B	C	D	E	F
1	[Table: Client Host Groups	(1 - 167 of 167)]				
2	Group	Avg Bits/s	Avg Packets/s	Avg Active Connections/s	Avg Net RTT (ms)	
3	Dallas	5.74578E+11	150884932.5	494.2102778		
4	LA	5.05261E+11	147422290.4	372.5733333		
5	Boston	5.01259E+11	138248085.4	693.0861111		
6	Chicago	4.67779E+11	111573736.7	371.2497222		
7	NYC	2.3262E+11	61733496.65	374.3780556		
64	Oakland	2595332.782	417.5869444	0.057222222		
65	Branch_East	336595.7089	32.4275	0.173611111	51.324	
66	Branch_West	276402.3133	43.56333333	0.304166667	22.522	
67	Alameda	66968.38222	13.57277778	0.205	125.828	
68	Branch_Central	26344.84667	4.6575	0.205	41.13	
69	DC_West	361.14	0.181111111	0.033888889	17.409	
70	Others	873456790.9	85240.50708	1.605277778		
71	Total	2.44638E+12	654736000.5	2492.33		
72						

A way to discover the ungrouped hosts is to run a Traffic Hosts report and look for hosts that are not assigned to a group:



Sort on the Group column and unassigned hosts will show on top:



Client Hosts 1 - 20 of 7696

Host	Group		Avg Bits/s	Avg F
2600:809:200:507:21e:67ff:fea5:3689		1,310,720	(< 0.01%)	109.23
2600:809:200:55a:250:56ff:fea8:dbe7		218,453	(< 0.01%)	18.20
107.178.234.54		14,564	(< 0.01%)	18.20
208.70.196.25		11,068	(< 0.01%)	18.20
135.148.100.14		11,068	(< 0.01%)	18.20
192.189.65.187		11,068	(< 0.01%)	18.20
52.34.199.159		5,825	(< 0.01%)	18.20
17.57.144.25		4,260	(< 0.01%)	4.55
239.255.255.250		645.99	(< 0.01%)	< 1
127.0.0.1		58.02	(< 0.01%)	< 1
ff02::fb		42.54	(< 0.01%)	< 1
142.251.46.195		31.75	(< 0.01%)	< 0.01
224.0.0.251		19.49	(< 0.01%)	< 1
ff02::1:2		4.50	(< 0.01%)	< 0.01
ff02::2		1.72	(< 0.01%)	< 0.01
ff02::16		< 1	(< 0.01%)	< 0.01
ff02::1:ff75:94ad		< 1	(< 0.01%)	< 0.01
192.168.36.51	Alameda	17,811	(< 0.01%)	3.58
192.168.36.31	Alameda	17,713	(< 0.01%)	3.60
192.168.46.12	Alameda	15,112	(< 0.01%)	3.11
Others		2,445,407,112,821	(100%)	654,480,826
Total		2,445,408,751,288	(100%)	654,481,059

1 2 3 4 5 ... go to page 1

Note that the Traffic report is limited to 10,000 hosts. For larger networks you may not see all hosts, but you will see the top 10,000 talkers on the network.

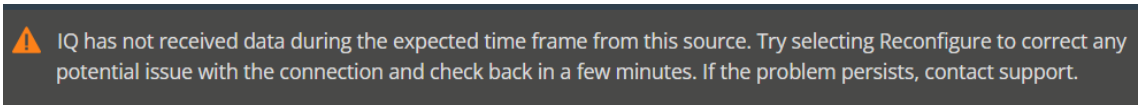
4. In Riverbed IQ, create and configure each NetProfiler as a data source.

Troubleshooting Data Source Connections

This topic describes how to troubleshoot and resolve error messages you may see relative to your data sources:

- IQ has not received data during the expected time frame from this source
- Authentication failure
- The data source has not provided any recent status update

IQ has not received data during the expected time frame from this source



- Instructions for a NetProfiler data source
- Instructions for an AppResponse data source
- Instructions for a NetIM data source

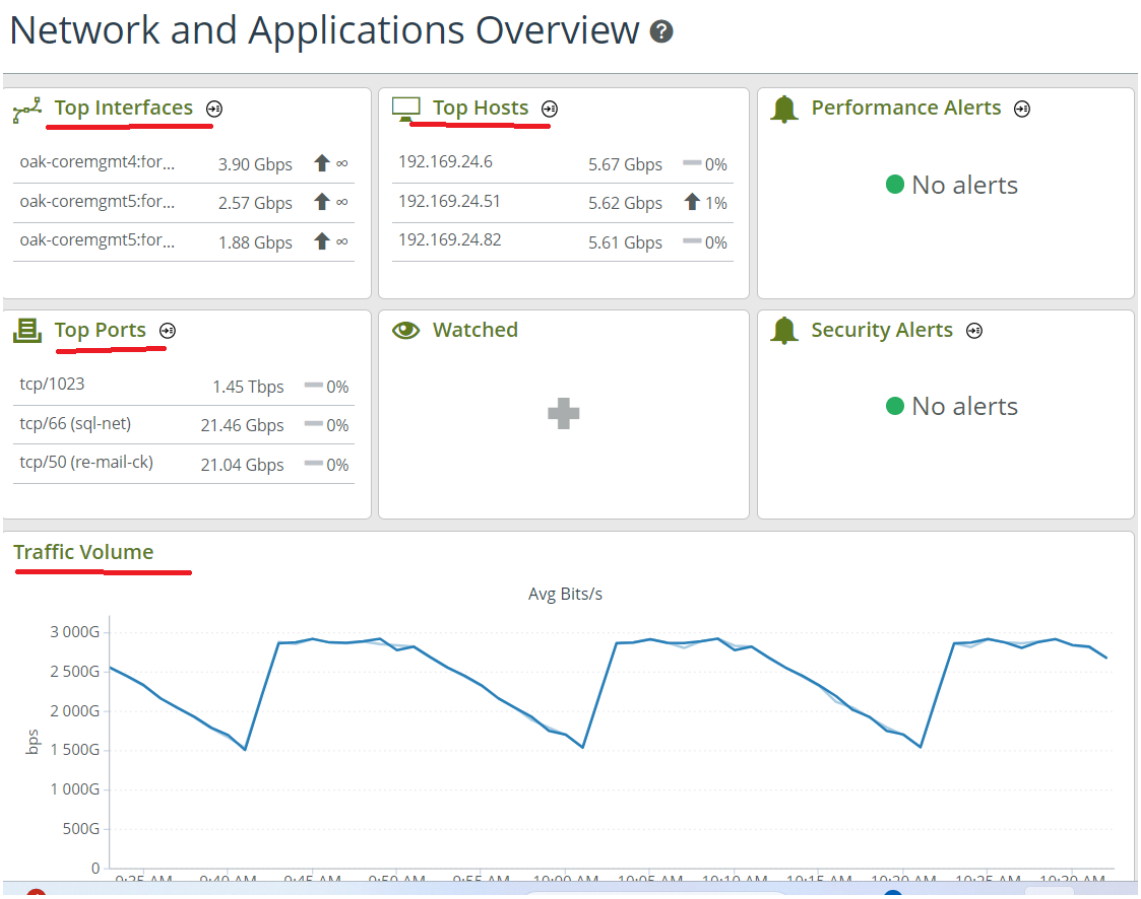
For a NetProfiler data source:

There can be several issues:

- Data is not flowing to NetProfiler
- The clock is not synchronized
- There is missing configuration for host data
- There is missing configuration for interface data

Data is not flowing

Log in to NetProfiler and navigate to the Home screen. Make sure the Top Interfaces, Top Hosts, Top Ports, and Traffic Volume charts are populated:

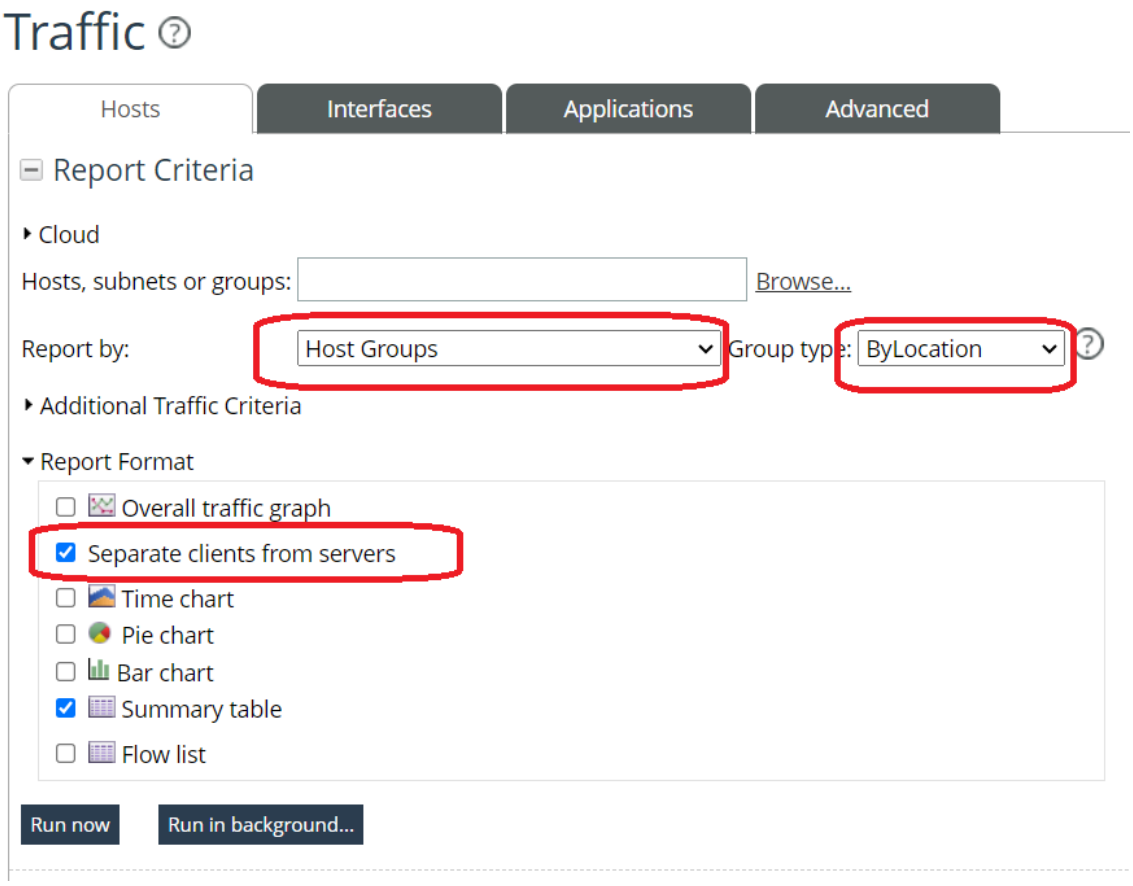


Clock synchronization

Ensure that the NetProfiler clock is synchronized. When you log in to NetProfiler, the date/time is shown in the banner at the top right. If the clock appears to be in error, go to Administration > General Settings and make sure that synchronizing to an external NTP server is set.

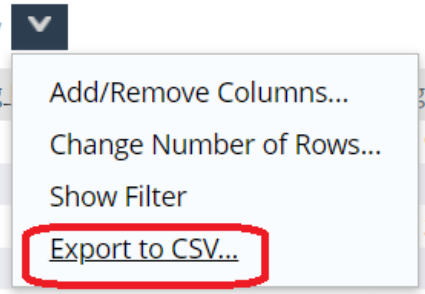
Missing configuration: host data

A successful integration with Riverbed IQ requires the host group definitions to be configured and working. In the NetProfiler UI, go to Definitions > Host Group Types and review the ByLocation host groups. Riverbed IQ is monitoring how applications are performing for each configured ByLocation group. To ensure the definition is complete, go to Reports > Traffic and run a host groups report:

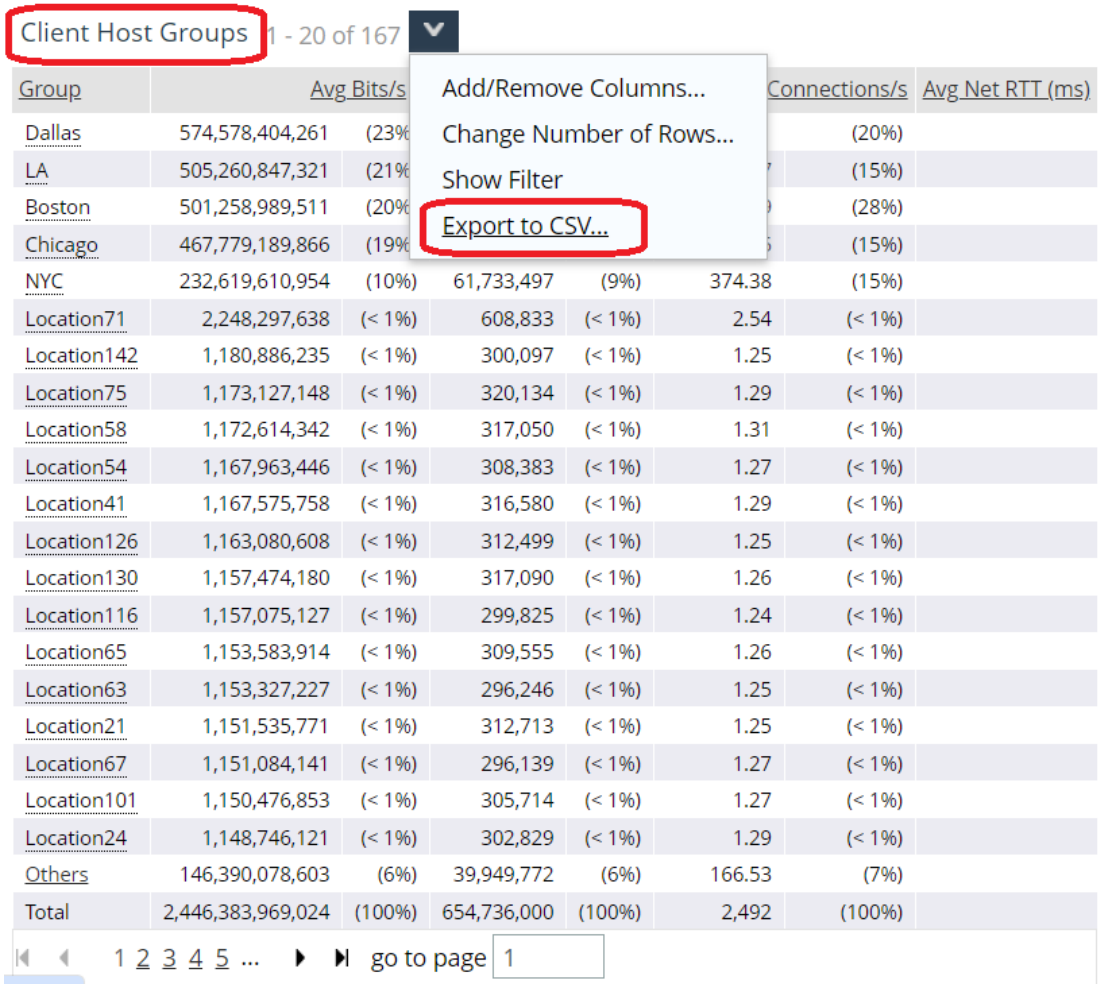


Check that the server and client groups are returned, and the results are as expected. To do so, export the Server Host groups table and the Client Host groups table to CSV, and check the resulting files.

Server Host Groups 1 - 17 of 17

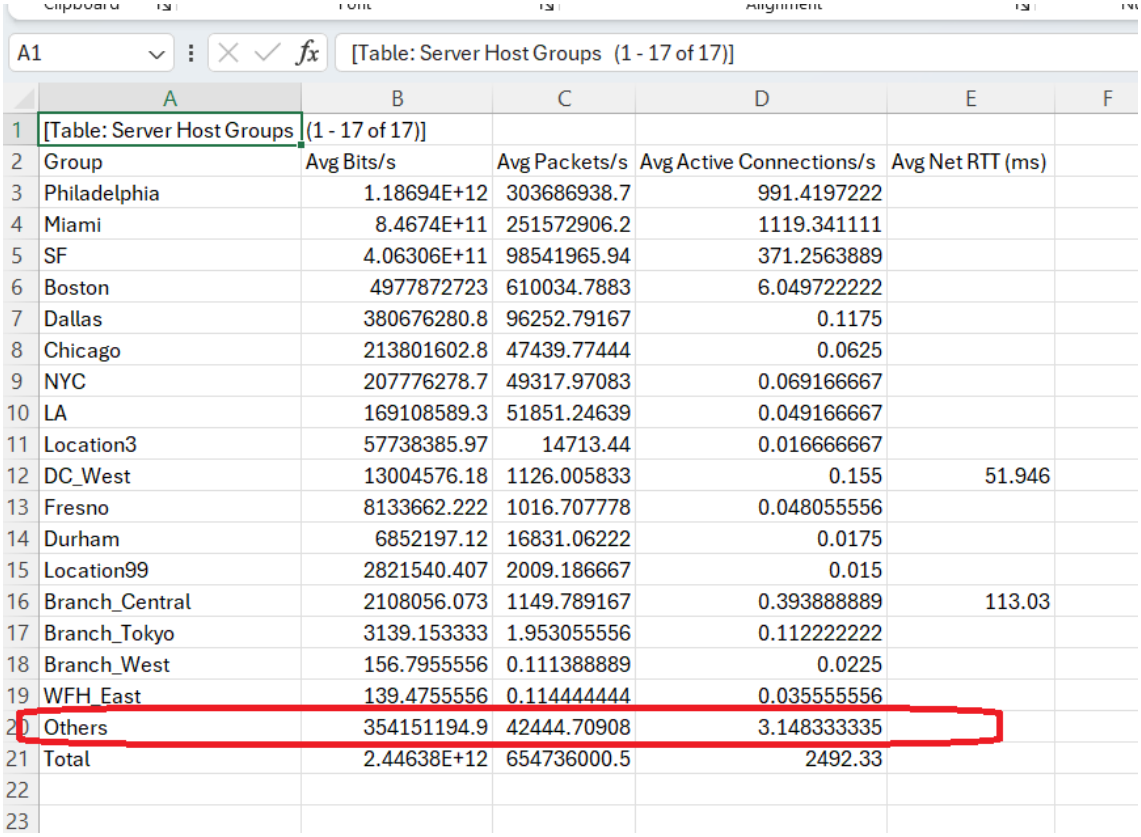


Group	Avg_				Active Connections/s	
Philadelphia	1,186,944,560,880				991.42	(40%)
Miami	846,739,591,448				1,119	(45%)
SF	406,305,768,173				371.26	(15%)
Boston	4,977,872,723				6.05	(< 1%)
Dallas	380,676,281	(< 1%)	96,253	(< 1%)	< 1	(< 0.01%)
Chicago	213,801,603	(< 0.01%)	47,440	(< 0.01%)	< 1	(< 0.01%)
NYC	207,776,279	(< 0.01%)	49,318	(< 0.01%)	< 1	(< 0.01%)
LA	169,108,589	(< 0.01%)	51,851	(< 0.01%)	< 1	(< 0.01%)
Location3	57,738,386	(< 0.01%)	14,713	(< 0.01%)	< 1	(< 0.01%)
DC_West	13,004,576	(< 0.01%)	1,126	(< 0.01%)	< 1	(< 0.01%)
Fresno	8,133,662	(< 0.01%)	1,017	(< 0.01%)	< 1	(< 0.01%)
Durham	6,852,197	(< 0.01%)	16,831	(< 0.01%)	< 1	(< 0.01%)
Location99	2,821,540	(< 0.01%)	2,009	(< 0.01%)	< 1	(< 0.01%)
Branch_Central	2,108,056	(< 0.01%)	1,150	(< 0.01%)	< 1	(< 1%)
Branch_Tokyo	3,139	(< 0.01%)	1.95	(< 0.01%)	< 1	(< 0.01%)
Branch_West	156.80	(< 0.01%)	< 1	(< 0.01%)	< 1	(< 0.01%)
WFH_East	139.48	(< 0.01%)	< 1	(< 0.01%)	< 1	(< 0.01%)
Others	354,151,195	(< 1%)	42,445	(< 0.01%)	3.15	(< 1%)
Total	2,446,383,969,024	(100%)	654,736,000	(100%)	2,492	(100%)



The CSV lists all host groups and their traffic. All groups summed up together add up to 100% of the traffic; if there are ungrouped hosts in your network, that traffic will show up in the Others row. The Others row represents ungrouped data; Riverbed IQ is not monitoring the ungrouped traffic and you may be missing important incidents. If the Others row in the CSV is present and has a large value, go to Definitions > Host Group Types and edit the ByLocation definitions. Then,re-run the report, export to CSV, and ensure that the Others group traffic is reduced.

An example CSV export for Server host groups:

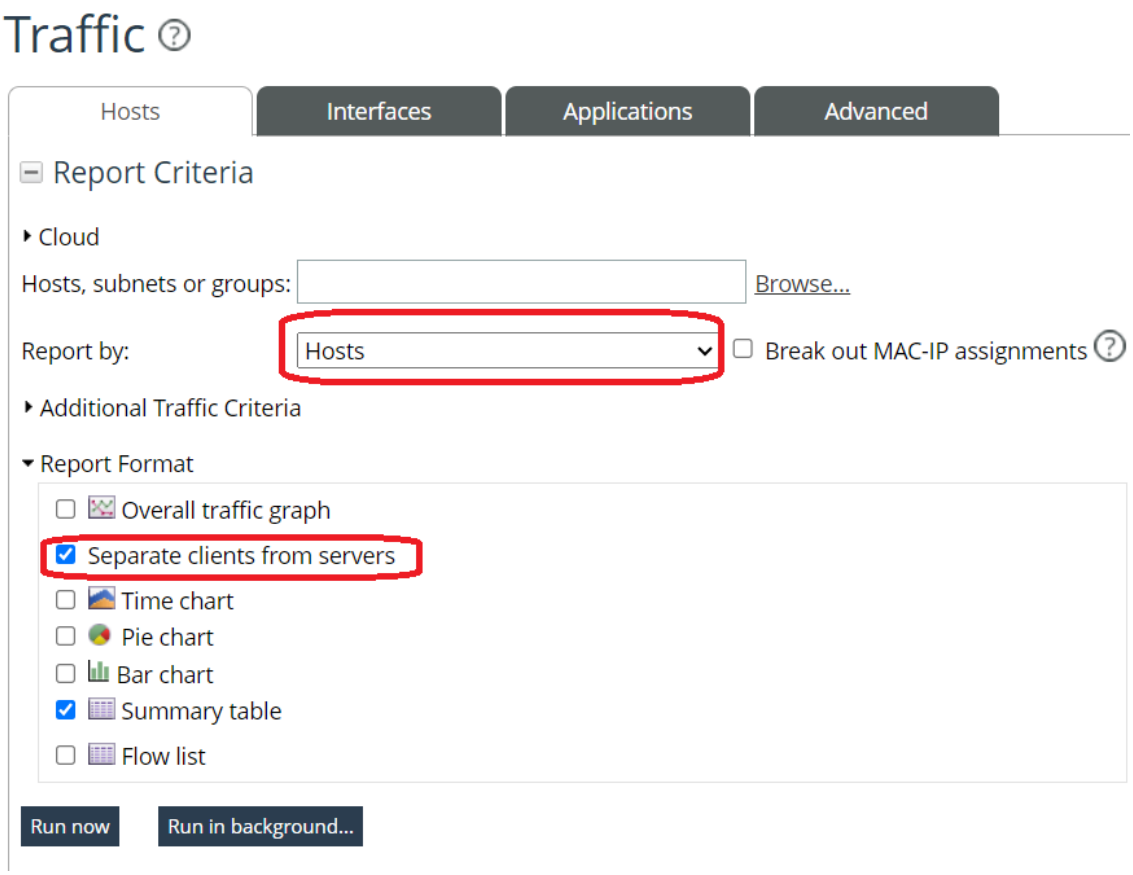


	A	B	C	D	E	F
1	[Table: Server Host Groups	(1 - 17 of 17)]				
2	Group	Avg Bits/s	Avg Packets/s	Avg Active Connections/s	Avg Net RTT (ms)	
3	Philadelphia	1.18694E+12	303686938.7	991.4197222		
4	Miami	8.4674E+11	251572906.2	1119.341111		
5	SF	4.06306E+11	98541965.94	371.2563889		
6	Boston	4977872723	610034.7883	6.049722222		
7	Dallas	380676280.8	96252.79167	0.1175		
8	Chicago	213801602.8	47439.77444	0.0625		
9	NYC	207776278.7	49317.97083	0.069166667		
10	LA	169108589.3	51851.24639	0.049166667		
11	Location3	57738385.97	14713.44	0.016666667		
12	DC_West	13004576.18	1126.005833	0.155	51.946	
13	Fresno	8133662.222	1016.707778	0.048055556		
14	Durham	6852197.12	16831.06222	0.0175		
15	Location99	2821540.407	2009.186667	0.015		
16	Branch_Central	2108056.073	1149.789167	0.393888889	113.03	
17	Branch_Tokyo	3139.153333	1.953055556	0.112222222		
18	Branch_West	156.7955556	0.111388889	0.0225		
19	WFH_East	139.4755556	0.114444444	0.035555556		
20	Others	354151194.9	42444.70908	3.148333335		
21	Total	2.44638E+12	654736000.5	2492.33		
22						
23						

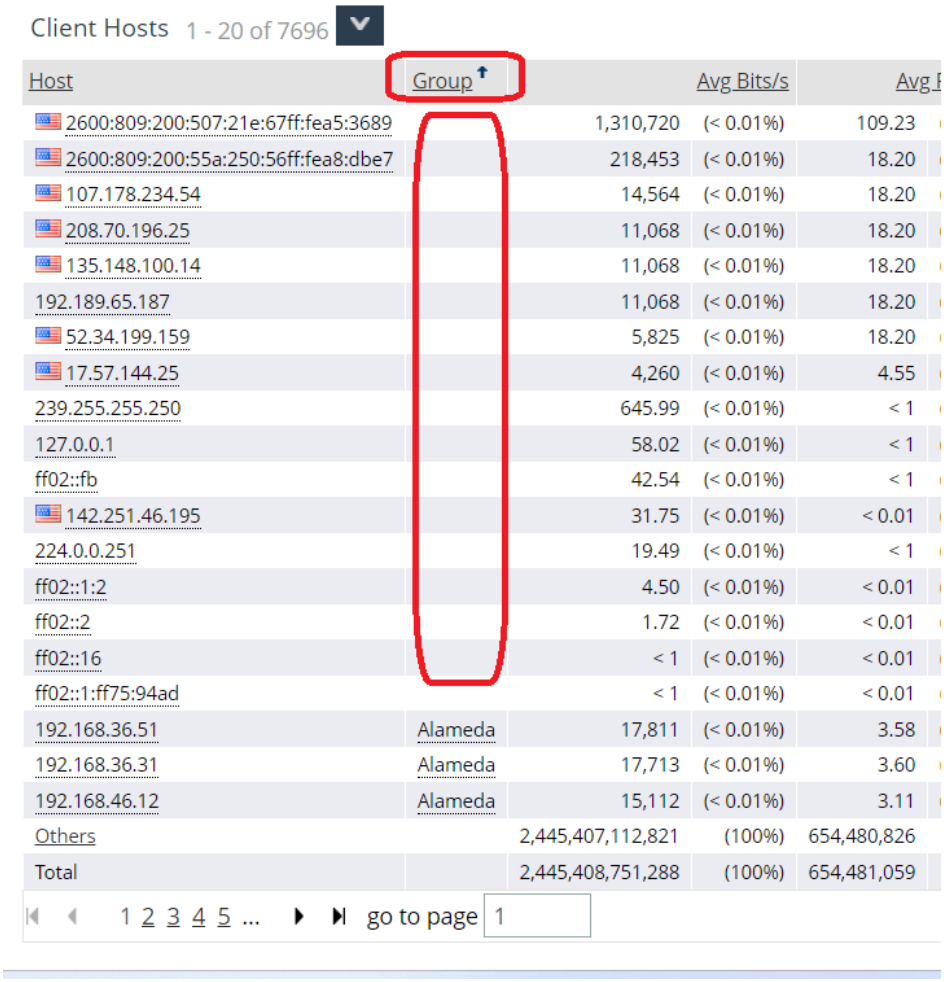
An example CSV export for Client host groups:

	A	B	C	D	E	F
1	[Table: Client Host Groups	1 - 167 of 167)]				
2	Group	Avg Bits/s	Avg Packets/s	Avg Active Connections/s	Avg Net RTT (ms)	
3	Dallas	5.74578E+11	150884932.5	494.2102778		
4	LA	5.05261E+11	147422290.4	372.5733333		
5	Boston	5.01259E+11	138248085.4	693.0861111		
6	Chicago	4.67779E+11	111573736.7	371.2497222		
7	NYC	2.3262E+11	61733496.65	374.3780556		
64	Oakland	2595332.782	417.5869444	0.057222222		
65	Branch_East	336595.7089	32.4275	0.173611111	51.324	
66	Branch_West	276402.3133	43.56333333	0.304166667	22.522	
67	Alameda	66968.38222	13.57277778	0.205	125.828	
68	Branch_Central	26344.84667	4.6575	0.205	41.13	
69	DC_West	361.14	0.181111111	0.033888889	17.409	
70	Others	873456790.9	85240.50708	1.605277778		
71	Total	2.44638E+12	654736000.5	2492.33		
72						

A way to discover the ungrouped hosts is to run a Traffic Hosts report and look for hosts that are not assigned to a group:



Sort on the Group column and unassigned hosts will show on top:



Client Hosts 1 - 20 of 7696

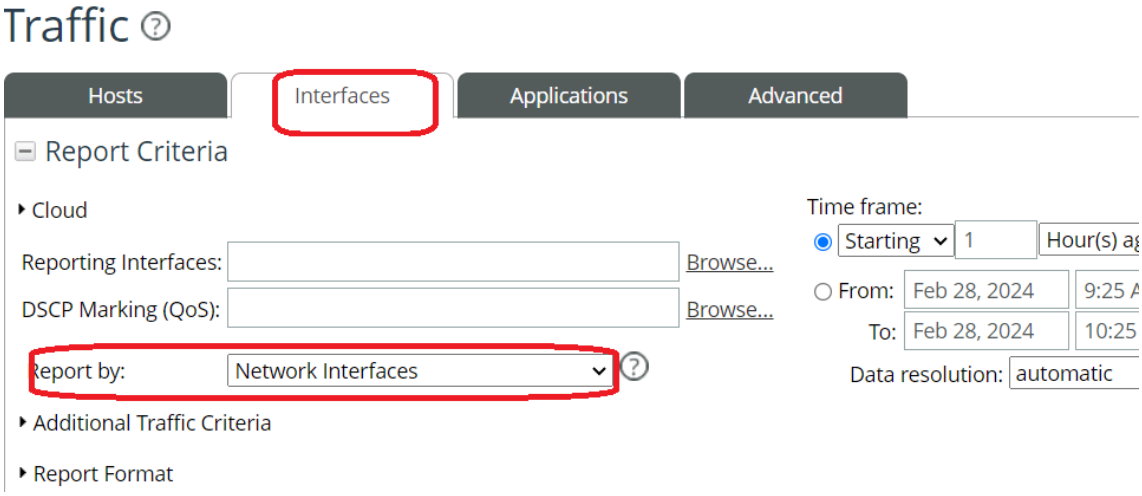
Host	Group		Avg Bits/s	Avg
2600:809:200:507:21e:67ff:fea5:3689		1,310,720	(< 0.01%)	109.23
2600:809:200:55a:250:56ff:fea8:dbe7		218,453	(< 0.01%)	18.20
107.178.234.54		14,564	(< 0.01%)	18.20
208.70.196.25		11,068	(< 0.01%)	18.20
135.148.100.14		11,068	(< 0.01%)	18.20
192.189.65.187		11,068	(< 0.01%)	18.20
52.34.199.159		5,825	(< 0.01%)	18.20
17.57.144.25		4,260	(< 0.01%)	4.55
239.255.255.250		645.99	(< 0.01%)	< 1
127.0.0.1		58.02	(< 0.01%)	< 1
ff02::fb		42.54	(< 0.01%)	< 1
142.251.46.195		31.75	(< 0.01%)	< 0.01
224.0.0.251		19.49	(< 0.01%)	< 1
ff02::1:2		4.50	(< 0.01%)	< 0.01
ff02::2		1.72	(< 0.01%)	< 0.01
ff02::16		< 1	(< 0.01%)	< 0.01
ff02::1:ff75:94ad		< 1	(< 0.01%)	< 0.01
192.168.36.51	Alameda	17,811	(< 0.01%)	3.58
192.168.36.31	Alameda	17,713	(< 0.01%)	3.60
192.168.46.12	Alameda	15,112	(< 0.01%)	3.11
Others		2,445,407,112,821	(100%)	654,480,826
Total		2,445,408,751,288	(100%)	654,481,059

1 2 3 4 5 ... go to page 1

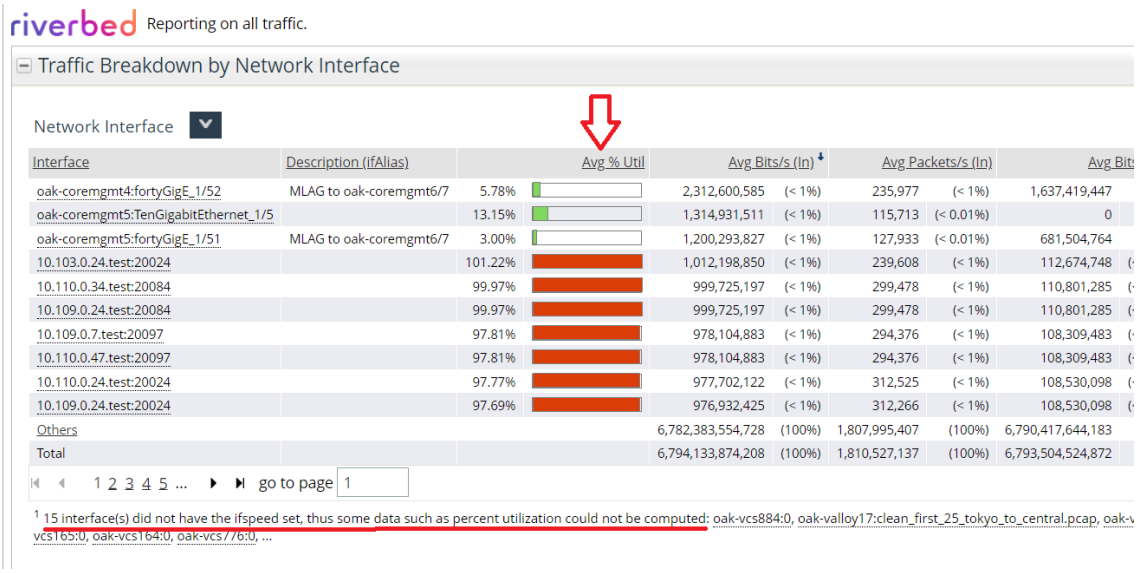
Note that the Traffic report is limited to 10,000 hosts. For larger networks you may not see all hosts, but you will see the top 10,000 talkers on the network.

Missing configuration: network interface data

A successful integration with Riverbed IQ requires the network interface speed to be configured and working. To ensure the definition is complete, go to Reports > Traffic and run a Network interfaces report:



Ensure that the % Utilization data is correct. If there are interfaces missing configuration, they will be listed in the warning below the table:



If Utilization is not available, go to Administration > Global SNMP settings to ensure that SNMP polling is enabled.

For an AppResponse data source:

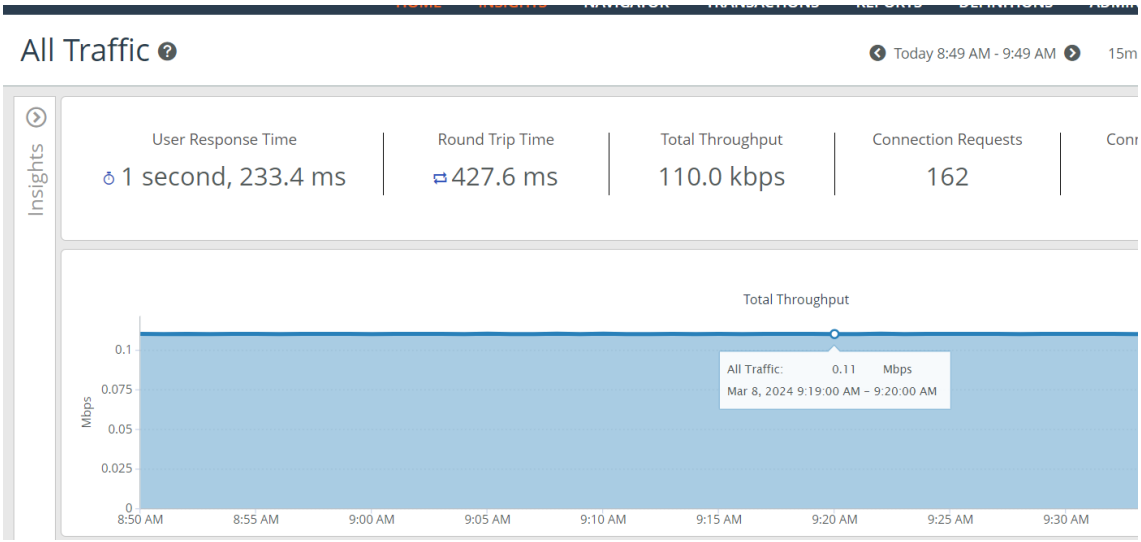
There can be several issues:

- Data is not flowing to AppResponse
- The clock is not synchronized

- There is missing configuration for host data

Data is not flowing

Log in to AppResponse and navigate to the Home page. Make sure data shows up in Total Throughput:



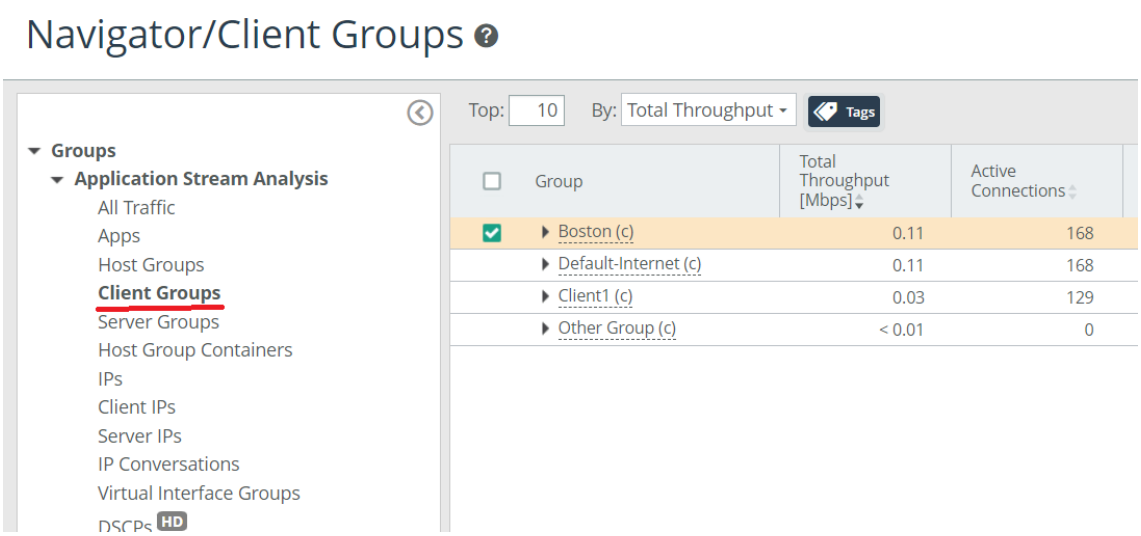
Clock synchronization

Ensure that the AppResponse clock is synchronized. When you log in to AppResponse, the date/time is shown in the banner at the top right. If the clock appears to be in error, go to Administration > General > Time and ensure that NTP or PTP synchronization is enabled.

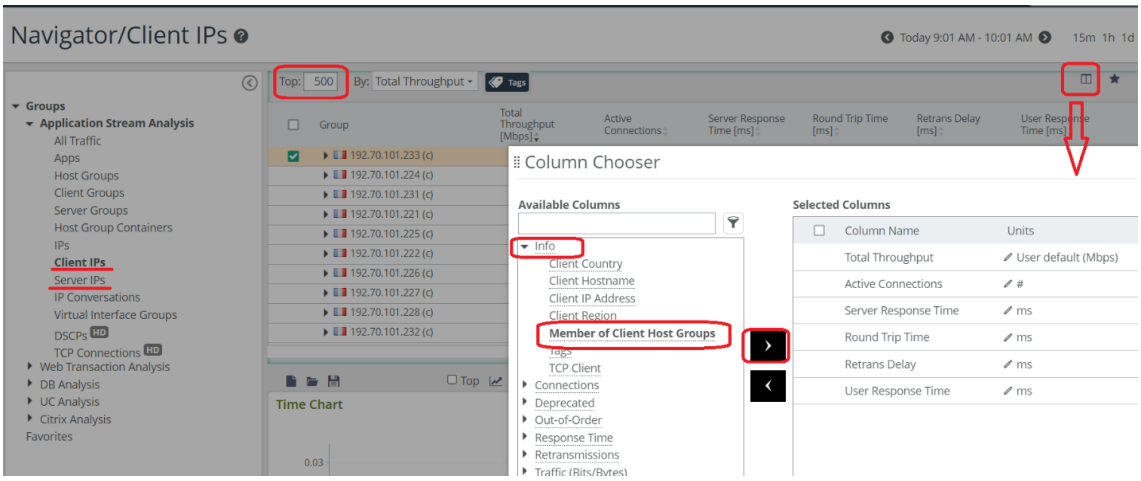
Missing configuration: Host data

A successful integration with Riverbed IQ requires Host Group definitions to be configured and working. If there are ungrouped hosts in your network, Riverbed IQ is not monitoring the ungrouped traffic, and you may be missing important incidents.

In the AppResponse UI, go to Definitions > Host Group and review the host groups. IQ is monitoring how applications are performing for each configured client host group. To ensure the definition is complete, go to Navigator > Client Groups and ensure that the groups are reporting traffic data.



To verify that the groups are complete, you can also go to Navigator > Client IPs and make sure all IPs are reported within a group. Click on Navigator > Client IPs, then change the top to 500 (500 is the maximum you can look at), then click on the column chooser icon on the top right, and, under the Info section, select the Member of Client Host Groups column and add it:



You can then click on the Member of Client Host Groups column header to sort on it and verify that there are no hosts that do not belong to groups, e.g., in this example, there are two ungrouped IPv6 hosts:

Group	Total Throughput [Mbps]	Member of Client Host Groups	Active Connections
fe80::20c:29ff:feb7:9ae9 (c)	< 0.01		0
fe80::20c:29ff:fee1:2ec3 (c)	< 0.01		0
192.70.1.234 (c)	< 0.01	Default-Internet, Boston	4
192.70.1.240 (c)	< 0.01	Default-Internet, Boston	3
192.70.1.237 (c)	< 0.01	Default-Internet, Boston	2
192.70.1.239 (c)	< 0.01	Default-Internet, Boston	3
192.70.1.231 (c)	< 0.01	Default-Internet, Boston	2
192.70.1.229 (c)	< 0.01	Default-Internet, Boston	3
192.70.1.232 (c)	< 0.01	Default-Internet, Boston	2
192.70.1.238 (c)	< 0.01	Default-Internet, Boston	3

Go to Definitions > Host Groups and include those hosts in group definitions.

Repeat the same process for Navigator > Server IPs.

Once the host group definitions are changed, allow 5 to 10 minutes for the AppResponse definition to take effect, and verify in the Navigator that the IPs are now showing in groups.

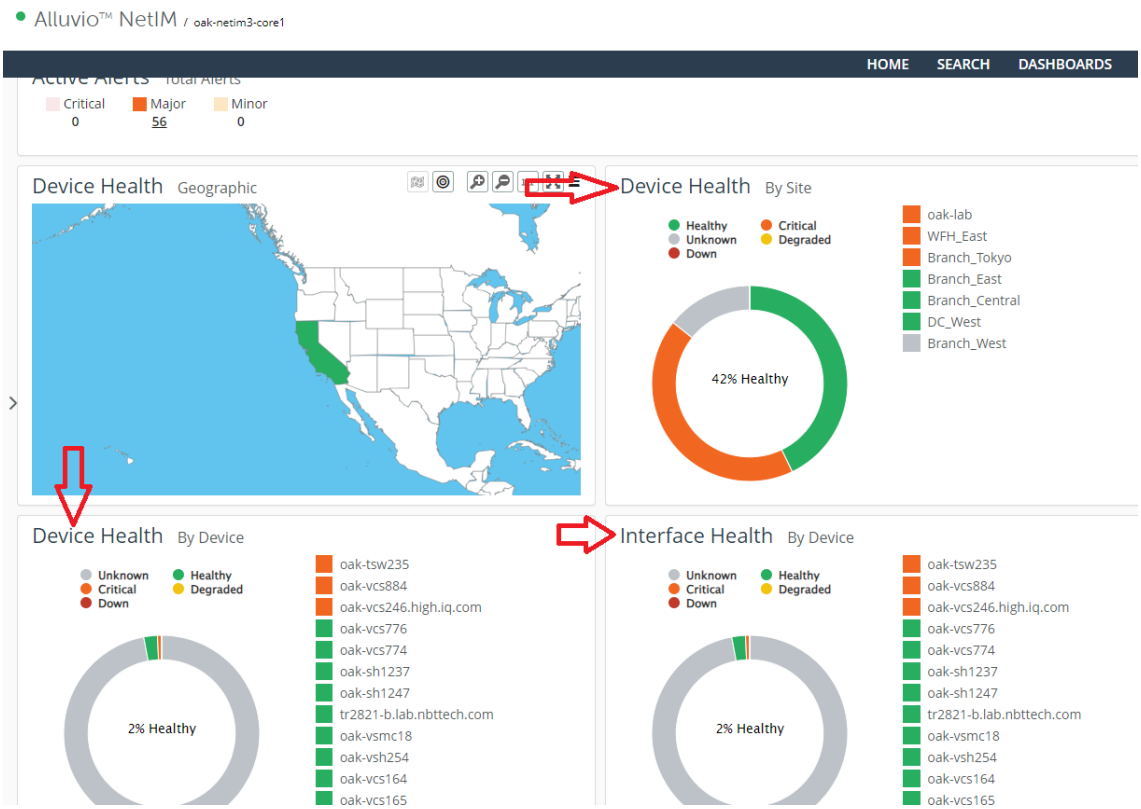
For a NetIM data source:

There can be these issues:

- Ensure that NetIM is configured to poll devices
- The Riverbed IQ connector module is operational

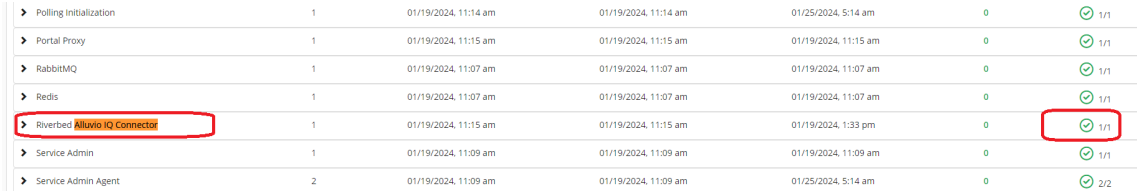
Check device polling configuration

Log in to the NetIM UI and ensure that the Device Health By Site, Device Health By Device, and Interface Health widgets are populated. If not, go to the Configure menu and configure devices for polling.



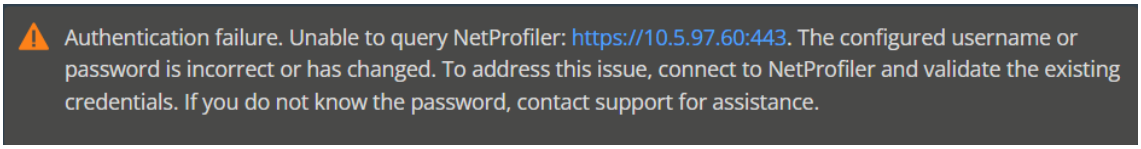
Check the Riverbed IQ Connector

Go to Help > System Status and look for the connector in Swarm services.



If it's not OK, click on it, download the logs, and contact Riverbed IQ support.

Authentication failure



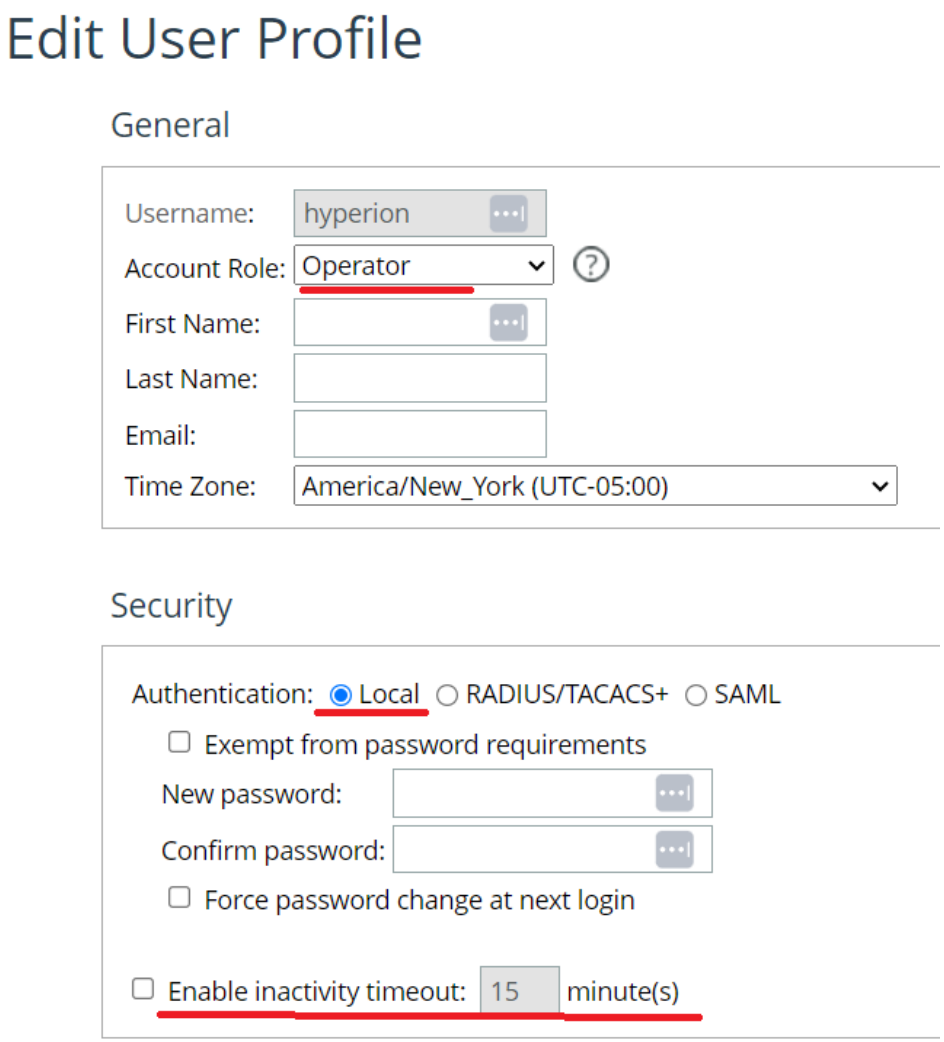
- Instructions for a NetProfiler data source
- Instructions for an AppResponse data source

- Instructions for a NetIM data source

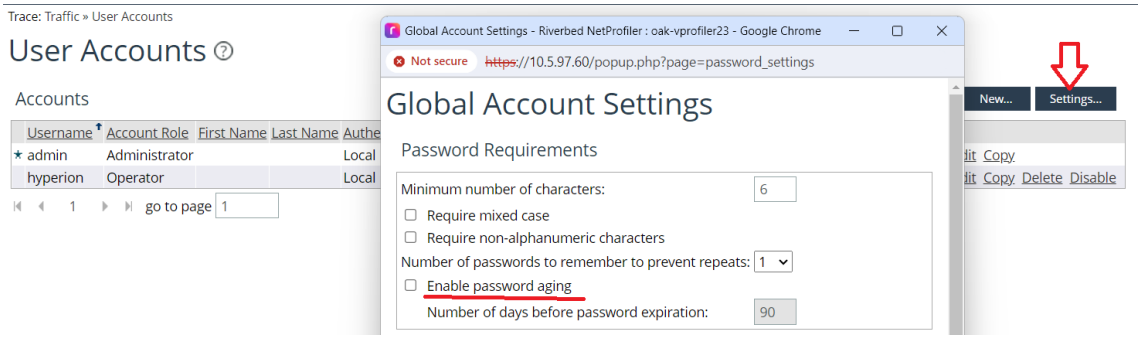
NetProfiler data source

Go to Administration > User Accounts and edit the user account for the Riverbed IQ integration:

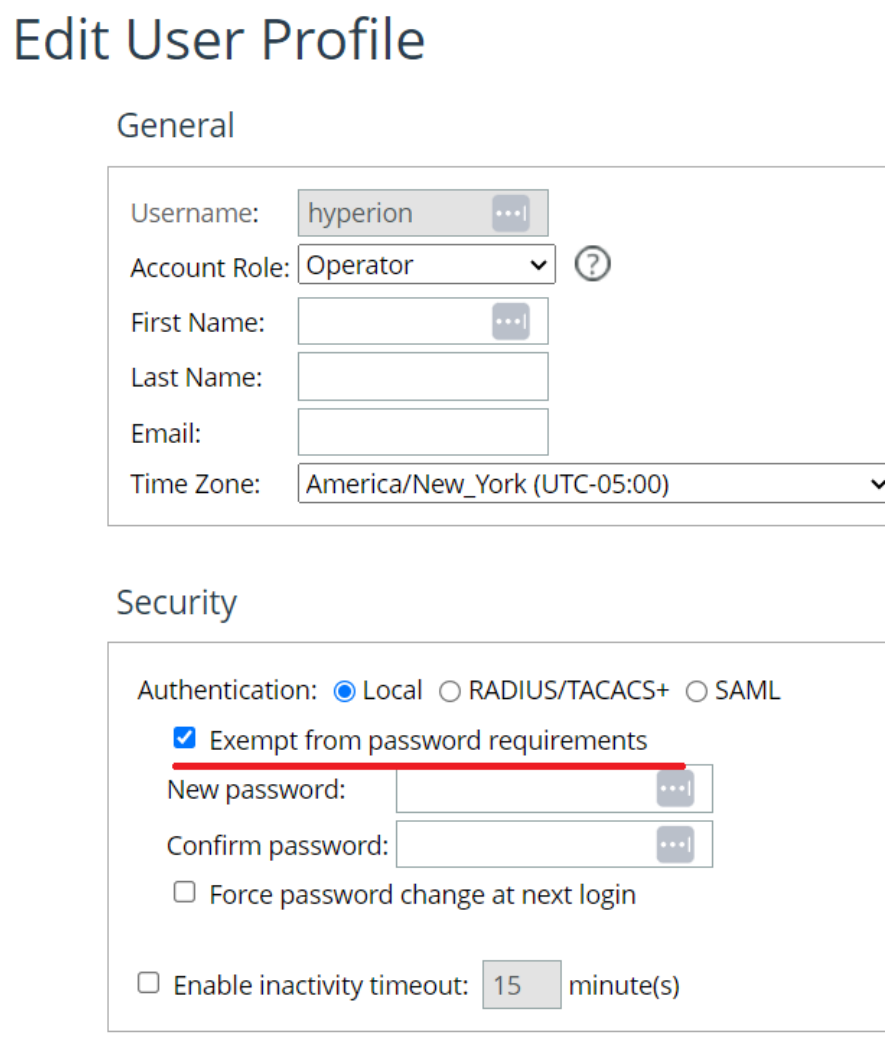
1. Ensure that the configured user has a role of Operator (recommended), or Administrator.
2. Check that the account is using local authentication. You can also use RADIUS/TACACS+. SAML cannot be used for this integration.
3. Ensure the inactivity timeout is not set.



Go to the global password settings and check if Enable password aging is set.



If it's set, then you can select the Exempt from password requirements in the Edit User Profile dialog:



If all NetProfiler settings are as expected, ensure that the password is configured correctly in the Set Credentials dialog.

AppResponse data source

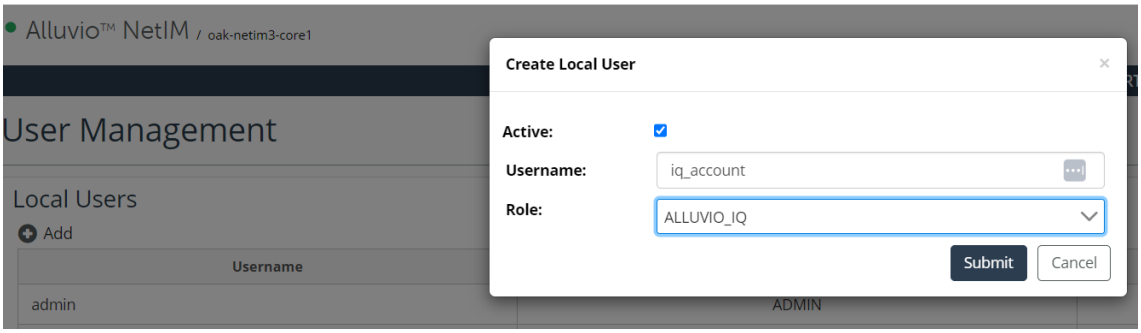
In AppResponse, go to Administration > User Administration and look at the user used for the IQ integration. The user should have an assigned role that has Full Control for Feature Settings and Definitions. To check the role permissions go to Roles and Permissions tab and edit the role:

Permission	No Access	View Only	Full Control
System configuration Configuration settings in Administration menu	◉	○	○
User Management User account, roles and password policy	◉	○	○
Feature Settings and Definitions Feature Settings sub-menu and definitions	○	○	◉
Access Packets / Live Views Packet export/download, apply PA+ live views	◉	○	○
Configure Trace Clips Trace clip configuration	◉	○	○
Access Other Users' Objects Read/Edit other users' private Insights, reports, schedules etc.	◉	○	○

If all AppResponse settings are as expected, ensure that the password is configured correctly in the Set Credentials dialog.

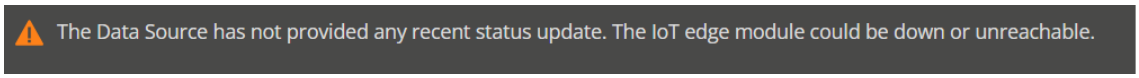
NetIM data source

Ensure that the configured user has a role of ALLUVIO_IQ.



If all NetIM settings are as expected, ensure that the password is configured correctly in the Set Credentials dialog.

The data source has not provided any recent status update



Check the status of the Riverbed Edge module. If it's down, then:

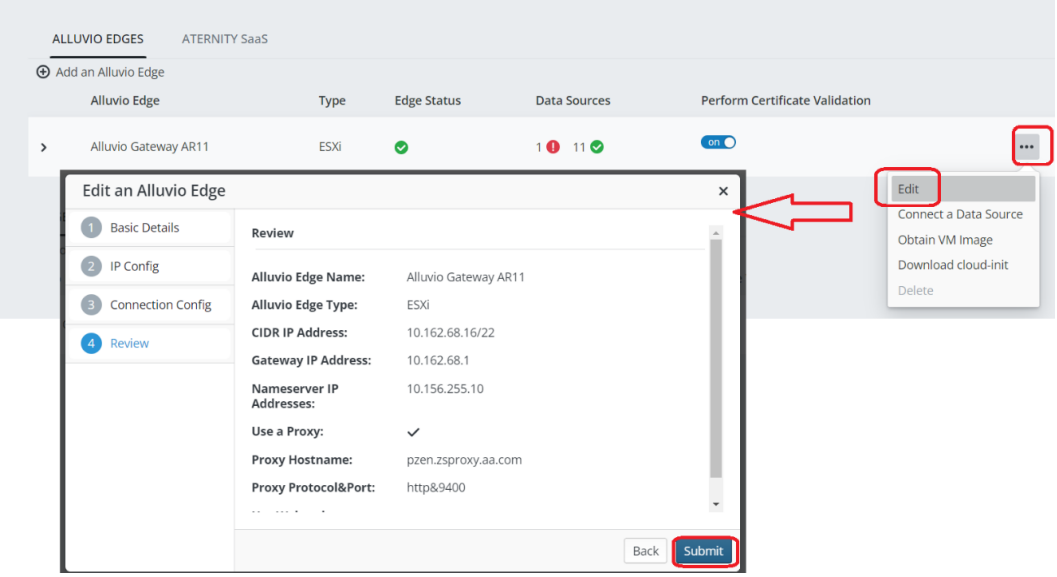
1. Check the Firewall rules.
2. Try to access the VM via the hypervisor and see if it can connect to the internet. (Get on the VM and try to ping a known web site such as google.-com.)

   If the VM can't connect to the internet, discuss with your networking team how to establish the connection. If you have a complex proxy setup, contact Riverbed IQ support for additional help before beginning.

   If the VM can connect to the internet, then the next step is to SSH to the edge VM and execute the command, "sudo iotedge check". Contact Riverbed IQ support for additional help with the command output.

   If the Edge status is Up, try to go through the Edge Edit dialog. Review the configurations and click Submit to force the edge to be reconfigured. If the issue persists, reboot the Edge VM (via hypervisor or SSH and

reboot), and, if that does not work, contact Riverbed IQ support.



Preferred Data Sources

Entities and objects that are streamed to Riverbed IQ can originate from more than one data source; for example, the User Response Time metric can be sent to IQ from AppResponse and NetProfiler for the same entity. The computations of some of these metrics (generally, application location metrics, and, particularly, User Response Time) vary between data source types (AppResponse and NetProfiler), which can complicate incident behavior and management in IQ. When more than one data source type is present, IQ collects the data sent from the preferred data source and ignores the data sent from other data sources. Note that AppResponse is the default preferred data source, and you can change the Edge's configuration to make NetProfiler the preferred data source, if necessary.

Important Considerations

The Preferred Data Source setting applies only to Application-Location metrics that can be sourced either from AppResponse or NetProfiler (Eg: User Response Time and %Total Retrans metrics). This setting does not apply to metrics that are streamed only from Netprofiler (Eg: Inbound/Outbound Network Utilization), these single-source metrics will be streamed regardless of the Data Source Preference.

Note these considerations pertaining to preferred data sources:

- If both AppResponse and NetProfiler are present, only one is allowed to stream to IQ, never both at the same time.
- AppResponse is the default preferred data source. When AppResponse is the preferred data source, streaming from NetProfiler is disabled.
- If you change the preferred data source to be NetProfiler, all streaming from AppResponse will stop.
- The preferred data source controls the data streamed for triggering incidents, but still allows runbooks to query either data source for information.

- MOS metrics are streamed only from NetProfiler, and, if AppResponse is preferred, IQ will use throughput metrics for MOS-based detections.
- AppResponse streams App-Location Throughput and % Failed Connections, neither of which is available when NetProfiler is the preferred data source.
- In general, changing the preferred data source will result in detections from different metrics, which affects runbook behavior. A warning message describing the effects of the change will be shown; see Changing the Preferred Data Source, below, for the text of that message.
- If the data source that is streaming is disabled, streaming stops. There is no fallback to the other data source.

Changing the Preferred Data Source

The preferred data source can be changed on the Data Sources page.

When you open an Edge to see its data sources, the Data Ingestion column will indicate the Preferred Data Source. If you wish to change this, click the **Change** button to open the Change Preferred Data Source dialog, which enables you to specify whether to prefer AppResponse or NetProfiler. This is accompanied by a warning about the effects of actually changing the data source: *Changing the preferred data source will reset baselines used for anomaly detection and it will require a few days for the system to train against the data ingested from the new preferred data source. Also, there could be performance implications with a large AppResponse deployment.*

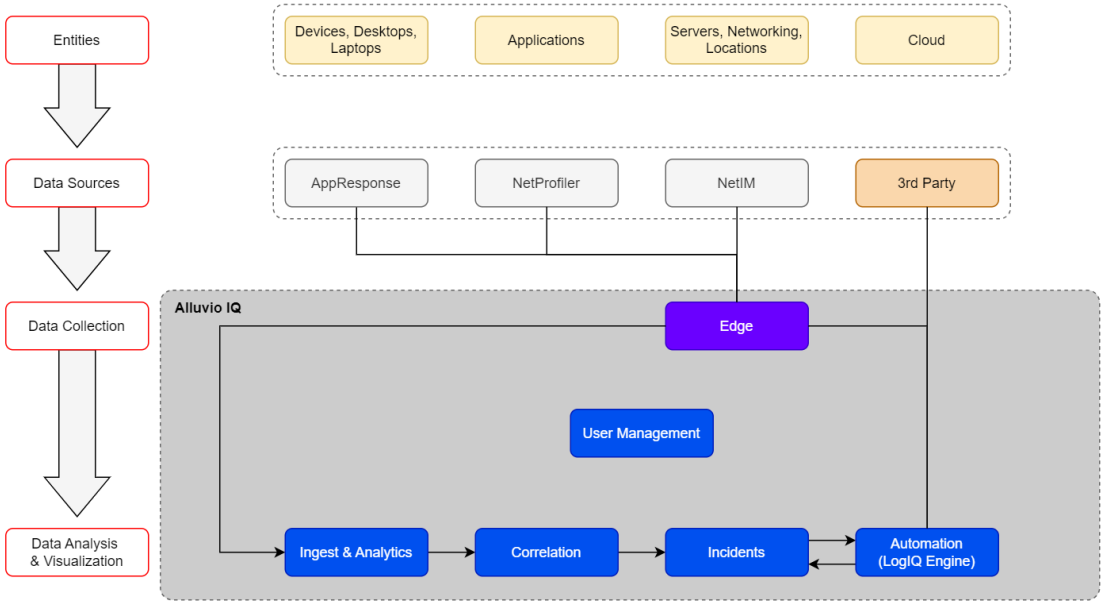
Consider talking with Riverbed Support prior to changing the preferred data source.

System Overview

Riverbed IQ comprises multiple components that apply artificial intelligence (AI) and automation technologies to:

- Observe and model a customer environment
- Detect anomalous behavior
- Correlate related anomalies, reducing noise by grouping the multiple facets of a common event
- Surface these detections (and all associated context) as incidents
- Execute an automated investigation to assess business impact and streamline remediation workflows to ensure a high level of user experience.

This system diagram shows Riverbed IQ components relative to the underlying entities and data sources that reside in the customer environment:



System Diagram

Customer Environment

Entities Things deployed in the customer environment that are needed to run the business, e.g.,

- Devices, desktops, laptops
- Applications
- Servers, networking equipment, locations
- Cloud

Data Sources One or more monitoring/observability tools that watch over the entities in the customer environment, e.g.,

- Riverbed AppResponse
- Riverbed NetProfiler
- Riverbed NetIM
- other third party tools

Riverbed IQ

Data Collection Riverbed IQ can interact with a multitude of data sources, with native support for Riverbed's NPM portfolio and its full fidelity data (i.e. Riverbed AppResponse, Riverbed NetProfiler, Riverbed NetIM), and integration-capabilities for other third party tools.

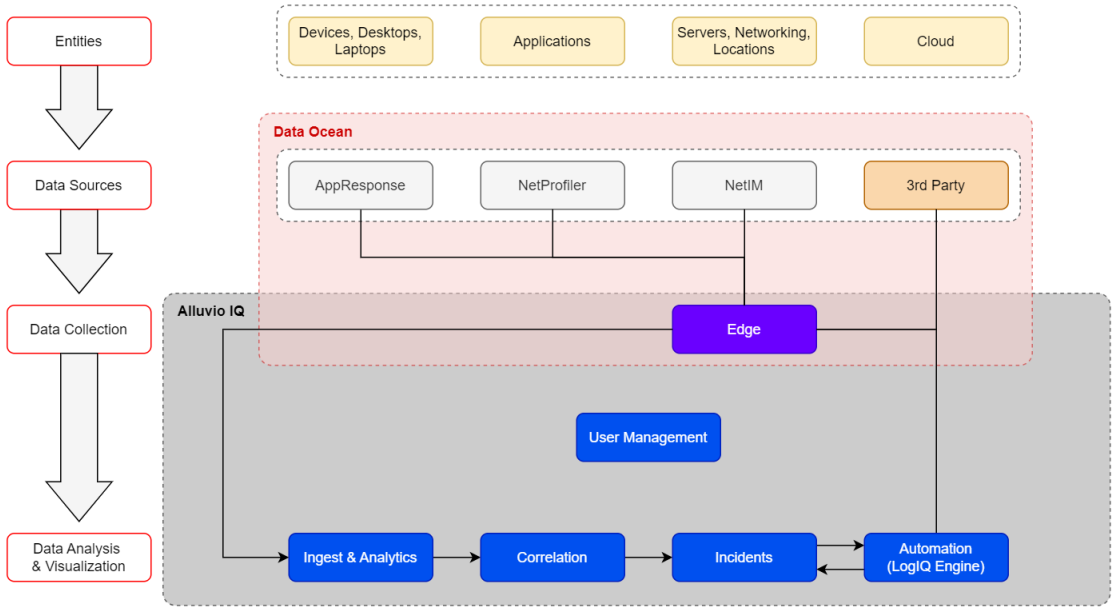
- **Edge gateway** is a component of Riverbed IQ that resides at the edge of the customer environment and provides two-way communication between data sources residing in the customer environment and the Riverbed IQ software-as-a-service components in the cloud. Riverbed employs a shared responsibility model for Riverbed Edge that comprises a Riverbed-supplied Edge ISO image or cloud-init file that is deployed on top of a customer-supplied and managed virtual machine (VM).

Data Analysis & Visualization Riverbed IQ employs a pipeline of components to process information streamed in from data sources to intelligently detect issues, execute automations, and, ultimately, surface actionable insights that assess impact(s) and expedite remediation:.

- **Ingest & Analytics**
    - observes and models the data arriving from data sources (key measurements) in the customer environment
    - analyzes the key measurements arriving from data sources in the customer environment for anomalous behavior (i.e., a key measurement value that does not match the model, generates an indicator that represents the anomaly in the system
- **Correlation**
    - Correlates related anomalies into a single detection (in order to reduce noise by grouping the multiple facets of a common event)
    - Detection may contain one or more indicators
- **Incidents**
    - Surface detections as a report (incident) that contains all information gathered over the course of processing through the pipeline and the associated Runbook automation.
- **Automation (LogIQ Engine)**
    - Executes the automated investigation (Runbook) associated with this event, and attaches the resulting analysis to the Incident report (e.g. assessed business impact, supporting data/context, …).
- ***Miscellaneous*** *Components related to administration and configuration of Riverbed IQ*
    - ***User Management***
    - ***Configuration***

Connecting a Customer Environment to Riverbed IQ SaaS

This illustration shows how data sources residing in the customer environment connect to Riverbed IQ:



Customer Environment Connectivity to IQ SaaS

Note these prerequisites:

- The target data sources must be configured with credentials that permit Riverbed IQ to connect and establish the two-way conversation that is needed for the data source to stream key measurements to Riverbed IQ, and for the data source to process/reply to Runbook requests that occur over the course of an automated investigation.
- The target Riverbed Edge platform needs to be selected and prepared (currently, Riverbed IQ supports deploying Riverbed Edge on the following: AWS, Azure, and VMware ESXi).
- Riverbed IQ employs a shared ownership model for the Edge in which Riverbed supplies an Edge image for deployment on top of a customer-supplied and managed VM.

        - Configure the Riverbed Edge gateway on your Riverbed IQ tenant (see Configuring Riverbed Edge and Data Sources).
        - The target customer-owned/managed VM environment needs to be selected and prepared (currently, Riverbed IQ supports deploying Riverbed Edge

          in a number of VM environments).
        - Use the Riverbed-supplied cloud-init or ISO file to configure the Edge on your target VM-environment (see Configuring Riverbed Edge and Data Sources).
    - Once the Riverbed Edge gateway is configured, configure the target data sources on your Riverbed IQ tenant (see Configuring Riverbed Edge and Data Sources).

Once all Riverbed Edge(s) and data source(s) are configured on the Riverbed IQ tenant, data will begin to flow into the system and:

- the *Data Ocean* will begin to take shape:
    - This is a distributed repository of customer environment information that is accessible to Riverbed IQ. It comprises a small amount of core information that is cached in the customer's Riverbed IQ tenant and the vast full fidelity data that resides in the native Riverbed data sources (e.g., while executing an automation, the Runbook may quickly retrieve cached information from the customer's Riverbed IQ tenant, or may issue a query directly to a data source to tap into high fidelity data).
- *Key Measurements* will begin to flow into the pipeline:
    - Ingest & Analytics: will begin processing key measurements
        - Certain key measurements have a simple model (e.g. static threshold) that enable anomalies to be immediately detected/processed all the way through incident-generation and

Runbook execution.

- Other key measurements employ a more complex model (e.g time-series baseline) that will require time to build the model. As a result, it will take time to detect anomalous behavior for these key measurements (and further process those anomalies into Incidents with associate Runbook-executions), e.g.,
    - It takes two days to build an initial daily-seasonal time-series baseline model, and 14 days to build an initial weekly-seasonal time-series baseline model.

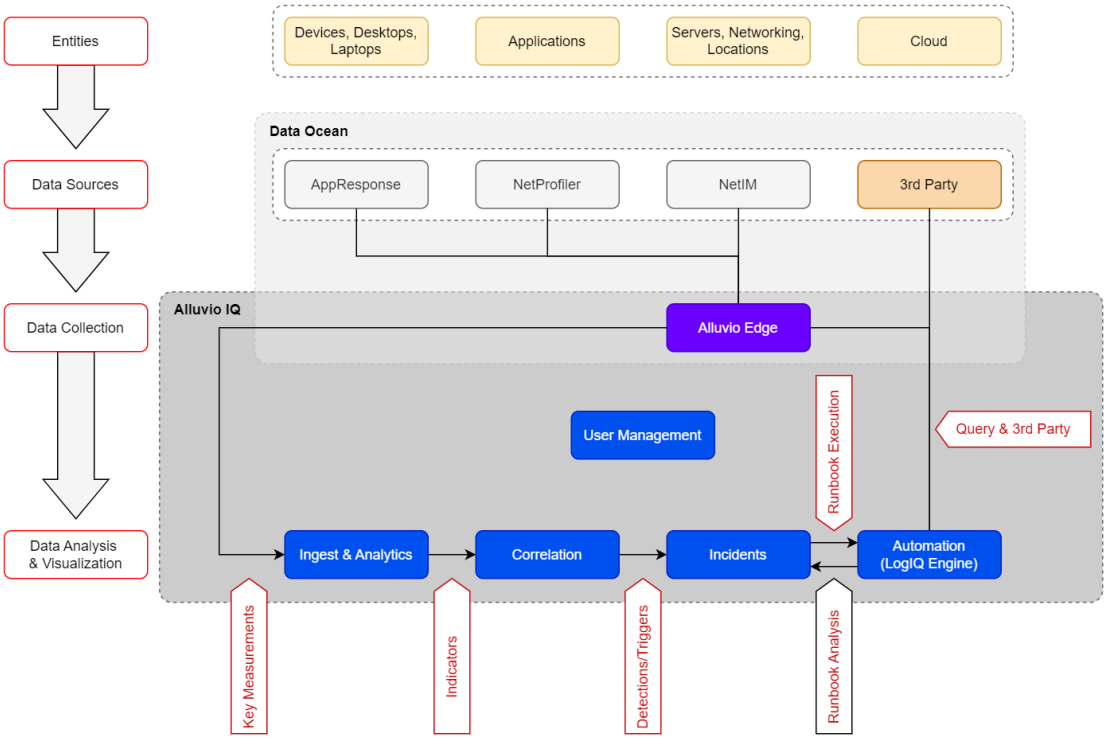
Incident Generation and Runbook Execution

Introduction

This topic describes in detail ideas introduced in Connecting a Customer Environment to Riverbed IQ SaaS :

1. Key measurements are modeled by ingest & analytics
2. Anomalous measurements generate associated indicators
3. Correlation groups assemble indicators into detections that form the basis of incidents
4. Incidents trigger Runbook executions

This diagram illustrates how information flows and is processed to become an incident that executes a Runbook.



Incident Generation and Runbook Execution

Data Sources: Key Measurements

The key measurements flowing into a customer's Riverbed IQ tenant depend directly on the data sources present in the customer environment and connected to that Riverbed IQ tenant.

The key measurements streamed from each data source type are:

Pipeline Inputs: Key Measurements

Data Source	Entity[s]	Metric
NetProfiler	Application / Client Location	User Response Time (*see Table Note-1 below*)
		MoS
	Interface	In Utilization
		Out Utilization
AppResponse	Application / Client Location	User Response Time
		Throughput (*see Table Note-2 below*)
		% Retrans Packets
		% Failed Connections
NetIM	Device	Device Status
		Device Uptime
	Interface	Interface Status
		In Packet Error Rate
		Out Packet Error Rate
		In Packet Drops Rate
		Out Packet Drops Rate
		In Utilization
		Out Utilization
Table Notes:		
1 - **[Metric: User Response Time]** is: > An approximation of AppResponse [user-response-time] because NetPro-		

filer does not currently account for [connection_setup_time], while AppResponse does.

> Only processed for named applications (e.g., excludes: ICMP, SNMP, TCP_Unknown, UDP_Unknown)

2 - **[Metric: Throughput]** is monitored only for VoIP-related applications: {VOIP, SIP, RTP}.

Ingest & Analytics: Indicators

Overview

For each key measurement flowing into a customer's Riverbed IQ tenant, Ingest & Analytics either has a simple model (e.g., Static Threshold) or builds a more dynamic, complex model (e.g. Time Series Baseline) of that key measurement's behavior, and any significant variance from that model is classified as an anomaly (hence, the modeling technologies are called anomaly detection algorithm(s)). Riverbed IQ generates an indicator for each anomaly detected (the indicator contains contextual information associated with the anomaly that can be leveraged by further processing).

Anomaly Detection Algorithms

Riverbed IQ supports the following anomaly detection algorithms:

Explainable AI

Static Threshold (ThresholdDetector)

- **IQ ThresholdDetector** models configured simple threshold(s). It can be configured to model "too-high" thresholds and/or "too-low" thresholds.
    - When configured to model "too-high" thresholds: any **[observed-value]** that exceeds the model's **[high-threshold]** is classified as an anomaly.

    - When configured to model “too-low” thresholds: any **[observed-value]** that falls below the model's **[low-threshold]** is classified as an anomaly.
- **Example Graph:** [out_packet_drops_rate] observed values stay below the [high-threshold] up until Jan-29th, 12:15PM when [observed-value: 2.3%] exceeds the [high-threshold: 1%] and generates an Indicator (red-zone):

Time-series Baseline (BaselineDetector)

- **IQ BaselineDetector** model leverages Season/Trend/Level (STL) history/data to predict an **[expected-range]** of values (i.e. “green-river”) within which it expects the next **[observed-value]** to lie. The initial model is built using twodays of data (i.e., models daily-seasonality), and is replaced by a final model after two weeks of data (i.e. models weekly-seasonality). The model is updated with each new **[observed-value]**.
    - Any **[observed-value]** that falls above of the predicted **[expected-range]** will be classified as an anomaly.
    - Any **[observed-value]** that falls below the predicted **[expected-range]** will be classified as an anomaly.
- **Example Graph:** [user_response_time] observed-values stay within the green river up until Jan-29th, 11:00AM when [observed-value: 1.18s] exceeds the [expected-range: 0ms - 1.02s] and generates an Indicator (red-zone):

Dynamic Threshold (DynamicThreshold)

- **DynamicThresholdDetector** models a target dataset using an exponential distribution that is “fitted” to the target dataset in order to profile the pattern. This profile is then used to calculate the **[dynamic-threshold-value]** for the dataset when a new **[observed-value]** arrives. The model is updated with each new **[observed-value]**.

    - Any **[observed-value]** that falls above the calculated **[dynamic-threshold-value]** will be considered anomalous.
- **Example Graph:** [activity_network_time] observed values stay within the green-river up until Jan-10th, 12:00PM when [observed-value: 0.90s] exceeds the [dynamic-threshold-value: 0.78s] and generates an Indicator (red-zone):

State Change (ChangeDetector)

- **IQ ChangeDetector** model does not make any dynamic predictions; it models a configured **[expected-value]**, and any state change in **[observed-value]** that does not match **[expected-value]** is treated as an indicator of bad behavior.
- **Example Graph:** [device-status] observed-value is consistently up ([observed-value: 1]) up until Jan-10th, 1:45PM:
    1. then [observed-value: 0] occurs (i.e. state-change from prior [observed-value: 1]), and
    2. the result of that state-change is that [device-status] no longer matches [expected-value: 1] and generates an Indicator (red-zone):

Continuously Increasing (AlwaysIncreasingDetector)

- **AlwaysIncreasingDetector** models a system where the dataset has ever-increasing values during normal operation.
    - Any **[observed-value]** that is less than **[previous-value]** is treated as an indicator of bad behavior.
- **Example Graph:** [device_up_time] observed-values continue to increase up until Jan-11th, 6:45AM when [observed-value: 23h 59m 16s] - then there is absence of samples for 10-minute period, and when the next sample arrives [observed-value: 9m 10s], is determined to be less

than the last known measurement (i.e. 23h 59m 16s) and an indicator is generated (red-zone):

Current Key Measurements and Associated Anomaly Detection Algorithms

For each key measurement flowing into a customer's Riverbed IQ tenant, Ingest & Analytics has been pre-configured apply specific anomaly detection algorithm(s).

The association of key measurements to anomaly detection algorithms is shown here:

Riverbed IQ Analytics Pipeline - Quick Reference

Data Source	Entity	Metric	[1] Change Detection	[2] Always Increasing	[3] Thresholds	[4] Baselines	[5] Dynamic Threshold
NetProfiler	Application / Client Location	User Response Time[1]	✗	✗	✗	✓	✗
		MoS	✗	✗	✓	✗	✗
	Interface	In Utilization	✗	✗	✓[3]	✓[3]	✗
		Out Utilization	✗	✗	✓[3]	✓[3]	✗
AppResponse	Application / Client Location	User Response Time[1]	✗	✗	✗	✓	✗
		Throughput[2]	✗	✗	✗	✓	✗
		%	✗	✗	✓	✗	✗

NetIM	Device	Retrans Packets	✗	✗	✓	✗	✗
		% Failed Con-nections	✗	✗	✓	✗	✗
		Device Status	✓	✗	✗	✗	✗
		Device Uptime	✗	✓	✗	✗	✗
	Inter-face	Interface Status	✓	✗	✗	✗	✗
		In Packet Error Rate	✗	✗	✗	✓	✗
		Out Packet Error Rate	✗	✗	✓	✗	✗
		In Packet Drops Rate	✗	✗	✗	✓	✗
		Out Packet Drops Rate	✗	✗	✓	✗	✗
		In Util-ization	✗	✗	✓[3]	✓[3]	✗
		Out Util-ization	✗	✗	✓[3]	✓[3]	✗

Notes:

1 - [Metric: User Response Time] is:

- an approximation of AppResponse [user-response-time] because NetProfiler does not yet account for [connection_setup_time] - while AppResponse does, i.e.

AppResponse User Response Time calculation: [user-response-time]

= ([connection_setup_time] / [connection_setup_time_n])

+ ([request_network_time] / [request_network_n])

+ ([response_network_time] / [response_network_n])

+ ([server_delay] / [server_delay_n])

NetProfiler User Response Time calculation: [user-response-time]

= ~~([connection_setup_time] / [connection_setup_time_n])~~

+ ([request_network_time] / [request_network_n])

+ ([response_network_time] / [response_network_n])

+ ([server_delay] / [server_delay_n])

- only processed for “named” applications (e.g. excludes: ICMP, SNMP, TCP_Unknown, UDP_Unknown)

2 - [Metric: Throughput]: only monitored for VoIP-related “applications”: {VOIP, SIP, RTP}.

3 - [Anomaly Detection Algorithms]: for compounded Anomaly Detection Algorithms an Incident is only generated when both Algorithms detect anomalous behavior.

Correlation: Detections

Every indicator generated by Ingest & Analytics is further processed by Correlation to identify associations/commonalities that will enable them to be grouped together into a single detection (that will eventually be surfaced as an incident).

As [Correlation] groups related Indicators together, it also has to identify which Indicator is “Primary” (i.e. the leading Indicator of a problem). The Primary Indicator is also the basis upon which an Incident is generated - and also determines which Runbook will be executed). All other Indicators in the Detection take on a supporting-role as “Correlated” Indicators.

NOTE: It is possible for a Detection to have a single Indicator (i.e. no associations/commonalities found), in which case the Detection only contains a Primary Indicator.

Incidents

Overview

Riverbed IQ surfaces anomalous events (represented by a Detection) in an Incident report.

For new events, Riverbed IQ generates a new Incident to contain the event-information gathered so far (i.e. the Detection) - and so an Incident contains Primary Indicator information, and Correlated Indicators information (if any exist). Whenever a new Incident is created, an associated Runbook is automatically executed and its output is associated with the Incident (in “Runbook Analysis” section).

For recurring events, Riverbed IQ matches the recurrent Detection to an existing active Incident. Runbooks are not automatically executed for recurrences.

Each Incident is a combination of the information gathered throughout the Analytics Pipeline (i.e. Detection, Primary Indicator, Correlated Indicators), and the result of the Runbook-execution (i.e. Impacts, Prioritization, Prioritization Factors, and Runbook Analysis).

Basic Anatomy of an Incident

Each Incident report contiains is a combination of the information gathered throughout the Analytics Pipeline (i.e. Detection, Primary Indicator, Correlated Indicators, …), and the result of the Runbook-execution (i.e. Impacts, Prioritization, Prioritization Factors, and Runbook Analysis). The content generated by Runbook-execution will likely change for each instance of a Runbook-execution.

Please refer to the diagram below for a high-level view of the basic anatomy of an Incident (i.e. what information it contains, and the source of that information).

- Section: <Header> — Top-level summary information for this Incident:
    - [Priority] — The default is [Priority: Low] when Incident is created, and then [Priority] is updated based on analysis performed during Runbook Execution. Ultimately, it is the Runbook logic/analysis that determines [Priority].
    - [Description] — This information is gleaned from the Primary Indicator provided by Correlation::Detection (e.g. “MS Teams at Boston shows an increase in % Retrans Packets”).
- Section: <Management/Lifecycle> — Information/tools related to the Incident and its lifecycle.
    - [State] — [State: New] when an Incident is created. Users can change the [State] to represent the phase an Incident is in its lifecycle.

        - [Start Timestamp] — This is the timestamp when an Incident is created.
        - [End Timestamp / Ongoing] — This timestamp represents when an Incident ends (i.e. no recent recurrence of an Indicator) - AND: is flagged as [Ongoing] for Incidents that have not yet ended.
        - [Notes] — provides access to Note-tool that provides ability for Users to provide comments on an Incident. Clicking on this tool will display history of [Notes] for review, and ability to add more [Notes].
        - [Activity Log] — provides access to Logging-tool that provides Users visibility to activity associated with an Incident. Clicking on this tool will display listing of [Activity Log] - e.g. “Runbook status changed to Completed”, with timestamp.
        - [Share] — provides quick/easy access to the unique URL associated with a specific Incident. Clicking on this tool will provide the User the opportunity to [Copy Link] into their Copy-Paste buffer. (NOTE: User can also copy the URL from their Web-browser to share a specific Incident).
    - Section: [Impact Summary] — Renders information generated by Runbook logic that assesses the potential business impact of an Incident.
        - [Impacts: Users] — a card that provides listing of Users potentially impacted by this Incident (based on assessment made by associated Runbook execution).
            - NOTE: If there is a large listing of impacts, then a “View All” link will appear in the lower-right of the card so the full list can be displayed.
        - [Impacts: Locations] — a card that provides listing of Locations potentially impacted by this Incident (based on assessment made by associated Runbook execution).

        - NOTE: If there is a large listing of impacts, then a “View All” link will appear in the lower-right of the card so the full list can be displayed.
    - [Impacts: Applications] — a card that provides listing of Applications potentially impacted by this Incident (based on assessment made by associated Runbook execution).
        - NOTE: If there is a large listing of impacts, then a “View All” link will appear in the lower-right of the card so the full list can be displayed.
- Section: [Prioritization Factors] — Listing of information used by Runbook logic to determine [Priority] of an Incident.
- Section: [Incident Source] — Renders the components of the Detection built by Correlation:
    - [Primary Indicator] — this represents the anomaly that Correlation identified as the likely primary cause for this Incident, and includes supporting information:
        - Textual summary of the Entity/Metric/Location and Timestamp for the anomaly.
        - Graphical representation of the model (recent history), with:
            - annotations to indicate:
                - [Primary Indicator] — this is when/where Anomaly Detection Algorithm detected an unexpected Key Measurement value.
                - [Incident Start] — this is when/where an Incident was generated to surface this event. (There are propagation and processing delays that result in [Incident Start] after [Primary Indicator])

- [Runbook Executes] — this is when/where the associated Runbook is executed to: gather additional data/context from the Customer Environment (Data Sources, and 3rd Party), apply logic/analysis to assess impact/prioritization — and likely cause. It is the result of a Runbook execution that is rendered in Section: [Runbook Analysis] of an Incident.
- [Incident End] — this is when/where an Incident ends after a period in which either the Anomaly is recovered (e.g. Device returns to [Status: UP]) or there has not been a recent recurrence of the anomaly.

- … and tools to navigate through the graphical model-history:
  - [Rewind-button] — expand the graphical model-history view to encompass an additional earlier time-period (in chunks of 6-hours per button press).
  - [FastForward-button] — narrow the graphical model-history view to encompass an more recent time-period (in chunks of 6-hours per button press).
  - [Home-button] — returns to default view of graphical model-history.

- [Correlated Indicators] — this is a listing of one or more anomalies/Indicators that Correlation identified as having some relationship/association with the Primary Indicator for this Incident, and includes supporting information:

- Correlated Indicator high-level summary/history information:
    - Summary Count of the number of Correlated Indicators.
    - Graphical Summary depicting “Count of Indicators Over Time”.
- Per Correlated Indicator detail information - including:
    - Entity Name & Details, Entity Type, Metric (the Key Measurement that exhibited anomalous behavior), Indicator Type (type of anomaly detected), Expected Range (the Key Measurement value that the model expected for this Entity/Metric), Observed Value (the Key Measurement value that was observed for this Entity/Metric), Deviation (variance from expected range), and a timestamp for when Correlation of this Indicator began.
    - Graphical representation of the model (recent history), with:
        - annotations to indicate:
            - [Incident Start] - this is when/where an Incident was generated to surface this event. (There are propagation and processing delays that result in [Incident Start] after [Primary Indicator])
            - [Runbook Executes] - this is when/where the associated Runbook is executed to: gather additional data/context from the Customer Environment (Data Sources, and 3rd Party), apply logic/analysis to assess impact/prioritization - and likely cause. It is the result of a Runbook execution that is rendered in Section: [Runbook Analysis] of an Incident.

- [Correlation Starts] - this is when/where Correlation grouped this Indicator into the Detection that formed the basis for this Incident.

- … and tools to navigate through the graphical model-history:
  - [Rewind-button] - expand the graphical model-history view to encompass an additional earlier time-period (in chunks of 6-hours per button press).
  - [FastForward-button] - narrow the graphical model-history view to encompass an more recent time-period (in chunks of 6-hours per button press).
  - [Home-button] - returns to default view of graphical model-history.

- Section: [Runbook Analysis] — Listing of Runbook information - including overall summary information, and per-execution detail information:
  - <Runbook — Overall Summary Information>
    - [Trigger Summary] — This is information generated by Correlation based on the source Entity-type in the Primary Indicator - and is used to identify the appropriate Runbook to associate for execution.
    - [Runbook/History] — This is a pick-list of the Runbook executions associated with this Incident.
      - Typically there is a single Runbook execution associated with an Incident (i.e. the Runbook execution that was triggered when the Incident was first generated).
      - The User can [Rerun] a Runbook over the course of triaging/analyzing an Incident (e.g. to gather a more

current view of the analysis), these additional Runbook executions will appear in the pick-list.

- For each Runbook execution the pick-list entry will contain:
    - Runbook execution determination of [Priority] for this particular run.
    - Runbook Name (the name of the Runbook that was executed for this Incident).
    - Timestamp for when the Runbook execution occurred.

- [Runbook/Execution-status] — This is rendered as a small icon that, when clicked, pops-up a [Runbook Errors, Warnings and Variables] window that provides additional detail with respect to the Runbook execution:
    - [exclamation-in-red-circle] — for Runbook executions that “Completed with Errors”
    - [info-in-green-circle] — for Runbook executions that “Completed”.
- [Runbook/Execution-disposition] — this describes the level of success for a Runbook execution:
    - [Last Run: Completed] — All Runbook nodes/logic executed as expected.
    - [Last Run: Completed With Errors] — There were issues encountered with the Runbook execution. Additional detail regarding those issues may be discovered in [Runbook/Execution-status] icon that pops-up a [Runbook Errors, Warnings and Variables] window.

        - ***NOTE:*** *In these scenarios, the Runbook has typically executed nodes/logic but encountered issues (e.g. if Entities are not configured for monitoring/observability across all Data Sources, then it is possible that the Runbook will query a Data Source for information about an Entity configured on one Data Source but not on the queried Data Source)*
    - [<ellipses>-menu] — Provides access to some Runbook operations:
        - [Rerun Runbook] — Clicking this will cause a new Runbook execution (which will update the history, and have its own associated Runbook Analysis)
        - [Open Runbook] — will open an embedded panel that shows a read-only rendering of the Runbook (i.e. all nodes, their interconnections, flow/logic, and visualizations). This view can help the User understand the path[s] of execution that generated the Runbook Analysis.
            - ***NOTE:*** *the embedded panel also provides a link "Edit" (in upper-right) that will open the Runbook in the Runbook Editor (i.e. read/write access).*
    - ***NOTE:*** *There are rare scenarios where Runbook executions are throttled - in those cases the <Runbook - Overall Summary Information> will contain:*
        - *a [Runbook/History] pick-list that has a null-entry: "No Runbook Analysis Available".*
        - *absence of [Runbook/Execution-status] icon.*

        - *a [Runbook/Execution-disposition] where [Last Run: Unknown]*
- <Runbook - Per-execution Detail Information> - Runbook execution initializes with the Detection/Trigger information supplied by Correlation, and then proceeds through: connected nodes, their interconnections, flow/logic, and finally visualizations. The path(s) through a Runbook for a particular Runbook execution determines what is rendered in the Runbook Analysis section. The Visualization Runbook nodes are the elements that surface the information into the Runbook Analysis Section - and currently support the following Visualization types:
    - [Table]
    - [Pie Chart]
    - [Bar Chart]
    - [Timeseries Chart]
    - [Bubble Chart]
    - [Correlation Chart]
    - [Cards]
    - [Gauges]
    - [Connection Graph]
    - [Text]

Incident Lifecycle

Incidents surface Detections as a report that contains all information gathered over the course of processing through the pipeline and the associated Runbook automation.

When first created, an Incident has [State: New], and the following areas of the Incident are populated with the information gathered by Ingest & Analytics, and Correlation:

- [Priority: Low] <Default>
- [Description]
- [State]
- [Start Timestamp]
- [Ongoing]
- [Primary Indicator]
- [Correlated Indicators]
- [Trigger Summary]

All [State] changes are initiated/managed by the User, and when a User is working an Incident they may decide to update the [State] as follows:

- [State: Investigating] — To indicate that this Incident is undergoing active investigation.
- [State: Closed] — To indicate that this Incident has been closed.

Additionally, over the course of its life, an Incident may undergo a variety of events:

- [Impact Analysis Ready] — Runbook has executed, and Impact assessment has been completed - which populates/updates the following areas of the Incident:
    - [Priority]
    - [Impacts: Users]
    - [Impacts: Locations]
    - [Impacts: Applications]
    - [Prioritization Factors]
    - [Runbook/History]
    - [Runbook/Execution-status]

        - [Runbook/Execution-disposition]
        - Runbook Visualizations
- [Incident Indicators Updated] — The Analytics Pipeline (Ingest & Analytics, and Correlation) have identified a recurrence (i.e. recurrence of Indicators related to existing Incident).
    - [Primary Indicator] (recurrence count)
    - [Correlated Indicators] (recurrence count and/or correlation start)
- [Incident Note Added] — A User has added a Note to the Incident.
- [Incident Note Updated] — A User has updated a Note on the Incident.
- [Ongoing Incident Changed] — Incident has ended ([End Timestamp] is recorded), and the Incident is no longer [Ongoing] - which occurs when:
    - For performance-based Incidents: there has been no recurrence of associated Indicators after a period of time (e.g. 1-hour).
    - For state-based Incidents: some period of time after the last Indicator has entered the expected state.
- [Incident Status Changed] — A User has changed the [Status] of the Incident.

Automation (LogIQ Engine)

The LogIQ engine executes automated investigations (Runbooks) associated with an event (detection/trigger), and attaches the resulting analysis to the incident report (e.g., assessed business impact, supporting data/context,etc.).

The determination of which Runbook to execute is based on the detection/trigger information gathered by correlation (which aligns with system level incident triggers) and any user-defined custom triggers:

- System level incident triggers — Maps a specific Runbook to the supported system level incident triggers:

- Device Down Issue — This trigger is intended to initiate the associated Runbook when an individual device has been identified as unreachable or has rebooted.
    - Triggering entity: Device (with its properties).
- [Interface Performance Issue] — This trigger is intended to initiate the associated Runbook when an interface has been identified as down/unreachable, or is exhibiting performance degradation in terms of congestion, packet errors and packet drops.
    - Triggering entity: Interface (with its properties).
- [Multi Device Down Issue] — This trigger is intended to initiate the associated Runbook when multiple devices have been identified as unreachable at the same location.
    - Triggering entity: List of devices (with their properties).
- [Application Location Performance Issue] — This trigger is intended to initiate the associated Runbook when an application has exhibited possible performance degradation for a number of users at a specific location in terms of response time, retransmissions or lower network usage.
    - Triggering entity: Application and Location.

- User-defined Custom Triggers — Provides a mechanism for Users to refine the system-level Incident Triggers in order to perform more targeted automated investigations (e.g. associate a specific Runbook to investigate Detection/Trigger where [Application: DNS]). The User can refine System-level Incident Triggers by configuring a Custom Trigger based on:
    - Properties (e.g. Application, Application Type, Application Version, Importance, Location, Location_role …)
    - Metrics (e.g. Round Trip Time, Throughput, $ Retrans Packets, %

Failed Connections, User Response Time, …)

- Trigger information (e.g. Entity Type, Triggering Metric, …)

Managing Incidents

Current incidents are listed on the Incidents page. Click an incident in the list to see its details. Use the tools on the Incident Details page to monitor the incident until you're able to change its status to Closed or Dismissed.

On the Incident Details page, you can:

- Change the incident's status: New, Investigating, On Hold, Closed, or Dismissed.
- View the impact summary for the incident: How many users, locations, and applications does it affect?
- Add a note to the incident to document its cause and how it is being addressed.
- View the incident's Activity Log, which lists all the detections and user actions associated with it.
- Share the incident with other users by copying the link to the incident details and pasting it into a message of your choice.
- View the latest runbook analysis of the incident.

Riverbed IQ Pages

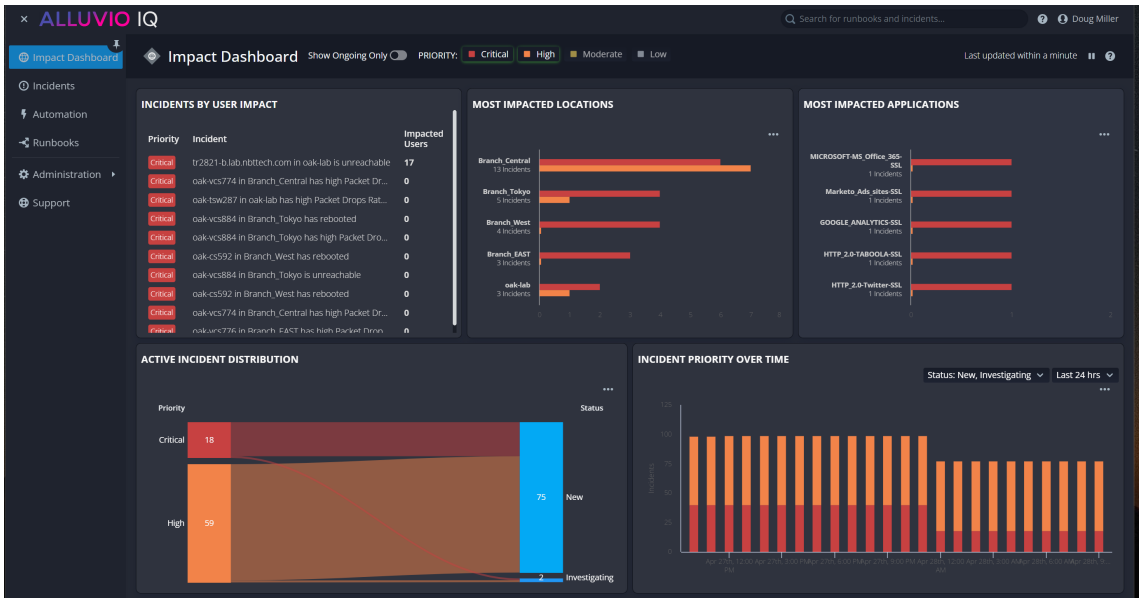
Riverbed IQ furnishes these pages for identifying and investigating events of concern in your network:

- Impact Dashboard
- Incidents
- Incident Details
- Runbooks
- Runbook Editor
- User Management

Impact Dashboard

The Impact Dashboard shows which resources in your network are affected most by current incidents. Click any object in any view to see the corresponding incident's details on the Incidents page.

The Impact Dashboard appears by default when you launch Riverbed IQ, and can be displayed at any time by clicking Impact Dashboard in the Navigation menu at the left side of the Riverbed IQ window.



The Impact Dashboard provides these views of your network resources:

- Incidents By User Impact — The current incidents affecting the largest numbers of individual users.
- Most Impacted Locations – The recognized locations affected by the largest number of current incidents.
- Most Impacted Applications – The specific applications affected by the largest number of current incidents.
- Active Incident Distribution – Current incidents summarized by priority and status.

- Incident Priority Over Time – Current incidents shown over a recent time interval, indicating when incidents have occurred, according to the priorities displayed.

The data displayed in the views can be filtered by Priority, using the selectable options at the top of the page. The data is updated frequently (every minute, by default), but you can pause the updates if you want to, using the Pause/Start control at the top of the page.

Incidents Page

The Incidents page lists occurrences in your network that have met criteria justifying further investigation (incidents). Click in an incident's row to show its details in a preview pane, or click the link in the incident's Description field to view those same details on their own dedicated page. The incident details include the incident's primary indicator and any correlated indicators, as well as analysis resulting from the runbook associated with the incident type. Display the Incidents page by clicking an element in the Impact Dashboard or by clicking **Incidents** in the Navigation menu at the left side of the Riverbed IQ window.

You can filter the incident list coarsely by **Ongoing** incidents only (this is the default behavior), by Status, by Priority, and by name (partial or full) and time. The Filters pane at the left enables you to search for incidents based on narrowly specified criteria.

This information is shown for each incident:

- Add to Watchlist — Click to include the incident in the Watchlist. Click again to remove the incident from the Watchlist.
- Priority
- Status — You can change the status by choosing **Change State** from the ... menu at the right end of the row.
- Description — Click this link to show the incident details on a dedicated page.
- Detected On
- Started
- Duration
- Impacted Users
- Impacted Locations

- Impacted Applications
- Runbook Status indicator
- **...** menu: **Open Incident**, **Open Incident In New Tab**, **Change State**

Automation Management Page

Click **Automation** > **Automation Management** in the Navigation pane to display the Automation Management page. The Automation Management page collects and shows the set of automations that are configured for your Riverbed IQ installation, organized by trigger category and type.

The Automation Management page shows trigger categories and types as a set of cards, one card for each trigger type:

- New Incident Triggers — The set of automated triggers configured for network phenomena comprising outages, congestion, anomalies, etc.
- Incident Lifecycle Triggers — The set of automated triggers configured for tracking changes to the incidents themselves, comprising status changes, priority changes, annotations, etc.
- External Triggers — The set of automated triggers configured for external API activity.

For each category, click the **+** to add an automated trigger for that category. This launches the Add an Automation wizard.

Each card shows the number of automations for that trigger type. Click the card to open a list of the corresponding automations; the automations are shown in the user-defined order in which they are executed according to their conditions being matched. The list shows these fields for each automation:

- Trigger — This is one of the built-in triggering entities, as listed in the Runbook Editor: Application Location Performance Issue, Device Down Issue, Interface Performance Issue, Multi-Device Down Issue, or Webhook.
- Execute On Trigger — This shows the name of the runbook that is associated with the trigger type. Click the link to open the runbook in the Runbook Editor.

- Description — An informative summary of what the associated runbook will do in response to the trigger.
- Last Updated By — The user who modified this trigger/runbook mapping most recently.
- Last Updated On — The timestamp for when this trigger/runbook mapping was modified most recently.
- Enable Automation — Execute the mapped runbook in response to the trigger. This is enabled by default; disable it only if you do not want a runbook to execute in response to the trigger.
  See the description of the **Allow Automation** control on the Runbooks page for details of enabling a runbook to be mappable to a trigger.
- **...** — Choose from:
  - **Edit** — Open the Edit an Automation wizard to change the automation definition.
  - **Duplicate** — Copy this automation definition.
  - **Delete** — Remove this automation definition.
  - **Change Order** — Specify this automation's precedence for the trigger type.
  - **Open Runbook** — Open the currently mapped runbook in the Runbook Editor.

Runbooks Pages

The various Runbooks page are for working with the different runbook types in your configuration:

- *Incident Runbooks* — Automated investigations that execute in response to triggers.
- *Lifecycle Runbooks* — Runbooks that track the status of ongoing incidents and execute actions when the incident status changes.
- *On-Demand Runbooks* — Investigations that you execute manually rather than automatically.
- *Subflows* — Sequences of runbook nodes that execute discrete actions and that are saved as independent reusable components for inclusion in other runbooks.

Access these pages at any time by clicking the corresponding item in the Navigation pane at the left of the Riverbed IQ window.

Each Runbook page lists all the runbooks (or subflows) of that type that exist in your configuration (built-in and custom), enabling you to edit existing ones, create new ones, import and export them via JSON file, and delete any that you no longer need.

Filtering the List of Runbooks

You can control which runbooks are listed by choosing a filter; each filter shows how many runbooks it comprises:

- **All Runbooks** — Show all existing runbooks. This is the default.
- **Automation Allowed** — Show only those runbooks that have been enabled to be mapped to trigger types.
- **Created By Me** — Show only those runbooks created and saved using the current user account.

- **In Error** — Show only those runbooks marked with an error state. These runbooks cannot be be set to Active, and cannot be executed until all errors are corrected.
- **Built-In** — Show only those runbooks provided by Riverbed IQ application.

Working On an Existing Runbook

For any runbook, click on its row or click the **>** at its left to show its definition as a sequence of nodes. Doing this also makes several controls available:

- **Edit** — Open the runbook in the Runbook editor to change it.
- **Export** — Write the runbook definition to JSON file.
- **Duplicate** — Make a copy of the runbook.
- **Delete** — Use with caution. This will prompt you for confirmation, then will permanently remove the runbook from the system. There is no undo, once the runbook is deleted.

Creating a New Runbook

Two controls reside at the top of the page:

- **New** — Open the Runbook Editor to create a new runbook.
- **Import** — – Create a new runbook by importing a JSON file of a previously exported runbook definition.

Runbook List Columns

These fields are shown for each runbook on the Runbooks page. For each column, you can sort by it, and you can click on the Funnel icon to search or filter by a value.

- *Runbook* — The runbook's name. This may be user-defined, or auto-generated if no user has renamed it.

- *Description* — A brief, user-defined explanation of what the runbook is used for.
- *Trigger* — The type of trigger that launches the runbook.
- *Entity Type* — For on-demand runbooks only, the type of network object for which the runbook will execute.
- *Built-In* — A checkmark is shown if the runbook is provided by Riverbed IQ.
- *Last Updated* — The timestamp from when the runbook was most recently saved.
- *Last Updated By* — The username of the account that saved the most recent change to the runbook.
- *Allow Automation* — For all trigger types mapped to the runbook, toggle the runbook On to enable it or toggle it Off to disable it. This control is inaccessible if the runbook has any errors.
  A runbook must be On to be mappable to a trigger type in the Automation page; if it is Off, it will not be listed in the Edit Mapping dialog as available to be mapped. If a runbook is Off, it will not execute when a mapped trigger fires, but an incident still will be created; this can be useful if you want to modify the runbook and don't want it to execute in an experimental or unvalidated state. Custom runbooks are Off by default.
- **Run** button — For on-demand runbooks only, execute the runbook now. The runbook's output will be listed on the Runbook Analyses page.
- **...** pop-up menu — Additional controls for managing runbooks. Most are also available from other places when working with runbooks.
  - **Export** — – Output the runbook definition to a JSON file that can be imported subsequently to create a new runbook.
  - **Edit** — Open the runbook in the Runbook Editor to change it.
  - **Duplicate** — Copy the runbook.

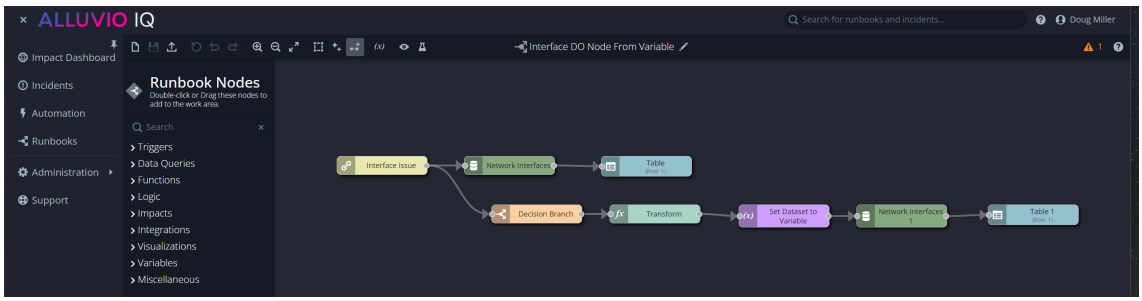
- **Delete** — Use with caution. This will prompt you for confirmation, then will permanently remove the runbook from the system. There is no undo, once the runbook is deleted.
- **Test Run** — Execute the runbook's logic immediately, without having to wait for a corresponding trigger.
- **Preview Output** — Show what data the runbook will generate after it executes in response to a trigger.

Runbook Editor/Subflow Editor

Note: Everything in this description that refers to a runbook also pertains to a subflow, except where noted. The Subflow Editor is a slightly pared down version of the Runbook Editor, used for creating and modifying subflows, which are sequences of runbook nodes that execute discrete runbook activities and that can be saved as independent reusable components for inclusion in other runbooks.

Create and edit runbooks using the Runbook Editor. This shows each runbook as an assembly of runbook nodes arranged in a sequence that defines the runbook's execution. The runbook nodes enable you to define the runbook in discrete logical blocks, with each node performing some action related to the trigger and subsequent queries to Riverbed IQ's data ocean.

Open the Runbook Editor from the Runbooks Page by choosing **Edit** from the **...** pop-up menu at the right of a runbook's name, or, if the runbook's definition is shown, by clicking **Edit** above the preview.



See Creating Custom Runbooks Using the Runbook Editor for how to create a new runbook and edit an existing one.

Tool Bar

The tool bar across the top of the Runbook Editor provides controls for working on the current runbook:

-  **New Runbook** — Begin defining a new runbook on a blank canvas.
-  **Save** — Keep the latest changes to the current runbook. The Save Runbook dialog will appear, prompting you to type or change a name and description for the runbook. The name and description will be visible in the list of runbooks on the Runbooks Page.
-  **Export** — Output the runbook definition to a JSON file that can be imported subsequently to create a new runbook.
-  **Reset** — Discard all unsaved changes and revert to the last saved definition. For a new, unsaved runbook, this discards everything.
-  **Undo** — Roll back the previous unsaved change to the runbook. You can click this repeatedly to roll back multiple unsaved changes sequentially.
-  **Redo** — Re-implement the previous undone unsaved change to the runbook. ("Undo the last undo.") You can click this repeatedly to re-implement multiple undone changes sequentially.
-  **Zoom In** — Magnify the canvas, focusing on nodes that are selected.
-  **Zoom Out** — Back away from the canvas to see more of it.
-  **Fit to Screen** — Adjust the canvas to see the entire runbook definition filling the pane.
-  **Multi-Select** — Enable multi-select for your next action with the cursor, so you can drag a box around multiple objects on the runbook

canvas to select them all at once. This is not sticky; you'll need to click this each time you want to select multiple objects. You can copy and paste multi-selected nodes between runbooks.

-  **Beautify** — Make the arrangement of nodes on the canvas neat and orderly. This will align nodes on the canvas horizontally and vertically to make the definition easier to read.
-  **Auto-Connect** — Create connections between nodes automatically as they are dragged onto the canvas and arranged in sequence with other nodes. If this is disabled, you'll need to connect nodes manually.
-  **Edit Variables** — Open the Variable Definition Editor for creating and modifying variables for manipulating data in runbooks.
-  **Preview Output With Mock Data** — (Not available for subflows.) Display the tables and charts that will result when this runbook executes, but with artificial data.
-  **Execute a Test Run** — (Not available for subflows.) Cause the runbook to execute immediately and generate output based on currently available data, and display the results.

Runbook Node Palette

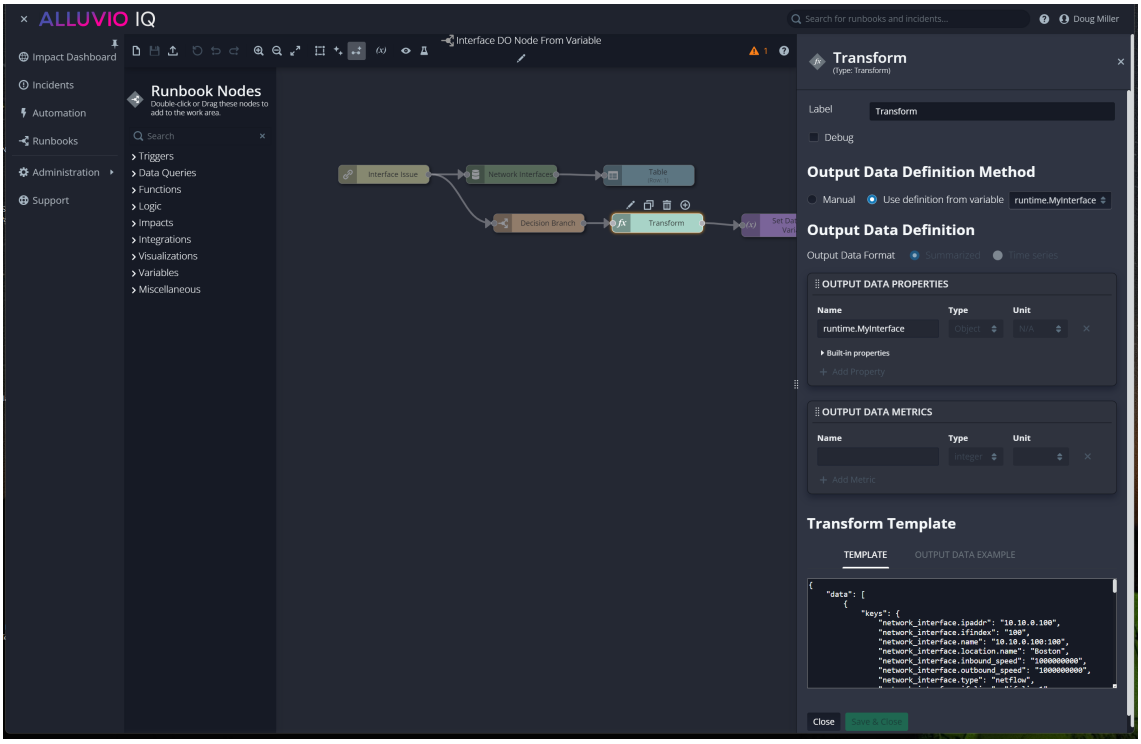
The palette lists all the possible runbook nodes. Select a node and drag it to the canvas, arranging it in sequence relative to other nodes that are on the canvas already. Type text in the palette's Search panel to filter the nodes by name, or to find a specific node.

Canvas

This is the main pane of the Runbook Editor, where you place and connect nodes to define the runbook.

Properties Blade

Select a runbook node on the canvas to display its properties at the right side of the canvas. You can modify the node's properties as necessary to configure the runbook's behavior.



Runbook Analyses Page

The Runbook Analyses page lists the outputs of on-demand runbooks that have executed in the last 30 days. Click **Runbook Analyses** in the Navigation pane to display it.

The Runbook Analyses page shows these properties for each runbook output that it lists:

- *Runbook Analysis* — The description of the output from an on-demand runbook execution. This defaults to the name of the runbook itself. Click on the description to view the output.
- *Runbook* — The name of the on-demand runbook that executed to generate this output.
- *Entity Type* — The category of entity that the runbook was executed for.
- *Execution Method* — For on-demand runbooks, this is Manual. (Other values can be Webhook and Correlation.)
- *Time* — The date and time the runbook output was generated.
- *Executed By* — The user account that executed the runbook to generate the output.
- *Status* — This will be InProgress, Completed, CompletedWithErrors, or Failed.

The Runbook Analyses page also provides these controls for modifying the set of runbook outputs that are shown on the page:

- Filters panel — Select filters to refine the set of runbook outputs that are displayed. Click **Apply Filters** to make your selection take effect on the page; click the **Save** icon if you want to hold on to a filter definition to reuse it later (using **Load Saved Query**). You can filter based on Output Name, Status, Entity Type, Execution Method, and Executed By.

- Search panel — Type a string to find matching instances in the list of runbook outputs. The search string must begin with text and does not support the use of "*".
- Run Runbook — Open the Run Runbook dialog to specify a runbook to execute now. The runbook's output will be listed here on the Runbook Analyses page.
- Time Period — Specify the window of time for which to show runbook outputs. The page lists outputs for the last 30 days by default.
- Column Chooser — Click this to change the set of properties shown for the runbook outputs.

Runbook Scheduling page

Click **Runbook Scheduling** in the Navigation pane to open the Runbook Scheduling page. You can schedule up to 200 *on-demand* runbooks to execute at times that you configure.

Although on-demand runbooks are named for their ability to be executed arbitrarily at your discretion (distinct from incident runbooks and lifecycle runbooks that Riverbed IQ executes in response to built-in logic), it is possible to execute on-demand runbooks on user-configured schedules. To clarify, "on-demand," when referring to runbooks, means that they are not dependent on IQ's built-in logic for executing runbooks in response to specific incident or lifecycle criteria; rather, "on-demand" means user-initiated, whether that's interactively and opportunistically for recently observed situations, or on a configured regular schedule that's independent of incident and lifecycle occurrences.

The Runbook Scheduling page shows this information:

- *Runbook* — The on-demand runbook to be executed by the schedule.
- *Schedule Description* — A brief explanation of the schedule.
- *Schedule* — The name of the schedule associated with the runbook.
- *Inputs* — A summary of the inputs used by the runbook for the schedule.
- *Next Run* — The date and time of the runbook's next scheduled execution.
- *Enabled* — Toggle the schedule to be enabled or disabled.
- *Actions* — Click the **...** to show a menu of actions for the scheduled runbook: **Edit** or **Remove**.

The list of runbooks on the Runbook Scheduling page can be filtered and searched.

Creating a Schedule

To create a schedule for executing an on-demand runbook:

1. Click **Schedule Runbook** near the top right corner of the Runbook Scheduling page to open the Schedule Runbook wizard.
2. Choose an existing on-demand runbook to schedule and click **Next**.
3. Specify the inputs for the runbook. These will vary according to the particular runbook that you've chosen to schedule. Make sure to specify the primary entity for which the runbook will execute, as well as any other inputs that the runbook requires. Click **Next** when you've done this.
4. Specify the schedule to use. This can be an existing saved schedule, or you can define a new one. Specify a date and time to begin and define the interval at which the schedule will repeat. Click **Next** when you've done this.
5. Review the schedule definition and click **Submit** to finalize it and add it to the list on the Runbook Scheduling page. The scheduled execution will be visible on the Runbook Analyses page, also.

Usage Considerations

Be mindful of these considerations when working with scheduled runbooks:

- You cannot delete a schedule that is used by one or more runbooks.
- You cannot delete a runbook that is scheduled.
- When you edit a runbook with an active schedule, the Save dialog will inform you of the potential effects of the change and notify you that the change will affect only future executions of that runbook.
- Changing a runbook's input definition will cause its scheduled executions to have unset or missing inputs (remember, inputs are saved with the schedule). You need to update the schedule explicitly, or the scheduled executions will continue erroneously without the new correct inputs.

Integrations Library Page

Riverbed IQ's Integrations Library furnishes numerous turn-key integrations, ready to be installed and configured, that enable you to integrate Riverbed IQ with third-party vendors such as ServiceNow and Microsoft Azure, among many others. These integrations increase the power and ease of use of IQ's third-party application support through integration-specific subflows that can be used in runbooks after an integration is installed.

Click **Integrations Library** in the Navigation pane to display the Integrations Library page. The page has two tabs, one showing cards for Installed integrations, the other showing cards for Available (uninstalled) integrations. Once installed, an integration's card moves from the Available tab to the Installed tab. Note that some integrations are marked with a Beta tag. The Search panel at the top of the page enables you to query for integrations across the two tabs, based on name and description.

Installing an Integration

To install an integration:

1. Open the Integrations Library's Available tab and find the card for an integration you want to install.
2. Click the **Install** button on the card to open the Installation dialog. The Installation dialog comprises two tabs: Description and Configuration. The Description tab summarizes the integration support and provides links to the third party's own information about its product. The Configuration tab provides the **Add a Connector** button for specifying the set of authentication details for communicating with the third-party product. At least one connector must be configured in order to install the integration because the authentication details will be used as input for the integration-specific subflow that will be added.
3. Click **Add a Connector** to display the Add a Connector wizard.

4. On the Basic Details page, specify a name for the connector and supply information for initiating authentication with the third party. The information that is requested will vary according to the third party's particular requirements, but will include the Authentication Method to be used. For most integrations, the Authentication Method is set to a default that cannot be changed. Click **Next** to advance to the Authentication Parameters page.
5. The Authentication Parameters page prompts you to supply the specific authentication details required by the third-party tool and the authentication method. Again, this information will vary according to the third party's particular requirements. Click **Next** to advance to the Test Query page.
6. Define a test query that is appropriate for the authentication method you are using. Click **Next** to advance to the Verify Response page.
7. Click **Run Test Query** to execute the query you defined in the last step. Click **Next** to advance to the Review page to see a summary of the connector's configuration and status.
8. Click **Save** to finalize the connector definition or **Back** to change some portion of it.
9. On the Installation dialog, click **Install** to execute the installation process.
10. Once the installation is complete, one or more subflows corresponding to the integration will become available in the Runbook Node palette and an authentication profile will be created for each connector you defined.

Modifying an Integration After Installation

Once an integration has been installed, you can click **Configure** on the integration's card in the Installed tab to display the Configuration dialog. Here, you can click **Uninstall Integration**, if you wish to do that (note that uninstalling an integration also removes its integration-specific subflows, which will affect any runbooks in which those subflows are used). Click the Configuration tab to access the integration's connector(s); for each connector, you can click the

Gear icon to open the Add a Connector wizard and change the connector's definition.

Analytics & Threshold Configuration page

Click **Administration > Analytics & Threshold Configuration** in the Navigator pane to display the Analytics & Threshold Configuration page. This page enables you to view and configure the settings that control when and how Riverbed IQ recognizes network behavior that constitutes an indicator, the fundamental building block of an incident.

A policy controls how the Riverbed IQ data pipeline creates indicators, and, therefore, incidents. Upon installation, IQ has a number of built-in policies in place that are configured according to best practices for monitoring networks and which measurements merit being called to your attention in the form of an incident. What merits being called to your attention is a subjective, user-/network-defined decision, so IQ provides the ability to override some policies with user-configurable settings. A single policy applies a rule to a measurement on a network entity.

The Analytics & Threshold Configuration page comprises three sections: Devices, Interfaces, and Applications. In each section, there is a set of relevant metrics that can be enabled/disabled, and, in the case of thresholds, configured:

Network Devices

- Device Status [Status Change]
- Device Uptime [Uptime Reset]

Network Interfaces

- Interface Status [Status Change]
- In Packet Drops Rate [Baselining]
- In Packet Error Rate [Baselining]

- In Utilization [Baselining]
- In Utilization [Threshold]
- Out Packet Drops Rate [Threshold]
- Out Packet Error Rate [Threshold]
- Out Utilization [Baselining]
- Out Utilization [Threshold]

Application + Location (i.e., application “MS-Exchange” in location “Branch Office Denver”)

- Activity Network Time [Dynamic Threshold]
- % Failed Connections [Threshold]
- MOS [Threshold]
- Page Load Network Time [Dynamic Threshold]
- % Total Retrans [Threshold]
- Throughput (VoIP) [Baselining]
- User Response Time [Baselining]

IQ has several types of rule for determining whether a measurement on a network entity is worthy of attention or not. Violations of these rules create indicators, which are then correlated into incidents. In addition to the configuration options listed for each type of rule, there are some options that can be configured on every rule.

Rule Type	Description	Configuration Options
Change	Detects a change in the measurement, for example the status of an interface going from “OK” to “Down”. This is used in interface and device status rules.	*OK* value — A change back to this value will not create an indicator. For

Rule Type	Description	Configuration Options
		example, an interface with status “admin down” that goes back to “OK” will not create an indicator or incident.
Always Increasing	Tracks a measurement and makes sure its value is always increasing. This is used to track Device Uptime measurements.	Not configurable.
Threshold	This looks for measurements above a certain value or below a certain value. For example, indicating on applications with a MOS score of < 3.5, or on interfaces with an inbound utilization of > 80%.	• *Upper Threshold* — Create an indicator if the measured value is greater than the upper threshold. • *Lower Threshold* — Create an indicator if the measured value is lower than the lower threshold.
Baseline	This uses machine learning techniques to characterize the measurement being tracked over time. If the measurement deviates excessively from what is con-	• *Noise Floor* — This value is given in the

Rule Type	Description	Configuration Options
	sidered normal, an indicator is created.	units of the measurement being tracked. The deviation of the measurement from normal must be at least this value in order to produce an indicator. For example, a noise floor value of 500ms of Response Time means there will never be an indicator produced where the absolute difference between the measurement and the expected value is less than 500ms . • *Upper Percent Change* — The

Rule Type	Description	Configuration Options
		multiplier on the expected value of the measurement that determ-ines what the allowed upper value of the measurement is. For example, an upper % change value of 1.2 on an expected value of 10 means that measured values over 10 * 1.2 = 12 will create an indic-ator. • *Lower Percent Change* — The multiplier on the expected value of the measurement that determ-ines what the allowed lower value of the

Rule Type	Description	Configuration Options
		measurement is. A value of 0.8 on an expected value of 10 would produce an Indicator on measurements below 10 * 0.8 = 8 . The noise floor and the upper/lower % change can be applied together. The setting providing the maximum deviation from normal is used to determine whether the rule has been violated.
Dynamic Threshold	This uses machine learning and statistical techniques to characterize the measurement being tracked over time. If the measurement deviates excessively from what is considered normal, an indicator is created.	• *Noise Floor* — This value is given in the units of the measurement being tracked. The deviation of the measurement from

Rule Type	Description	Configuration Options
		normal must be at least this value in order to produce an indicator. For example, a noise floor value of 500ms of Response Time means there will never be an indicator produced where the absolute difference between the measurement and the expected value is less than 500ms . • *Percentile* — Only create indicators for measurements that are this percentile (0-100) outside of normal. Higher percentiles

Rule Type	Description	Configuration Options
		mean fewer indicators. The noise floor and the percentile can be applied together, like the baseline rule; also like the baseline rule, the maxiumum deviation from normal is used.

Editing a Static Threshold Value

To edit a static threshold value, click **Edit** to open the Edit Static Threshold dialog box.

Specify the value at which to generate an indicator from this threshold and the number of measurements to use. You can create an indicator based on a single measurement of the metric, for some number of consecutive measurements, or for N out of M measurements. For example, specifying “2 out of 3” means that the rule must violate for 2 of the last 3 measurements for an indicator to be created. N and M default to 1. M has a maximum value of 10. N must be <= M. If M > 1, N must be > 1 also.

User Management Page

The User Management page enables you to administer user accounts, roles, and domains for Riverbed IQ.

Click **User Management** in the Navigation pane to display the User Management page.

Click **Invite User** to send Email to a specified account, asking the owner to register with Riverbed IQ as a user. The Email account must be a member of a previously authorized domain. You can also assign a role to the specified account: Admin, Write, or Read.

Click **Manage Authorized Domains** to add or delete domains that your organization trusts for authorizing users and sending and receiving Email. Only Email accounts that belong to authorized domains can be used to invite users to access Riverbed IQ.

Data Sources Page

This page enables you to manage Riverbed IQ's connections to the systems that forward it network data and incidents. The page lists existing Riverbed Edge instances and provides the means for adding others, as well as the means to manage individual data sources associated with each Edge.

The Data Sources page provides these controls:

- **Add a Riverbed Edge** — Click this to display the Add a Riverbed Edge wizard.
- *Riverbed Edge*
- *Type* — AWS, Azure, or ESXi, as specified at the Edge's creation.
- *Status*
- *Data Sources* — The data sources associated with the Edge, which have their own set of properties and controls:
    - Data Source — The data source name.
    - Type — The data source type (AppResponse, NetIM, or NetProfiler).
    - Hostname or IP — The fully qualified hostname or IP address for the data source.
    - Info — A summary of system status and software versions for the data source.
    - Data Ingestion — This shows when the Edge last received data from the data source, and indicates also whether the data source is preferred. If you wish to change this, click the **Change** button to open the Change Preferred Data Source dialog, which enables you to specify whether to prefer AppResponse or NetProfiler.
    - Enabled — Indicates whether the data source is enabled to stream data to the Edge. This is set in the Edit the Data Source dialog.

        - **...** menu — Additional commands for administering each data source.
            - **Edit** — Open the Edit the Data Source dialog to change settings for the data source such as its IP address, type, and protocol.
            - **Set Credentials** — Add the user account credentials for the data source to the Edge. Click this to display the Set Credentials dialog; type the username and password for accessing the data source, and click **Apply**.
            - **Delete** — Remove the data source from the Edge.
- **...** menu — Additional commands for administering each Edge.
    - **Edit** — Open the Edit the Riverbed Edge wizard to change the definition.
    - **Connect a Data Source** — Open the Connect a Data Source dialog to specify a data source to use with the Edge.
    - **Obtain VM Image** — Obtain the Ubuntu cloud image.
    - **Download Cloud-Init Script** — Obtain the cloud-init script that configures the Edge.
    - **Delete** — Remove this Edge from Riverbed IQ.

License and Usage Page

Click **Administration** > **License and Usage** in the Navigation pane to display the License and Usage page. This page displays your consumption of metrics over time. The current calendar month is shown by default; click the pulldown at the upper right of the chart to show metric consumption for a previous month.

Note that Riverbed Technology licenses metric consumption in packs of 100,000. The details of the metric licensing policy are available here.

Third Party Authentication Page

Click **Administration** > **Third Party Authentication** in the Navigation pane to display the Third Party Authentication page. This page enables you to manage user accounts and login credentials for Riverbed IQ to communicate with other applications.

The Third Party Authentication page shows the following information for each user account:

- Profile Name
- Description
- Method
- Created By
- Created On
- Last Updated By
- Last Updated On
- Enabled
- Verified
- References
- **...** menu
    - **Edit** — Open the Authentication Profile wizard to change this authentication profile's definition.
    - **View Referencing Runbooks** — Show a list of runbooks that use this authentication profile.
    - **Delete** — Remove this authentication profile. You will be warned if the authentication profile is used by any runbooks.

Click **Add an Authentication Profile** to display the wizard of the same name and create a new profile.

API Access Page

Click **Administration** > **API Access** to display the API Access page. This page enables you to manage OAuth2 clients for accessing Riverbed IQ's APIs from other applications in order to execute runbooks and obtain analysis outputs. All OAuth clients have write access to Riverbed IQ.

The API Access page provides these controls:

- **Create OAuth Client** — Display the Create OAuth Client dialog to define a new API client; type a name and a secret expiration interval. Riverbed IQ generates a client ID and a client secret for you. ***Once you close this dialog, the client secret will never be shown again. Copy the client secret now (note the Copy icon immediately after it) and move it to a secure place immediately.***
- *Name* — The user-defined name of the OAuth client.
- *Earliest Client Expiration* — The most immediate expiry date for any secret associated with this OAuth client.
- *Client ID* — This ID is autogenerated by Riverbed IQ and assigned to this OAuth client. Click the Copy icon to copy this ID to the clipboard to be able to use it elsewhere.
- *Number of Client Secrets* — The count of secrets associated with this OAuth client.
- **...** menu — Click on the "three dots" menu to access these additional commands for managing the OAuth client secrets:
    - **Manage Client Secrets** — Open the Manage Client Secrets dialog to list the existing secrets for this OAuth client. From here, you can add or delete secrets. To replace a secret, add a new one, so that two are active simultaneously until you revoke one of them. This enables you to replace a secret without incurring downtime.

- **Delete OAuth Client** — Remove this OAuth client permanently. A deleted client cannot make API requests.

Monitoring Health Page

This page summarizes the status and performance of your Riverbed IQ instance.

Overall Status

The Overall Status cards show you at a glance the performance of Riverbed IQ component systems:

- Cloud Status — The overall status of this Riverbed IQ instance. This will show if the overall status of the component systems is OK, or if a component system is impaired in some way.
- Riverbed Edges — The state of connectivity to the Riverbed Edge instances and their connectivity data sources. This comprises several health requirements:
    - Connectivity to Riverbed Edge from the cloud.
    - Riverbed Edge is consistently receiving streamed data from all configured and enabled data sources.
    - Riverbed Edge is able to connect to all enabled data sources.

  The status will be one of:
    - *Operational* indicates that all Riverbed Edges meet all health requirements.
    - *Partially Operational* indicates that at least one Riverbed Edge does not meet all health requirements.
    - *Not Operational* indicates that either no Riverbed Edge is reachable, or no data source connects successfully to an Riverbed Edge.
- Data Ingest — This summarizes the rate of metrics being streamed to your Riverbed IQ instance over time.
- Analytics — This summarizes the rate of indicators being generated over time.

- Incident Generation — This summarizes the number of incidents created and updated over time.
- Runbooks — This summarizes the number of runbooks executing over time.

Performance Charts

These charts show aspects of Riverbed IQ performance over time; floating the cursor over a graph shows relevant values at that specific time. When a legend is shown that catalogs the types of data that are graphed, click on one or more types to filter the graph accordingly.

- Data Ingest tab — This shows the number of metrics streamed from data sources over time.
- Analytics tab — This shows the number of indicators generated by Analytics against a threshold or a baseline.
- Incident Generation tab — This shows the number of incidents created over time.
- Runbooks tab — This shows how many runbooks have executed to completion or executed without finishing, as well as which runbooks have executed over time.

Incident Search

The Filters pane on the Incidents page enables you to define narrow criteria to find a specific set of incidents. The pane provides a text panel for typing a finely constrained search string, and also provides a complete set of filters, enabling you to refine results based on specific parameter values.

The Filters pane provides these tools for constraining and refining your search:

- Text panel — Type your search string, prepending text to exclude from the match with "-".
- Search Result count — This shows the number of individual results returned by the combination of the specified search string and the selected filters. This number will change as the search string or the filters are changed.
- Basic filters — Filter incidents based on their type (**All Incidents**, **Devices**, **Interfaces**, **Applications**, **Locations**) and date.
- Detailed filters — This lists all the parameters and all the values present in the total set of search results. Select one or more parameter/value combinations to refine the search results.

Using the Text Panel

The Text panel provides a means of isolating incidents that meet specific criteria:

- Type a simple string in the panel.
- Type text enclosed in double quotation marks, " ", to search for a phrase.
- Type "-" to exclude a term from the match criteria.

Support Page

The Support page enables you to contact Riverbed Technology directly to report a problem or request assistance specific to your use of Riverbed IQ.

You can access the general Riverbed Support web site at https://support.riverbed.com/

Built-In Runbooks

Riverbed IQ furnishes these runbooks, built in to the product:

- Interface Analysis
- Multi-Device Down Analysis
- Application Analysis
- Device Analysis

Use the Duplicate command to copy one or more of these runbooks to use them as templates for creating custom runbooks.

Runbook Nodes

Runbook nodes are the individual logical components, or automated actions, that make up runbooks. Runbook nodes are assembled in sequence to define an automated investigation and generate output to inform your investigation of an incident in response to a trigger. Each runbook node has a set of properties that control its behavior in a runbook, and these properties can be configured separately for each instance of the node that is used.

Runbook nodes are accessed in a palette in the Runbook Editor, where they are organized into categories. The palette lists the runbook node categories in approximate order of how they should be assembled in sequence (first a trigger, then a data query, then a logic operation, then an impact or a visualization), but this is broadly advisory, and a complex runbook may have nodes from different categories arranged in myriad ways.

Triggers

- Interface Issue
- Device Issue
- Application Issue

Data Queries

- Applications
- Devices
- Devices [CPU Utilization]
- Devices [Memory Utilization]
- Devices [Disk Utilization]
- Devices [Availability]
- Network Device [Configuration Changes]
- Hosts

- Client Hosts
- Server Hosts
- Client-Server Pairs
- Locations
- Network Interfaces
- Network Interfaces (Error Metrics)
- Protocols
- DSCPs
- Client-Server Location Pairs
- Client-Server Pairs and Protoport
- Azure Monitor

Function

- Aggregator

Logic

- Decision Branch

Impacts

- Set Incident Priority
- Tag As Affected Applications
- Tag As Affected Locations
- Tag As Affected Users

Visualizations

- Table
- Pie Chart

- Bar Chart
- Time Series Chart
- Cards
- Gauges
- Connection Graph

Miscellaneous

- Comment

Test

- Test

Runbook Node Categories

Runbook nodes are organized into these categories:

Triggers

Start the runbook with a single trigger only.

Data Queries

Get data about the trigger and forward it to other nodes in the runbook.

Functions

Aggregate or transform data forwarded from a preceding node.

Logic

Add conditions to branch the runbook.

Impacts

Mark network resources as affected by the incident.

Integrations

Work with other applications, such as HTTP.

Visualizations

Show data in a chart, graph, or note.

Variables

Define variables for manipulating data.

Miscellaneous

Annotate the runbook with comments.

Interface Issue

Execute this runbook based on an interface's behavior.

Properties

Type an informative name for the triggering entity in the Label field. "Interface Issue" is the system-provided default.

Data Query Reference Time

Specify how Data Query nodes that follow this node in the runbook will assess time; according to the origination of the primary indicator, plus an offset, or according to the time that the runbook executes.

Device Issue

Execute this runbook based on a device's behavior.

Properties

Label – Type an informative name for the triggering entity. You can keep the system-provided default of "Device Issue" if you wish.

Data Query Reference Time

Specify how Data Query nodes that follow this node in the runbook will assess time; according to the origination of the primary indicator, plus an offset, or according to the time that the runbook executes.

Application Issue

Execute this runbook based on an application's behavior.

Properties

Label – Type an informative name for the triggering entity. You can keep the system-provided default of "Application Issue" if you wish.

Data Query Reference Time

Specify how Data Query nodes that follow this node in the runbook will assess time; according to the origination of the primary indicator, plus an offset, or according to the time that the runbook executes.

Location Issue

Execute this runbook based on network behavior relative to a place. This can be a site or another collection of network resources as defined by the data source.

Properties

Label – Type an informative name for the triggering entity. You can keep the system-provided default of "Location Issue" if you wish.

Data Query Reference Time

Specify how Data Query nodes that follow this node in the runbook will assess time; according to the origination of the primary indicator, plus an offset, or according to the time that the runbook executes.

Webhook

Execute this runbook based on a webhook from another application.

Properties

Label – Type an informative name for the triggering entity. You can keep the system-provided default of "Webhook" if you wish.

Data Query Reference Time

Specify how Data Query nodes that follow this node in the runbook will assess time, according to the time that the runbook executes. Note that *Primary Indicator Start* is not used with Webhook triggers.

Input Node

Specify the details an on-demand runbook needs to be able to execute. The Input node requires you to explicitly supply these details to give runbook the context it needs to be able to execute, information that an incident runbook acquires implicitly from its Trigger node.

Label — Type an informative name for the Input node. You can keep the system-provided default of "Input" if you wish.

Starting Entity

Specify the object for which the on-demand runbook will execute by choosing it from the dropdown menu. This determines the Primary Entity that will need to be identified in the Run Runbook dialog when the runbook is executed.

Additional Parameter Values Received From Caller

Specify the names of variables that correspond to parameters that will be used to execute the runbook. (Each variable needs to be defined separately.) When the runbook is executed, these will be listed in the Run Runbook dialog, and each will require a value to be supplied.

Applications Query

Find all applications matching the input filters, returning selected metrics for each. By default, this query is based on the Throughput metric.

Properties

Type a descriptive name for the node in the Label field. ("Applications #" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Devices Query

Find all network devices matching the input filters, returning selected metrics for each. By default, this query is based on the Throughput metric.

Properties

Type a descriptive name for the node in the Label field. ("Devices" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Device Status

Find network device status matching the specified filters.

Properties

Type a descriptive name for the node in the Label field. ("Device Status" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Devices CPU Utilization Query

Find all network devices matching the input filters, returning CPU metrics for each. By default, this query is based on the CPU Utilization metric.

Properties

Type a descriptive name for the node in the Label field. ("Device Status" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Devices Memory Utilization Query

Find all network devices matching the input filters, returning Memory metrics for each. By default, this query is based on the Memory Utilization metric.

Properties

Type a descriptive name for the node in the Label field. ("Applications #" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Network Device Disk Utilization Query

Find all network devices matching the input filters, returning Disk Utilization data for each.

Properties

Type a descriptive name for the node in the Label field. ("Devices (Disk Utilization)" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Devices Availability Query

Find all network devices matching the input filters, returning Availability metrics for each.

Properties

Type a descriptive name for the node in the Label field. ("Devices (Availability)" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Devices Up Time Query

Find all network devices matching the input filters, returning Up Time metrics for each.

Properties

Type a descriptive name for the node in the Label field. ("Devices (Up Time)" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Network Device Configuration Changes Query

Find all network devices matching the input filters, returning configuration change metrics for each.

Properties

Type a descriptive name for the node in the Label field. ("Network Device (Configuration Changes)" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Device Performance

Find all network devices matching the input filters, returning performance metrics for each.

Properties

Type a descriptive name for the node in the Label field. ("Devices (Performance)" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Hosts Query

Find all hosts matching the input filters, returning selected metrics for each.

Properties

Type a descriptive name for the node in the Label field. ("Hosts" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Client Hosts Query

Find all client hosts matching the input filters, returning selected metrics for each.

Properties

Type a descriptive name for the node in the Label field. ("Client Hosts" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Server Hosts Query

Find all server hosts matching the input filters, returning selected metrics for each.

Properties

Type a descriptive name for the node in the Label field. ("Server Hosts" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Client-Server Pairs Query

Find all client-server pairs matching the input filters, returning selected metrics for each.

Properties

Type a descriptive name for the node in the Label field. ("Client-Server Pairs" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Locations Query

Query for location data associated with the preceding trigger.

Label – Type an informative name for the data query. You can keep the system-provided default if you wish.

Filters

Location

Application – Reduce the set of applications examined by this runbook to investigate the incident.

- **None** – No applications are excluded from consideration by the runbook.
- **Specific Apps** – Exclude all applications from consideration by the runbook except those that you list here, separated by commas.
- **Apps From Trigger** – Exclude all applications from consideration by the runbook except those that are forwarded from the trigger node.
- **Apps From Previous Node** – Exclude all applications from consideration by the runbook except those that are forwarded from the preceding node in the sequence.

Limit

Limit To Top *n* objects – Specify a maximum number of objects to consider in the runbook investigation. The default number is 10.

Based On – Constrain the runbook investigation by focusing on a single metric.

Over The Last – Specify a time interval for which to look back at the selected metric.

Metrics

Metrics

Collect Metrics – Toggle this to specify which metrics to collect, and how that metric data will be formatted, for what time interval.

Output Data Format – Choose **Summarized** or **Time Series**.

Collect Data For Last – Specify a time interval over which to acquire metric data.

Done

Client Locations Query

Query for client data related to the location associated with the preceding query.

Properties

Type a descriptive name for the node in the Label field. ("Client Locations" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Server Locations Query

Query for server data related to the location associated with the preceding query.

Properties

Type a descriptive name for the node in the Label field. ("Server Locations" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Network Interface Traffic Query

Find all interfaces matching the input filters, returning selected traffic metrics for each.

Properties

Type a descriptive name for the node in the Label field. ("Network Interfaces" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Interface Status Query

Find all network interfaces matching the input filters, returning status for each.

Properties

Type a descriptive name for the node in the Label field. ("Interface Status" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Network Interface Error Query

Find all interfaces matching the input filters, returning selected error metrics for each.

Properties

Type a descriptive name for the node in the Label field. ("Network Interfaces (Error Metrics)" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Interface Performance

Find all network interfaces matching the input filters, returning performance metrics for each.

Properties

Type a descriptive name for the node in the Label field. ("Interfaces (Performance)" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Protocols

Find all network protocols associated with the preceding trigger that match the input filters.

Properties

Type a descriptive name for the node in the Label field. ("Protocols" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

DSCPs

Find all DSCPs associated with the preceding trigger that match the input filters.

Properties

Type a descriptive name for the node in the Label field. ("DSCPs" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Client-Server Location Pairs

Find all client/server location information associated with the preceding trigger that match the input filters.

Properties

Type a descriptive name for the node in the Label field. ("Client-Server Location Pairs" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

Client-Server Pairs and Protoport

Find all host protocol port information associated with the preceding trigger that match the input filters.

Properties

Type a descriptive name for the node in the Label field. ("Client-Server Pairs and Protoport" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Properties

Show the parameters that this node receives from the preceding node in the runbook.

User Device Node

Find user devices and their relevant metrics.

By default, this node collects and forwards these metrics for user devices:

- Business Location
- User Device Name
- User Name
- User Device ISP
- Physical Location
- Physical Location City
- Physical Location State
- Physical Location Country
- User Connection Type
- WiFi BSSID
- WiFi Channnel
- WiFI SSID

Properties

Type a descriptive name for the node in the Label field. ("User Device" is the system-provided default.)
Select **Debug** if you want to receive debug data when the node executes.

Filters

Filter the query.

Limit

Define a limit to the query.

Metrics

Specify metrics for the query to include.

Aggregator Node

Group the results from the preceding runbook node by a specific metric, or apply an operation on the metrics forwarded from that node.

The supported aggregation operations are:

- min
- max
- average
- count (distinct values of a string)
- sum

These aggregation operations are context-sensitive (e.g., count is the only operation available for the Client IP metric).

An Aggregator node can follow a Data Query node or a Decision Branch node; the preceding node must output summarized data, not time series data. Metrics are collected from the last preceding Data Query node.

For each aggregator node, specify whether the corresponding data will be presented by all metrics all together (the default), or grouped by a single specified metric. You can select which metrics to include, and the aggregation operation to be performed on each.

Properties

Type a descriptive name for the node in the Label field. ("Aggregator" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Grouping

Specify whether data from the preceding node is presented by all metrics all together (the default), or grouped by a single specified metric.

Metrics

Select which metrics to include, and the aggregation operation to be performed on each.

Transform

Modify output from a preceding node in the runbook into a JSON format that Riverbed IQ can use to provide input to another node. For example, transform the response from an HTTP Query node into a JSON format that Riverbed IQ can use to display data in visualizations such as tables and charts.

A Transform node can follow and take input from these node types in a runbook sequence:

- Data Query (any)
- Impact (any)
- Decision Branch
- HTTP Query
- Transform

A Transform node can output data to these node types in a runbook sequence:

- Data Query (any)
- Impact (any)
- Decision Branch
- Visualization
- Transform

Type a descriptive name for the node in the Label field. ("Transform" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Output Data Definition

Specify which data (properties and/or metrics) you want to forward to other runbook nodes via JSON.

Output Data Format — Choose Summarized or Time Series.

Output Data Properties — Choose a property to output and specify its data type and units. Click Add Property if you want to output more than one.

- Name
- Type
- Unit
- Add Property

Output Data Metrics — Choose a metric to output and specify its data type and units. (For Time Series output, you must have at least one metric.) Click Add Metric if you want to output more than one.

- Name
- Type
- Unit
- Add Metric

Transform Template

Type the Liquid template to use for generating JSON output. Click the Output Data Example tab to show a preview of what the Liquid template's output will look like.

Decision Branch

Pass data in to this node from a Data Query node and apply a condition to it to execute a child node or branch.

Use as many Decision Branch nodes in your runbook as you need, but be careful to avoid creating branches that produce contradictory, redundant, or confusing results. The Runbook Editor will not prevent you from defining unproductive branches.

Each Decision Branch node requires at least one child node or "output." This can be either the True path or the False path, but, for clarity and consistency, it's a good practice to define both paths, even if one is only a stub. Each output connector has a customizable label shown on the canvas to help you annotate and keep track of what each branch does.

The Decision Branch properties are context-sensitive, based on the data forwarded by the immediately preceding node.

Set Incident Priority

Assign the priority of an incident based on the execution of logic in a preceding connected Decision Branch node.

The resulting incident priority is shown on the Incidents page and is included in the summarized incident information shown on the Impact Dashboard.

You can have multiple Set Incident Priority nodes per runbook, with each one assigning the priority resulting from a preceding connected decision branch.

Properties

Label – Type an informative name for the priority assignment. You can keep the system-provided default, if you wish.

Set Priority To: Choose **Critical** (the default), **High**, **Moderate**, or **Low**.

Tag As Affected Applications

Mark the output from the preceding connected node as the application affected by the incident.

This result is forwarded to the Impact Dashboard, where the set of applications most affected by incidents are summarized.

Properties

Label – Type an informative name for the tagging of the affected application. You can keep the system-provided default, if you wish.

Tag As Affected Locations

Mark the output from the preceding connected node as the location affected by the incident.

This result is forwarded to the Impact Dashboard, where the set of locations most affected by incidents are summarized.

Properties

Label – Type an informative name for the tagging of the affected location. You can keep the system-provided default, if you wish.

Tag As Affected Users

Mark the output from the preceding connected node as the user affected by the incident.

This result is forwarded to the Impact Dashboard, where the set of users most affected by incidents are summarized.

Properties

Label – Type an informative name for the tagging of the affected user. You can keep the system-provided default, if you wish.

HTTP Request

Use an HTTP method to communicate with another application.

This node will output:

- HTTP status code (e.g., 400)
- HTTP response headers in JSON format
- HTTP response body in JSON format

The HTTP request relies on variables for its operation.

Type a descriptive name for the node in the Label field. ("HTTP Request" is the system-provided default.)
Select **Debug** if you want to receive debug data when the node executes.

Authentication

Choose an existing authentication profile to communicate with a different application, or define a new one.

Endpoint

Specify the HTTP method and URI the request will use as its endpoint.

HTTP Headers

Define the HTTP headers for the request to use, including the expected media type (Content-Type Header) and content type (Accept Header). The default is application/json.

HTTP Payload

Click **Include payload** to define a payload to include in the request. This can use variables for referencing specific parameters.

Retry Until Ready

Define a Liquid template to query a status API until a ready response is received, along with a timeout value for stopping query retries.

Context

Show the parameters this node receives from the preceding node in the run-book.

Table

Show resulting data in a table.

Pie Chart

Show resulting data in a proportional chart.

Bar Chart

Show resulting data in a horizontal bar chart.

Time Series Chart

Show resulting data in a time series chart.

Cards

Show resulting data as large text in a KPI widget.

Gauges

Show resulting data in a dial with a needle.

Connection Graph

Show resulting data as connections between individual hosts.

Text

Include a text statement in runbook output. The text can include variables.

Properties

Type a descriptive name for the node in the Label field. ("Text" is the system-provided default.)

Title

Type a title for the text that will appear in the runbook output.

Row

Type the number of rows of text that will appear in the runbook output.

Notes

Type the body text that you want to appear in the runbook output. This can include variables, e.g., "Data From {{runtime.MyCustomVariable}}"

Set Primitive Variable

Assign a value to a primitive (simple) variable, in which the variable represents a single, discrete value.

Properties

Type a descriptive name for the node in the Label field. ("Set Primitive Variable" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Variables to Set

Specify the variable.

- Scope — Choose the scope in which to set the variable's value. This is the environment, or set of conditions, within which the variable and its value are relevant.
    - Runtime Scope
    - Incident Scope
- Variable Name — Choose an existing variable from the pulldown.
- Set Another Variable — Click to add Scope and Variable Name fields for an additional variable.

Template to Assign Value to Variables

Assign a value to the variable.

- Template — Type an expression to set the value of each variable you specified above.
- Output Data Example — Preview the output this value assignment will produce.

Set Structured Variable

Assign a value to a structured (complex) variable, in which the variable represents output that is summarized or aggregated from preceding nodes in the runbook. There are two types of structured variable:

- Complex Summarized Data — Summarized output from a Data Query node, Decision Branch, or Transform node using definition from a variable. This must have a subtype to indicate what it represents, e.g. “Application Summary Data” or “Custom Summary Data”. Available only for Runtime scope and Incident scope.
- Complex Time Series Data — Any time series output from a Data Query node, Decision Branch, or Transform node using definition from a variable. This must have a subtype to indicate what it represents, e.g. “Application Time Series” or “Custom Time Series”. Available only for Runtime scope and Incident scope.

Properties

Type a descriptive name for the node in the Label field. ("Set Structured Variable" is the system-provided default.)
Select Debug if you want to receive debug data when the node executes.

Variables to Set

Specify the variable. (Only one can be set per node.)

- Scope — Choose the scope in which to set the variable's value. This is the environment, or set of conditions, within which the variable and its value are relevant.
    - Runtime Scope
    - Incident Scope
- Variable Name — Choose an existing variable from the pulldown.

Comment

Type text to annotate the runbook. Comments do not affect runbook execution, so use as many as you need. The comments are visible only in the Runbook Editor, not in the runbook output displayed for an incident on the Incident Details page.

Label

Label – Type an informative name for the data query. You can keep the system-provided default if you wish. Labels are not required to be unique within a runbook, but assigning each node its own name is good practice.

Filters

Reduce the set of objects (applications, devices, interfaces) examined by this runbook to investigate the incident.

Application – Reduce the set of applications examined by this runbook to investigate the incident.

- **None** – No applications are excluded from consideration by the runbook.
- **Specific Apps** – Exclude all applications from consideration by the runbook except those that you list here, separated by commas.
- **Apps From Trigger** – Exclude all applications from consideration by the runbook except those that are forwarded from the trigger node.
- **Apps From Previous Node** – Exclude all applications from consideration by the runbook except those that are forwarded from the preceding node in the sequence.

Client – Reduce the set of clients examined by this runbook to investigate the incident.

- **None** – No clients are excluded from consideration by the runbook.
- **Specific Client Hosts** – Exclude all clients from consideration by the runbook except those that you list here, separated by commas.
- **Client Hosts From Previous Node** – Exclude all clients from consideration by the runbook except those that are forwarded from the preceding node in the sequence.

Device – Reduce the set of devices examined by this runbook to investigate the incident.

- **None** – No devices are excluded from consideration by the runbook.
- **Specific Devices** – Exclude all devices from consideration by the runbook except those that you list here, separated by commas.
- **Device From Trigger** – Exclude all devices from consideration by the runbook except those that are forwarded from the trigger node.
- **Device From Previous Node** – Exclude all devices from consideration by the runbook except those that are forwarded from the preceding node in the sequence.

Host – Reduce the set of hosts examined by this runbook to investigate the incident.

- **None** – No hosts are excluded from consideration by the runbook.
- **Specific Hosts** – Exclude all hosts from consideration by the runbook except those that you list here, separated by commas.
- **Hosts From Previous Node** – Exclude all hosts from consideration by the runbook except those that are forwarded from the preceding node in the sequence.

Interface – Reduce the set of device interfaces examined by this runbook to investigate the incident.

- **None** – No interfaces are excluded from consideration by the runbook.
- **Specific Interfaces** – Exclude all interfaces from consideration by the runbook except those that you list here, separated by commas.
- **Interface From Trigger** – Exclude all interfaces from consideration by the runbook except those that are forwarded from the trigger node.
- **Interface From Previous Node** – Exclude all interfaces from consideration by the runbook except those that are forwarded from the preceding node in the sequence.

Location – Reduce the set of locations examined by this runbook to investigate the incident.

- **None** – No locations are excluded from consideration by the runbook.
- **Specific Locations** – Exclude all locations from consideration by the runbook except those that you list here, separated by commas.
- **Locations From Previous Node** – Exclude all locations from consideration by the runbook except those that are forwarded from the preceding node in the sequence.

Server – Reduce the set of servers examined by this runbook to investigate the incident.

- **None** – No servers are excluded from consideration by the runbook.
- **Specific Server Hosts** – Exclude all servers from consideration by the runbook except those that you list here, separated by commas.
- **Server Hosts From Previous Node** – Exclude all servers from consideration by the runbook except those that are forwarded from the preceding node in the sequence.

Metric Collection

Specify the metrics to be collected for consideration by the runbook to investigate the incident.

Collect Metrics – Toggle this to specify which metrics to collect, and how that metric data will be formatted, for what time interval.

Output Data Format – Choose **Summarized** or **Time Series**.

Collect Data For Last – Specify a time interval over which to acquire metric data.

Limits

Constrain the runbook investigation in terms of a specific metric over a fixed time interval.

Limit To Top *n* objects – Specify a maximum number of objects to consider in the runbook investigation. The default number is 10.

Based On – Constrain the runbook investigation by focusing on a single metric.

Over The Last – Specify a time interval for which to look back at the selected metric.

Dialog and Wizard Descriptions

This section describes individual dialog boxes and wizards.

Add an Riverbed Edge Wizard

This enables you to define a new Riverbed Edge instance. The wizard comprises three pages:

- Basic Details — Specify a meaningful name for the Edge and indicate its type (AWS, Azure, or ESXi).
- Connection Config — If necessary, specify the connection type for the Edge:
  - If the Edge does not have access to the Internet, specify a proxy to use.
  - If the Edge has only port 443 open to the Internet, click **Use Websockets**.
- Review — Verify this summary of the Edge's definition before you click **Submit** to make it manageable with Riverbed IQ.

Add an Authentication Profile Wizard

Click **Add an Authentication Profile** on the Third Party Authentication page to display the wizard of the same name and create a new profile for communicating with another application. This wizard comprises six pages:

- Basic Details — Specify the Profile Name, Description, and Authentication Method (this can be **BasicAuthentication**, **ApiKeyAuthentication**, **Open Authorization 2.0**, or **Client Certificate (mTLS) Authentication**).
- Authentication Parameters — Provide the required credentials for the authentication method you chose.
  If you need an explanation for any of the credentials, refer to authoritative documentation for that authentication method.
    - For ApiKeyAuthentication: Specify the key and the method by which it is sent: **QueryParameter**, **AuthorizationHeader**, **RequestHeader**, or **RequestBodyField**. You can add multiple RequestHeader keys; this is useful for authenticating with APIs that require different keys for different types of access (such as some APIs used by Datadog).
    - For AWS Signature: Choose the Version: **AWS Signature Version 4** or **AWS Signature Version 4a (multi-region)**, then type the AWS Access Key ID and AWS Secret Access Key.
    - For BasicAuthentication: Specify the Username and Password.
    - For Open Authorization 2.0: Choose the Grant Type: **Client Credentials** or **Password**.
        - For Client Credentials, specify the Client ID, Client Secret, Authentication URI, and Scope URL. Click **Show Advanced Configuration** to work with custom parameters, and click **Add Custom Parameter** to specify a key, its value, and its

send via method.

- For Password, specify the Username, Password, Authentication URI, Scope URL, Client ID, and Client Secret. Click **Show Advanced Configuration** to work with custom parameters, and click **Add Custom Parameter** to specify a key, its value, and its send via method.

- Public or Private Endpoint — Specify whether the destination endpoint is public (the default) or private. If you want to communicate with an endpoint in a private network, click Private endpoint via an authorized Riverbed Edge, and choose an existing authorized Riverbed Edge to enable the communication. The profile can have more than one authorized Riverbed Edge assigned, although only one will be used at a time.
- Test Query — Define an HTTP query to use for validating communication with the other application. Specify an HTTP method to use and the target URI, as well as the HTTP header and HTTP payload.
- Verify Response — Execute the test query you defined in the Test Query page by clicking the **Run Test Query** button. Returned results will be shown in the panel below it. If you specified one or more private endpoints in the Public or Private Endpoint page, you'll need to choose one of the specified ones, here, as the destination of the test query.
  Note: If the Riverbed Edge selected as private endpoint uses a proxy, make certain that proxy is configured appropriately to communicate outside your network.
- Review — Show a summary of the authentication profile.

Add an Automation Wizard

Automations associate triggers with runbooks, causing a runbook to execute an investigation in response to a trigger, based on user-defined conditional criteria.

To define a new automation:

1. Click **Add an Automation** on the Automation Management page, either after a trigger category (following the cards for the trigger types) or at the top of the list of automations for a trigger type. The Add an Automation wizard appears.
2. Choose a trigger type from the pulldown menu on the Select Trigger page, then click **Next**.
   Note that the trigger pulldown is validated for the current trigger category, listing trigger types for that category only (i.e., if you're working with incident triggers, the pulldown will not list lifecycle triggers).
3. Specify one or more conditions that will cause the automation to execute when the conditions are matched, using the condition building tools provided. Click **Next** when you're finished.
4. Specify the runbook to execute when the conditions are matched. You can choose an existing runbook or click **Create New Runbook** to launch the Runbook Editor and define a new runbook. Click **Next** when you're done specifying the runbook to execute.
5. On the Finalize Automation page, type a name for the automation and a brief description of it. Also, you need to specify the Order in which the automation will be executed for the trigger type.
   Each automation must have a unique Order number for the trigger type; you cannot have two or more automations for a trigger type with the same Order number. If you create a new automation and assign it Order 1, all existing automations for that trigger type will be decreased in order (i.e., the previous Order 1 will become Order 2, and so forth down the list

for that trigger type).

The first automation that is matched for an incident or lifecycle trigger, according to the defined condition, is the automation that will execute for that trigger type. This means that, when more than one automation is defined for a trigger type, the first in order should have extremely specific and restrictive conditional criteria, with successive automations having increasingly broad and permissive conditional criteria.

6. Click **Submit** to complete the definition of the automation.

Explorer Overview

Riverbed IQ's Explorer pages show detailed inventories of the entities that originate incidents. These entities are the devices, interfaces, applications, and locations that populate your network. In addition, the Explorer enables you to define and manage custom properties for entities; these are user-defined entity properties that you can use to describe and identify specific entities of interest in your network. Each entity type, plus custom properties, has its own page under the Explorer menu:

- Devices
- Interfaces
- Locations
- Applications
- Custom Properties

For each entity, the Explorer shows the properties that describe it; these vary, according to the entity type, but include details such as name, IP address, model, inbound speed, and location, among many others. Note that one entity type can be a property of another: A location can be a property of a device and a device can be a property of an interface, to give two examples. For each entity listed on the page, you can click it to view all its properties at once in the Details blade.

Every property name can be used as a column name on an Explorer page. In addition, every property name and value on an Explorer page can be:

- used as a filter
- used as a search string
- shown in the Details blade

Details blade

The Details blade appears at the right side of the Explorer page when you click on a listed entity, showing all of that entity's properties at a glance.

- The Details blade for devices has a second tab listing corresponding interfaces.
- The Details blade for interfaces has a second tab listing corresponding devices.
- The Details blade for locations has a second tab listing corresponding devices.

In addition, the Details blade provides the **Set Custom Property** button. Clicking this opens the Set Custom Property dialog for working with custom properties and values.

Custom Properties

Custom properties are user-defined properties that you can apply to one or more entity types (Application, Device, Interface, Location) to make it easier to find entities of special interest via incident search, runbooks, and automations.

Click **Explorer** > **Custom Properties** to display the Custom Properties page, which lists all of the existing custom properties in your configuration.

Custom Properties Page

The Custom Properties page shows these columns for each custom property:

- Name — The user-assigned name of the custom property.
- Description — The user-defined description of the custom property.
- Locations — The count of location entities that use this custom property.
- Devices — The count of device entities that use this custom property.
- Interfaces — The count of interface entities that use this custom property.
- Applications — The count of application entities that use this custom property.
- Last Updated By — The ID of the user who changed this custom property's definition or assignment most recently.
- Last Updated — The most recent time at which this custom property's definition or assignment was changed.

Defining a Custom Property

Click **New** at the upper right corner of the Custom Properties page to open the Set Custom Property dialog and define a new custom property. Type a name and a description for the custom property, then, select one or more entity types to which the custom property can be applied: Application, Device, Interface, or

Location. Add one or more values that it's possible for the custom property to have. This is a list of possible valid values that you define explicitly.

An error message will appear if a custom property is assigned to an entity and you make the entity type invalid for the custom property.

Importing and Exporting Custom Properties

Custom properties, names, and values, can be exported and imported via CSV file.

Searching By Custom Property

Global Search

Global search results include two categories:

- Custom Properties — result will show name, description and 2 links: to incidents and custom property details
- Locations — result will show name, description and 2 links: to incidents and custom property details

Incident Search

Incident search includes filters for each custom property category. Fuzzy search should not be applied on custom property. Custom properties are not shown as columns for the table.

Custom Properties in Runbooks

Custom properties for an entity type are shown in runbooks and the Runbook Editor the same as built-in properties. This includes Visualization nodes.

Custom Properties in Automations

Custom properties for an entity type are shown in automations the same as built-in properties.

Introduction to Runbooks

Riverbed IQ provides a variety of automation capabilities in the form of runbooks. A runbook is a workflow built using Riverbed's low-code/no-code Runbook Editor, which provides a graphical drag-and-drop method for assembling the workflow. Riverbed IQ supports these types of runbooks:

- Incident automations — These are runbooks that execute automatically whenever a new incident is created.
- Incident lifecycle automations — These are runbooks that execute automatically whenever a triggering lifecycle event occurs on an incident.
- API-driven automations — These are runbooks that are triggered in response to a configured API call.
- On-demand automations — These are runbooks that are triggered as required (either on-demand, or as scheduled).
- Subflows — These are runbook "macros:" Chunks of reusable automations that perform frequently-used functions (e.g., open a ticket in an external system), and can also be used to implement integrations with third party systems.

Each runbook automation consists of a single entry point (a Triggering Entity) and a sequence of connected nodes, each performing a specific function. The nodes make up one or more execution paths, with each path being a component of the larger processing logic that is traversed according to the data/-context available at execution time.

The set of runbook nodes available for an automation is displayed in the Runbook Editor in a palette at the left side, and depends on the type of automation; for example, incident automations will have a slightly different collection of nodes available than incident lifecycle automations will. Additionally, the collection of runbook nodes displayed in the Runbook Editor may vary depending on how Riverbed IQ is configured, because certain nodes that require a

specific data source type will not be available if that data source type is not configured.

Incident Automations

High-level Workflow

Incident automations are runbooks that execute automatically whenever a new incident is created.

New incidents are created whenever the Riverbed IQ analytics pipeline detects anomalous behavior in the key measurements streaming in from data sources. Key measurements can be associated with a variety of observed entities: applications, devices, interfaces, and locations. An anomalous event associated with any of these entity types will be surfaced as a new incident, which will trigger an associated runbook based on that source entity type.

The associated runbook automation is determined based on the Triggering Entity associated with the source entity type: Application, Device, Interface, or Location. The processing logic then executes to perform investigation and analysis based on the available data/context, and the resulting runbook analysis is attached to the incident.

Riverbed IQ provides a number of built-in Incident automation runbooks that provide default templates you can customize, optimize, and tune for your specific requirements. These built-in incident automation runbooks are covered in detail in their own topics:

- Device Down Issue
- Interface Performance Issue
- Multi-Device Down Issue
- Application Location Performance Issue

Creating and Editing Custom Runbooks Using the Runbook Editor

This topic is a high level overview of the workflow for creating and editing a custom runbook. For detailed instructions for choosing specific nodes and assembling them on the runbook canvas, refer to Adding and Assembling Runbook Nodes. Refer to Runbook Examples for walkthroughs of some sample runbooks.

Click **New** on the Runbooks page to open the Runbook Editor and begin defining a new runbook. For an existing runbook, click **Edit**.

The general workflow is to select a runbook node in the palette at the left and move it onto the runbook canvas at right. You can do this by dragging it to the canvas or by double-clicking it on the palette. The palette lists the runbook node categories in approximate order of how they should be assembled in sequence (first a trigger, then a data query, then a logic operation, then an impact or a visualization), but this is broadly advisory, and a complex runbook may have nodes from different categories arranged in myriad ways. Selecting a node on the canvas opens its properties at the right, enabling you to configure that node's behavior in the context of the larger runbook.

Begin defining a new runbook by choosing a trigger on the palette and moving it to the canvas. Each runbook can have a single trigger, which specifies the set of anomalies that are considered for investigation by the runbook.

In most cases, you'll want to add a data query next, to filter the Data Ocean in the context of the trigger. Choose a data query on the palette and move it to the canvas, to the right of the trigger you chose. If you have the **Auto-Connect** option active (it is active by default), an arrow will be drawn from the trigger to the data query automatically, connecting the two nodes in a sequence. If **Auto-Connect** is disabled, you'll need to select the trigger and draw the connection to the data query manually.

Each time you place a new node on the canvas, you can edit its properties, though you can do this in the Runbook Editor at any time, not only at placement. Node properties vary according to the node type, and can be as simple as setting the node name or as sophisticated as specifying metrics and thresholds to control the runbook's behavior.

Make sure to include at least one visualization for each runbook. The runbook will execute and complete its investigation without one, but you won't see any of the output unless there's at least one visualization to show it.

The Comment node (under the Miscellaneous category) enables you to annotate the runbook as you deem appropriate. Comment nodes don't execute or affect the runbook output, and you can use as many of them as you want to document the runbook's behavior.

When you're finished, click the **Save** icon in the tool bar to save the runbook. It will appear in the list of runbooks shown on the Runbooks page.

Trigger Inputs For Testing Runbooks

When you test a runbook (using **Execute a Test Run** in the Runbook Editor), the Inputs dialog appears, prompting you to provide inputs interactively that the trigger and other preceding nodes will supply when the runbook executes in automated use. The test inputs are:

- Application/Device/Interface/Location — Type the name of the object (application or device or interface or location, as appropriate for the run-book trigger) that you are testing this runbook for.
- Client Location — Choose the location that you are testing this runbook for.
- mimic the multi-device automation trigger — [For Device trigger only] Simulate the existence of multiple device triggers.
- Time — The default is Now.
- Trigger Metric — Choose the metric passed from the trigger that you are testing this runbook for.
- Actual Value — Type the value to use for that metric.
- Acceptable Low Value — Type the value at the low end of the metric's acceptable range.
- Acceptable High Value — Type the value at the high end of the metric's acceptable range.
- Advanced:
    - Incident Variables — Type the incident variables to be used in the test, along with their values. For example:
      {
      "incident.builtin.status": "New"
      }
    - Timeout — The default is five minutes.

Automation and Trigger Overview

Automations associate triggers with runbooks, causing a runbook to execute an investigation in response to a trigger, based on user-defined conditional criteria. Automations are used for all trigger categories (new incident, incident lifecycle, and external) and all of their constituent trigger types.

Automations are managed using the Automation Management page at **Automation** > **Automation Management**.

Each automation comprises these components, which are defined using the Add an Automation wizard:

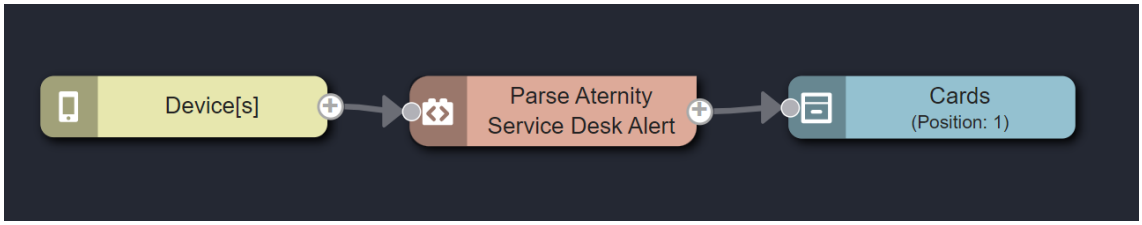
- Trigger — The occurrence to be investigated or recorded.
- Condition — The user-defined criteria that determines when a runbook will execute in response to the trigger.
- Runbook — The automated investigation to be executed in response to the trigger.
- Order of execution — The automation's precedence for its trigger type. Each automation must have a unique Order number for the trigger type; you cannot have two or more automations for a trigger type with the same Order number. If you create a new automation and assign it Order 1, all existing automations for that trigger type will be decreased in order (i.e., the previous Order 1 will become Order 2, and so forth down the list for that trigger type).
  The first automation that is matched for an incident or lifecycle trigger, according to the defined condition, is the automation that will execute for that trigger type. This means that, when more than one automation is defined for a trigger type, the first in order should have extremely specific and restrictive conditional criteria, with successive automations having increasingly broad and permissive conditional criteria.

Subflow Overview

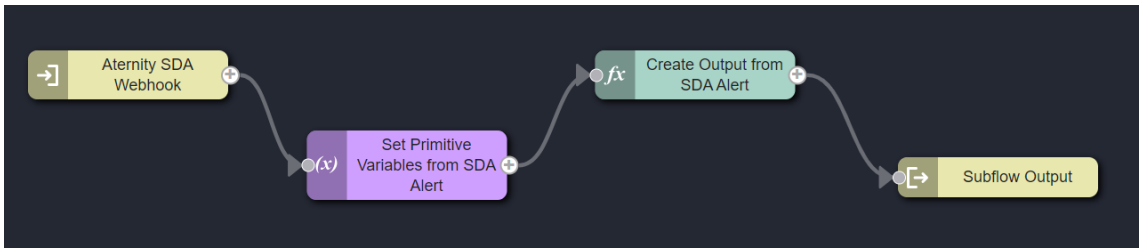
Work with subflows by clicking **Automation** > **Subflows** in the lefthand Navigation pane to display the Subflows page.

A subflow is a sequence of runbook nodes that execute a discrete runbook activity and that can be saved as an independent reusable component for inclusion in other runbooks. This makes a subflow a convenient means of implementing a common runbook operation once and including it in different runbooks where it's useful. For example, you could create a subflow to acquire the top traffic generating applications in the network, and reuse that subflow in multiple runbooks in which that application data is meaningful.

This image shows a built-in Subflow node (Parse Aternity Service Desk Alert) used in an Incident runbook:



This image shows that same subflow open in the Subflow Editor:



[Image: Subflow in the Subflow Editor]

Subflows can be used in any type of runbook (Incident, On-Demand, or Lifecycle), and a single subflow can be used interchangeably across multiple runbook types, if the nodes it includes are valid for each of the types. The subflow

is validated for the runbook type in the context of the nodes that precede it and follow it in the runbook definition.

Subflows have version numbers (major.minor) associated with them to help you track the progression of changes to the subflow definition and recognize exactly which version is used in a runbook or an integration. This is useful for tracking changes to Riverbed IQ's built-in subflows as well as your own custom subflows. When a subflow is used in a runbook, the Runbook Editor shows that subflow's version in the subflow node's configuration blade. When you change a subflow's definition, the changes will take effect in every runbook in which that subflow is used, though the method for updating the runbook varies depending on whether the change is minor or major. The criteria for what constitutes a major revision vs. a minor revision are subjective, and for your own custom subflows, you'll need to define the criteria to differentiate the two. Separate from the definition of what makes a major change or a minor change is the *effect* that major and minor changes have on existing subflows.

Subflow Use Case Examples

Some examples of use cases for subflows are:

- Create ServiceNow ticket.
- Create a complex async query to Aternity remediation service
- Check whether a problem is server or network related
- Get a list of interfaces traversed by an app, with their actual utilization
- Parse Aternity webhook payload
- Display a time series with analysis (peak, outlier etc)
- Get the top apps in the network

Built-In Subflows

Riverbed IQ provides a number of subflows as part of the product. They are listed on the Subflows page, in the Built-In tab, with corresponding descriptions.

Including a Subflow In a Runbook

Add an existing subflow to a runbook as a Subflow node. Choose an existing subflow node in the Subflows category of the Node palette and drag it to desired place on the Runbook canvas. If the subflow definition is not valid for the nodes that you try to attach to, Riverbed IQ will notify you that the subflow cannot be connected.

When working with a runbook, the subflow node in the Node palette and the Subflow menu under the plus icon will always use the latest version. You cannot add an older major version to the runbook. As a result:

- You can't go back to an earlier working version in case the new major version is corrupted and is not working as expected.
- You can't add the same subflow node in an existing runbook that uses an older major version of a subflow.

Editing an Existing Subflow

The Subflow Editor behaves the same as the Runbook Editor. You can select a subflow in a Runbook definition to edit the subflow definition in context.

Editing Subflow Versions

For a subflow that has multiple existing versions, clicking the **...** menu provides options for **Edit Older Versions** and **Delete Older Versions**. Choosing one of these enables you to select a specific older version to work with.

When you save a subflow that you've edited, the Save Subflow dialog appears:

- Name — Type or replace the subflow's name. Renaming a subflow does not affect older versions of the same subflow, meaning that the same subflow can have different names in different versions.

- Version Update — Select a radio button to indicate how the subflow's version number should be incremented by this change: **None** (the default), **Minor**, or **Major.**
  Note:
  - Saving a new minor version will cause any runbook that uses the subflow at the same major version to update automatically.
  - Saving a major version will require any runbook that uses the subflow to be updated manually.
- Description — Type information about the subflow.

Note: Duplicating a subflow creates a new subflow with version 1.0.0.

Exporting and Importing Subflow Definitions

You can export and import subflow definitions via JSON file.

The subflow version is stored in the exported definition. When importing a subflow, Riverbed IQ will determine if there is a higher version, already, in which case it will reject the import and notify you that a more recent version of the subflow exists already.

Deleting a Subflow

You can delete a subflow only if it is not used by any Runbooks. Select a subflow on the Subflows page and click **Delete**. If the subflow is used by any Runbooks, Riverbed IQ will warn you and prevent you from completing the action. Deleting a subflow removes *all* versions of that subflow, not only the latest or selected version.

Updating a Subflow to a New Version

Updating Built-In Subflows to a New Version

For built-in subflows:

- When a new minor version of a built-in subflow becomes available, any runbook that uses it will update automatically to include the new minor version.
- When a new major version of a built-in subflow becomes available, you will need to manually update any runbook that uses it to include the new major version. Riverbed IQ will tag each affected runbook in the relevant page (Incident Runbooks, On-Demand Runbooks, Lifecycle Runbooks) with a warning. Updating the runbook manually requires you to replace the updated subflow explicitly.

Updating Custom Subflows to a New Version

Duplicating a subflow will create a new subflow with version 1.0.0.

Renaming a subflow does not affect older versions of the same subflow, meaning that the same subflow can have different names in different versions.

Updating Integrations to a New Subflow Version

When a subflow is used in an integration, you'll need to take action to update the subflow if its version changes:

- When a new minor version of a subflow is available and the integration is already installed, the integration will be marked with “Update available,” and you'll need to click the **Update** button to replace the subflow with the latest minor version. Any runbooks that use that subflow will update to use that latest minor version automatically.
- When a new major version of a subflow is available, the integration will be listed in the Integration Library's Available tab, even if an earlier major integration is installed already. You will need to re-enable the integration to use the updated major version of the subflow.

Vantage Points

A vantage point is a Riverbed AppResponse system, used as a filter for identifying AppResponse data handled by Application and Location triggers in runbooks. Accordingly, the Vantage Point filter is available in a runbook only when an AppResponse data source is available and either an Application trigger or Location trigger is in use.

There are four Vantage Point options:

- **No Filter** — Potentially, all AppResponse systems will be queried. It is possible for the query to include NetProfiler or other data sources.
- **All Vantage Points** — Potentially, all AppResponse systems will be queried. No other data sources will be queried.
- **Specific Vantage Points** — Select specific AppResponse systems to query and no others.
- **Vantage Point From Trigger** — Query only the AppResponse system associated with the anomaly that triggered the runbook.

If more than one App Response system is queried, and the response presentation is Table, then a dropdown with Vantage Points will be presented in the runbook output. By default it shows All Vantage Points; however, you can select a specific vantage point to filter the data further. For any other presentation (such as Pie, Bar, Line chart) the runbook response is limited to showing the data from one App Response system at a time.

Variables Overview

Riverbed IQ enables you to use variables in runbooks so that runbook data can be stored and exchanged with other applications. For example, you can use variables to do things like pass in information from a Slack message, or pass out data to an ITSM tool like ServiceNow. Riverbed IQ furnishes a number of built-in variables, and you can define custom variables, as well. Variables (built-in and custom) are accessed via the **x** button in the Runbook Editor's tool bar. Define variables in the Variable Definition Editor, and set their values in the Set Primitive Variable and Set Structured Variable nodes.

At this time, all variables are runtime scope variables, meaning that they are persistent over a single execution of a runbook.

Primitive and Structured Variables

A primitive (simple) variable represents a single discrete value. Set the values of primitive variables in the Set Primitive Variable node using a Liquid template. You can set the values of multiple primitive variables in a single Set Primitive Variable template.

A structured (complex) variable represents output that is summarized or aggregated from preceding nodes in the runbook, representing complex summarized data or complex time series data. Set the values of structured variables in the Set Structured Variable node. Structured variables can be used as filters in Data Ocean nodes; the structured variable type (e.g., "Application") defines what is available in the variable, and, consequently, where it can be used for filtering (e.g. application filter).

Syntax

Variables are named with syntax like, "runtime.interval_start" and "runtime.input.device.<property>", where "runtime" indicates the variable's scope, and

"<property>" stands for a device property used in the runbook. You could define a custom variable named, "runtime.MyCustomVariable".

Assign a value to a variable using the appropriate Variable node, Set Primitive Variable or Set Structured Variable. The node properties provide a field for assigning a value using a Liquid template. Some examples follow:

For a runtime scope variable of type integer named "myInt":

{

"runtime.myInt": "1948"

}

To assign that variable, "myInt" to the value of another variable ("anotherInt") multiplied by two:

{

"runtime.myInt": "{{ variables["runtime.anotherInt"]| times 2}}"

}

To assign that variable, "myInt" to the input to the node:

{

"runtime.myInt": "{{node_input.output.data[0].data.throughput }}"

}

Variable Definition Editor

Click the **(x)** button (**Edit Variables**) in the Runbook Editor tool bar to open the Variable Definition Editor. This enables you to create user-defined variables for use in runbooks, and also enables you to view the definitions of built-in variables (built-in variables cannot be modified).

The editor comprises a tab for each scope in which a variable can exist and be assigned a value:

- Runtime Scope — The variable is persistent over a single execution of the runbook.
- Incident Scope — The variable is persistent over the lifecycle of an incident associated with a runbook.

Add Variable

Click to add a user-defined variable for the current scope, and choose the variable type from the pop-up menu.

Each new entry is added to the list of user-defined variables.

User-Defined Variables

This lists the user-defined variables you've added to the current scope. Modify these as necessary.

For each variable, type a name and a default value, and fill in the other fields that are appropriate for that variable type (e.g., there's no Unit field for a String type variable).

Built-In Variables

This lists the built-in variables for the current scope. Built-in variables cannot be changed.

Working With Liquid Templates

In advanced workflows, you can use Liquid templates to transform data from one format to another. This typically is needed in these cases:

- Third party integrations using the HTTP node connected to a Transform node
- Saving variables using the Set Variables node

You can also use the Transform node with many of the other nodes in the Runbook node palette to transform their data into another format using a Liquid template.

If you want more information about Liquid, or how Liquid templates work, see the Liquid template language page for complete documentation.

Template Variables

The Liquid template allows you to access variables that are passed in to the template from the Trigger node or the direct parent nodes of the Transform or Set Variable nodes. The variables are surrounded by double curly braces as follows:

```
{{trigger["network_interfaces"]}}
```

In the variable, the **trigger** key specifies to get the data from the trigger, while the **network_interfaces** key specifies to get the **network_interfaces** object within the trigger. There are three root keys for all the variables. They are:

- trigger — Get data from the Trigger node.
- node_input.output — Get data from the parent node's output.
- variables — Get data from a runtime or incident variable within the Runbook.

The trigger has multiple objects in it, so, when you are pulling data from the trigger, you need to specify which object within the trigger you want to access. This is done by enclosing the object key in quotes and then in brackets. So, to get to the **network_interface** object, you use the following format for the variable:

```
{{trigger["network_interfaces"]}}
```

The following objects are available in the trigger:

- network_interfaces
- network_devices
- application_locations
- status_changed
- ongoing_state_changed
- runbook_completed
- note_added
- note_updated
- indicators_updated

Once you have specified the object you want to get the data from, it is useful to know the standard format that the data is in. A device, for example, has the following format:

```
{
  "data": [
    {
      "data": {
        "throughput": "25.4"
      },
      "keys": {
        "network_device.ipaddr": "10.5.45.64",
        "network_device.name": "Device 1",
        "network_device.location.name": "DC_West",
        "network_device.serial_number": "NF2PX0000394E",
        "network_device.os_version": "RiOS 9.12.2 (x86_64)",
        "network_device.model": "Riverbed SteelHead CXA580B110",
        "network_device.type": "WAN Accelerator",
```

```
          "network_device.vendor": "Riverbed Technology",
          "network_device.is_gateway": false,
          "is_primary_indicator": true
        }
      },
      {
        "data": {
          "throughput": "10.4"
        },
        "keys": {
          "network_device.ipaddr": "10.5.22.52",
          "network_device.name": "Device 2",
          "network_device.location.name": "Branch_East",
          "network_device.serial_number": "NF2MM000039A3",
          "network_device.os_version": "RiOS 9.12.2 (x86_64)",
          "network_device.model": "Riverbed SteelHead CXA580B110",
          "network_device.type": "WAN Accelerator",
          "network_device.vendor": "Riverbed Technology",
          "network_device.is_gateway": false,
          "is_primary_indicator": false
        }
      }
    ]
  }
```


So, if you want to get to the key **network_device.name** in the first element of the data array, you would use the following:

```
{{trigger["network_devices"]["data"][0].keys["network_
device.name"]}}
```

In the example above, this would result in the value "Device 1". If you want to get data from the second item in the array, just modify the array index as follows:

```
{{trigger["network_devices"]["data"][1].keys["network_
device.name"]}}
```

This will return "Device 2". At this time, triggers do not allow you to access the metric values they carry.

Now, if you want to access the interface data from the output of the previous node, use the **node_input.output** key and you have the following. If we use the network_device from the preceding example:

```
{{node_input.output["data"][0].keys["network_device.-
name"]}}
```

The **node_input.output** key does allow you to access the metric data, and that has the format:

```
{{node_input.output["data"][0].data["throughput"]}}
```

Finally, to access any of the runtime variables within the template, use the **variables** key as follows:

```
{{variables["incident.demoTOI"]}}
```

When accessing the trigger information from a Webhook trigger, the format of the information passed in by the Webhook is slightly different, and the following objects are available:

- requestBody
- requestQueryParameters
- requestHeaders
- requestPath

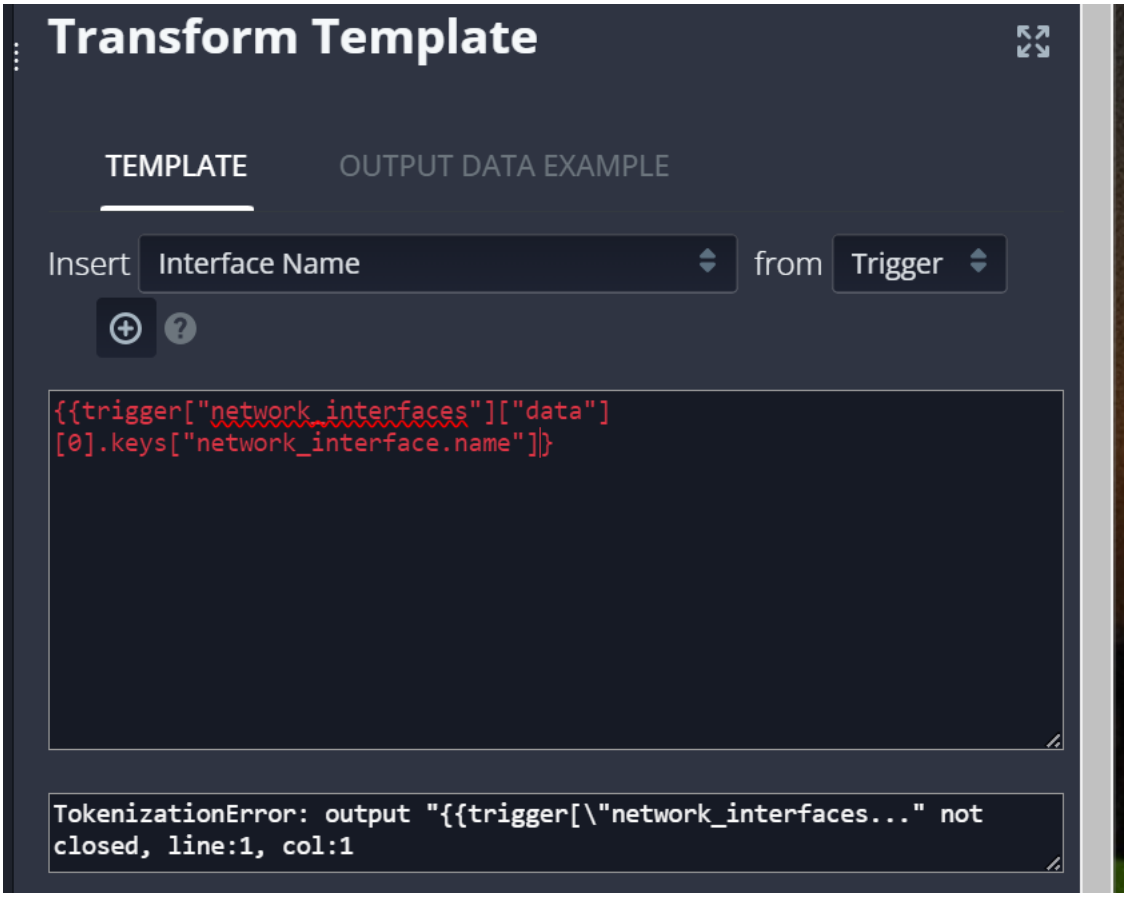
Here are examples of each for the Webhook trigger:

- `{{trigger["requestBody"]["application"]}}`
- `{{trigger["requestQueryParameters"]["application"]}}`
- `{{trigger["requestHeaders"]["Accept-Encoding"][1]}}`
- `{{trigger["requestPath"]}}`

Liquid Template Editor

The Template Editor helps you define a Liquid template.

The tool bar above the text panel helps you generate the variable token in the text, so you don't have to format the variable yourself. You can use the tool bar to generate the variable, and then you can edit it to do exactly what you want. The following screenshot shows the template editor with the toolbar above the text panel and the error information below that.



The editor has three components:

- Tool bar — The tool bar configures which variable you want to add, and, after making the appropriate selections in the tool bar, you can click the **+** (**Add**) button to add the variable in the template at the position of the cursor.
- Text content area — This is directly below the tool bar, and is where you type the Liquid template content. When you click the **+** button in the tool bar, variables are added in the content area at the position of the cursor.

- Error information — This is directly below the text content area, and shows error information including the line and position of the error in the Liquid template. This area will be empty when there are no errors.

The editor is simple, and the only part that needs explanation is the tool bar. The combo box after the From label allows you to select the top level key as described above. It has three choices that correspond to the three top level keys:

- **Trigger** — This will generate a variable that gets data from the trigger and inserts the **trigger** key in the variable. When you switch to Trigger in the combo box, it will update all the items in the combo box to the right of the Insert label with the trigger entities that can be used in the Liquid variable.
- **Parent** — This will generate a variable that gets data from the parent node and uses the **node_input.output** key. When you switch to Parent in the combo box, it will update all the items in the combo box to the right of the Insert label with the parent node entities that can be used in the Liquid variable.
- **Variable** — This will generate a variable that gets data from the runtime or incident variables used in the Runbook, and generates a variable with the top level **variables** key. When you switch to Variable in the combo box, it will update all the items in the combo box to the right of the Insert label with the list of runtime and/or incident variables that can be used in the Liquid variable.

Once you have selected the type of data in the from combo box and have then select the item to insert, you can then put your cursor anywhere in the text area and click the plus icon and it will insert the Liquid variable at that location.

When editing subflows, you do not have access to the trigger, so the trigger information will not be visible in the editor in this case.

Liquid Basics

This section is intended give you a few basic tips to get started, not to document the complete Liquid language. For that, see the Liquid template language page.

The first concept to understand is assignment; in Liquid, you can assign one variable to another. So, if you don't want to keep typing the text, "`node_input.output.data[0].keys.application`", then you can assign it to a variable. Use the "`{%`" delimiter to assign variables and make conditional statements.

Here is an example of assigning a variable:

```
{% assign appName = node_input.output.data[0].keys.application %}
```

In this case, you have assigned "`node_input.output.data[0].keys.application`" to the variable "`appName`".

You can use the same delimiter for conditional statements. For example, to emit different variables based on a condition:

```
{% if variable["runtime.MyIntVar"] == 1 %}
 {{variable["runtime.MyString"]}}
{% elseif appName == "NFS" %}
   {{variable["runtime.MyOtherString"]}}
{% else %}
  {{variable["runtime.MyOtherString2"]}}
{% endif %}
```

Tips

1. To discover the format of the data coming out of a node so you can decide on the format of the Liquid variable, turn on the Debug flag in the

Node Properties, then open the Debug dialog to see the format of the objects coming out of that node.

2. Another way to see the format of data from the various nodes is to open the Editor and look at the Advanced section, which will contain an example of the data coming out of the node.

Built-In Variables

Variables enable you to query for and manipulate specific information in HTTP requests.

Variables enable you to query for and manipulate specific information in various nodes throughout a runbook , such as HTTP Request, Decision Branch, and Data Ocean Query.

The following variables are built in to Riverbed IQ and are accessible in the Runbook Editor, provided as built-in Liquid templating language variables:

- Incident Scope
    - incident.title
    - incident.creation_time
    - incident.priority
    - incident.status
    - incident.primary_indicator.kind
    - incident.primary_indicator.metric.name
    - incident.primary_indicator.metric.value
    - incident.primary_indicator.threshold.value
    - incident.primary_indicator.baseline.value
    - incident.primary_indicator.start
    - incident.primary_indicator.end
    - incident.url
- Runtime Scope
    - runtime.interval_start
    - runtime.interval_end
    - runtime.input.app.<property>

        - runtime.input.client_location.<property>
        - runtime.input.device.<property>
        - runtime.input.site.<property>
        - runtime.input.interface.<property>
    - Runbook Scope
        - runbook.name
        - runbook.last_modified
        - runbook.last_modified_by
        - runbook.created_on
        - runbook.created_by
        - runbook.is_builtin

Webhooks in Riverbed IQ

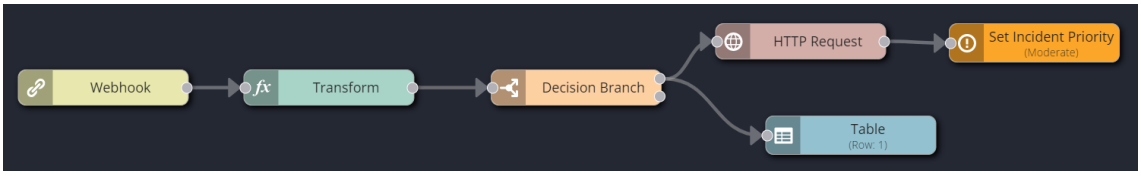
Riverbed IQ supports the execution of runbooks by webhooks sent by other applications. This is useful for creating incidents in Riverbed IQ based on alerts raised in other applications. In such cases, the other application sends a webhook to Riverbed IQ, which handles the webhook as a Webhook trigger, and executes the mapped runbook using the data acquired from the webhook.

The runbook can access these data from the trigger:

- HTTP Request URL — An object containing path parameters in the inbound request.
- HTTP Request Query Parameters — An object containing query parameters in the inbound request.
- HTTP Request Headers — An object containing headers in the inbound request.
- HTTP Request Body — A complex data object that defines the body structure of the inbound request.

When creating a runbook for use with a webhook, the first node in the runbook is the Webhook trigger, followed next by a Transform node, an HTTP Request node, or a Set Simple Variable node. Use a Transform node if you need to transform the webhook's JSON content into a format that can be used by Riverbed IQ. An HTTP Request node, whether it follows the Webhook trigger node immediately, or follows a Transform node, can acquire data from the webhook for use downstream in the runbook sequence.

For example:



Webhook Inputs

When you test a runbook (using **Execute a Test Run** in the Runbook Editor) that uses a Webhook trigger, the Inputs dialog appears, prompting you to provide inputs interactively that another application will supply when the runbook executes in automated use. The test inputs are:

- HTTP Request Body — Type a complex data object that defines the body structure of the inbound request. For example:
  ```
  {
    "test": "value"
  }
  ```
- HTTP Request Headers — Type the inbound request headers. For example:
  ```
  {
    "ACCEPT": ["*/*"]
  }
  ```
- HTTP Request URL — Type the inbound request path. For example: www.riverbed.com
- HTTP Request Query Parameters — Type the inbound request query parameters. For example:
  ```
  {
    "my param": ["my.param.value"]
  }
  ```
- Time — The default is Now.
- Timeout (min) — The default is five minutes.

Using JSON Variables With Webhooks

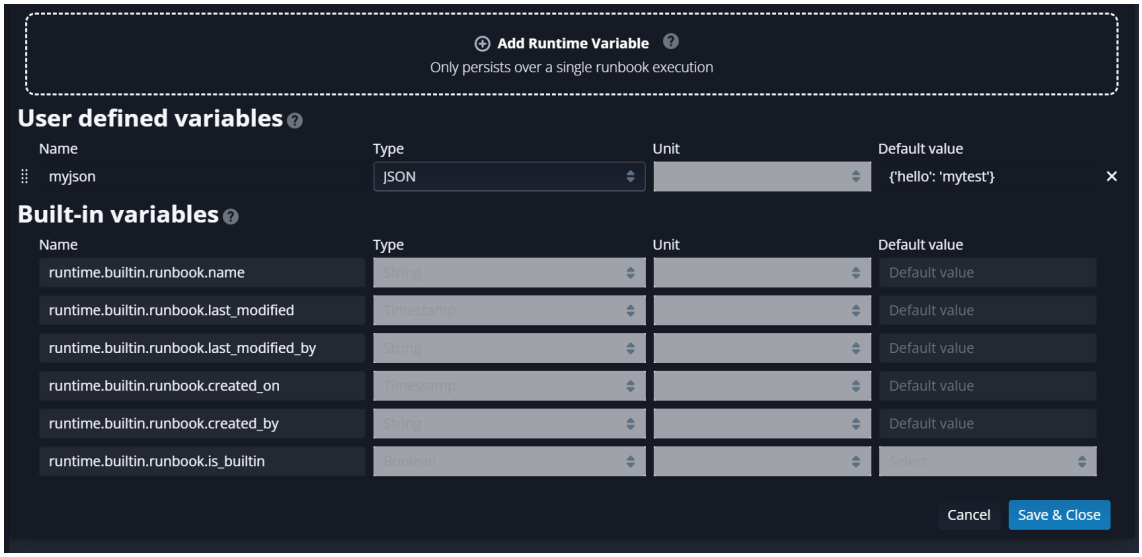
Variables of type JSON are useful in conjunction with Webhook triggers, making it easy to manipulate JSON content carried by a webhook so it can be used in other nodes in a runbook. Handling JSON in a variable also makes it usable as a subflow input, and you can use one or more subflows as ready-made components for processing JSON in runbooks (such as Riverbed IQ's built-in Parse Aternity Service Desk Alert subflow, for example). A JSON variable can be used in a number of node types, including HTTP Request and Transform nodes. In any case, your runbook or subflow will need to include a Transform node to convert the JSON to a format usable by other nodes.

Note that a JSON variable, being unstructured, cannot be used in a Decision Branch node.

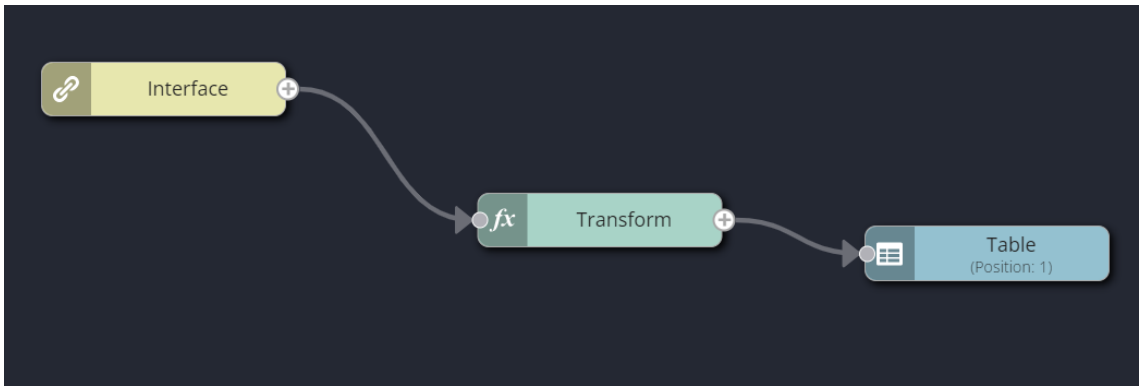
Example 1

A simple example follows to illustrate the use of a JSON variable and the Transform node.

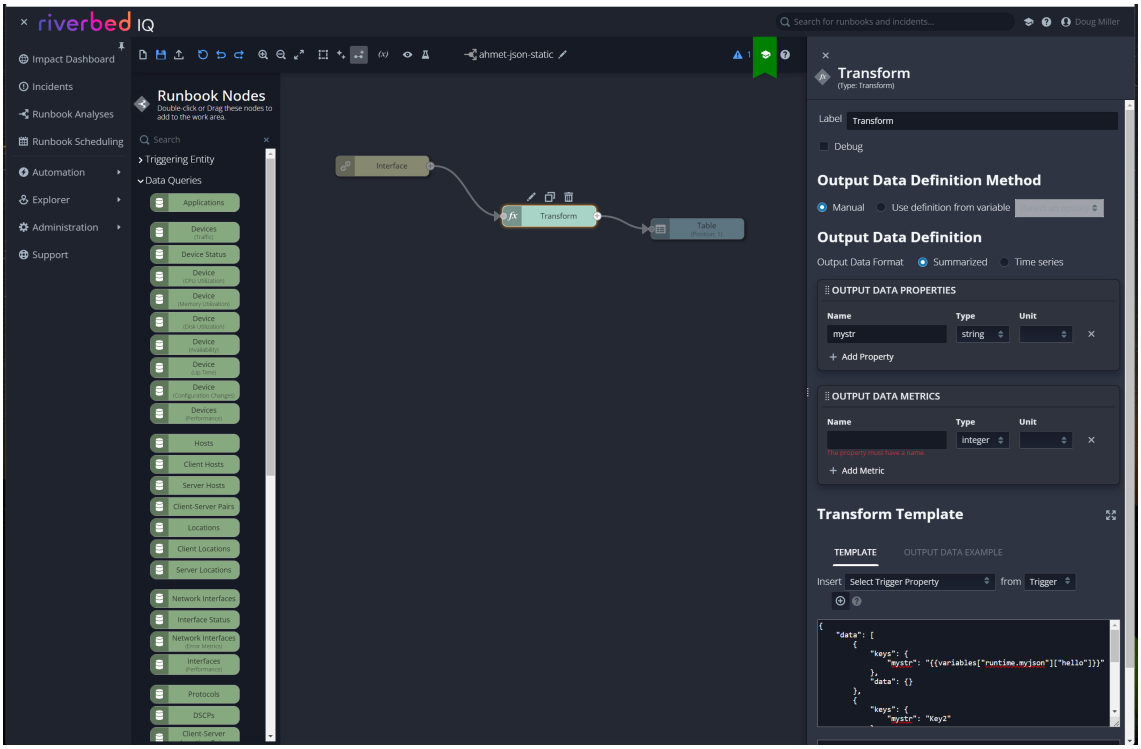
Add a user-defined variable such as this one, "myjson", shown below. The variable is initialized with a default key and value ("hello: mytest"), and can be used subsequently in the runbook.



This simple runbook illustrates the JSON variable's behavior:



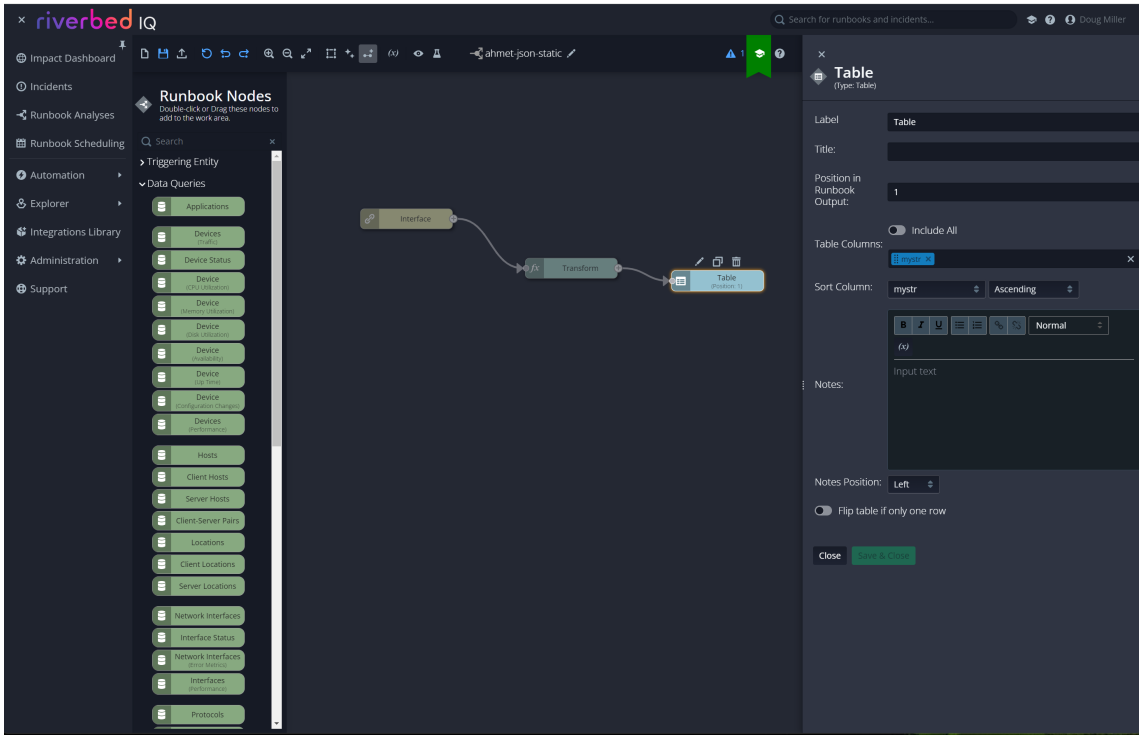
Edit the Transform node's properties. Note the addition of "mystr" in the Output Data Properties; this causes the Transform node to output the string value associated with "mystr". Now, look at the Transform Template. Specify Select Trigger Property from Trigger, and, in the Liquid template, assign "mystr" to the "myjson" runtime variable key "hello".



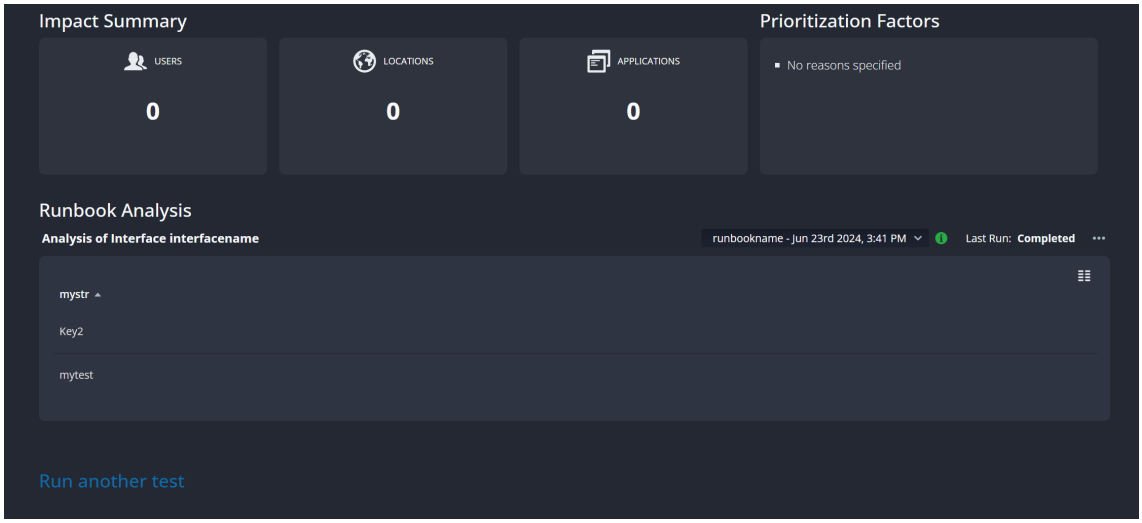
The complete Liquid template is:

```
{
    "data": [
        {
            "keys": {
                "mystr": "{{variables["runtime.myjson"]["hello"]}}"
            },
            "data": {}
        },
        {
            "keys": {
                "mystr": "Key2"
            },
            "data": {}
        }
    ],
    "info": {
        "keys": [
            {
                "id": "mystr",
                "label": "mystr",
                "type": "string",
                "unit": ""
            }
        ],
        "metrics": []
    }
}
```

Now, edit the Table node's properties. This output table will show a single column containing "mystr".

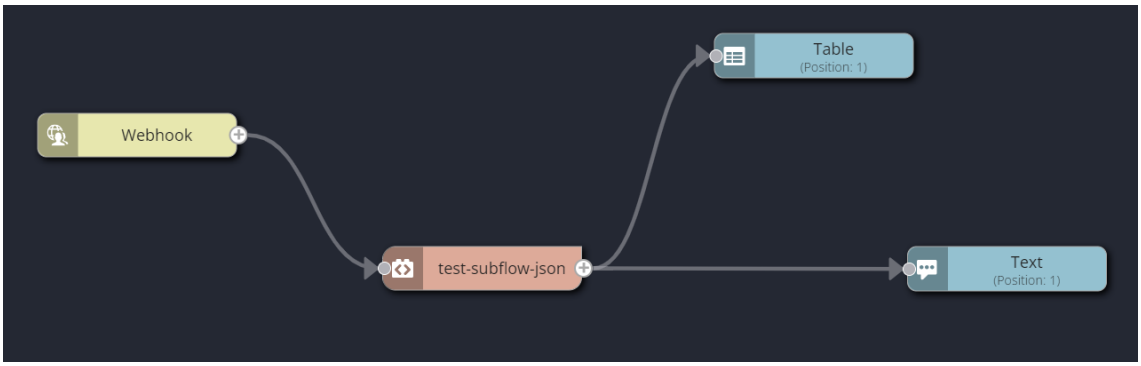


Execute a test run of the runbook; the output will look like so. Note the value "mytest" output for "mystr".

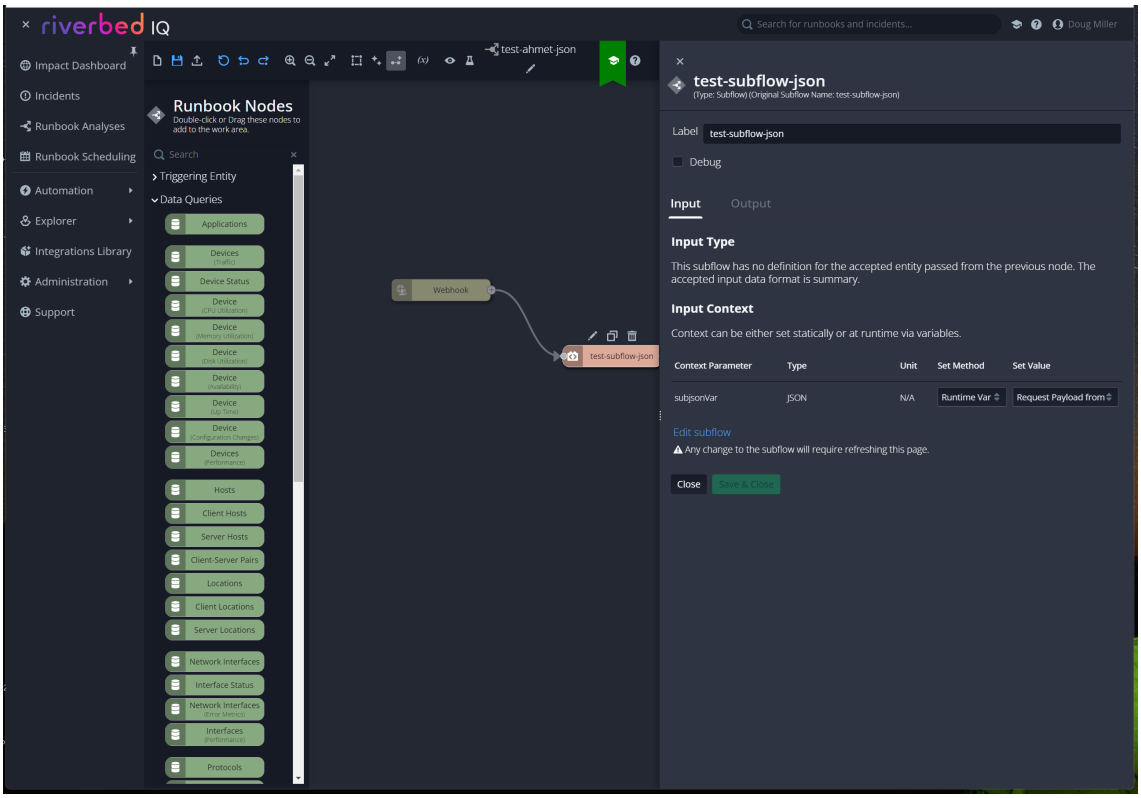


Example 2

A practical example of using a JSON variable with a Webhook trigger uses a subflow to parse data streamed to Riverbed IQ via a webhook. JSON carried by the webhook trigger is passed to the subflow, which will parse it and process it and pass it to the Table and Text visualizations.

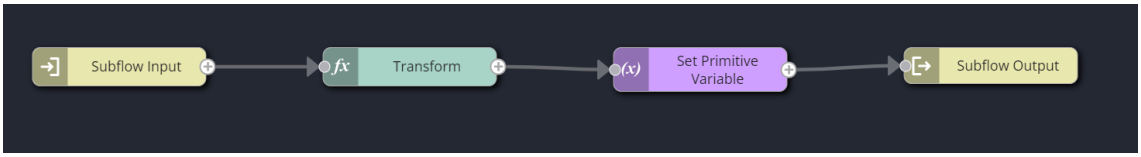


The subflow, named test-subflow-json, has these properties:

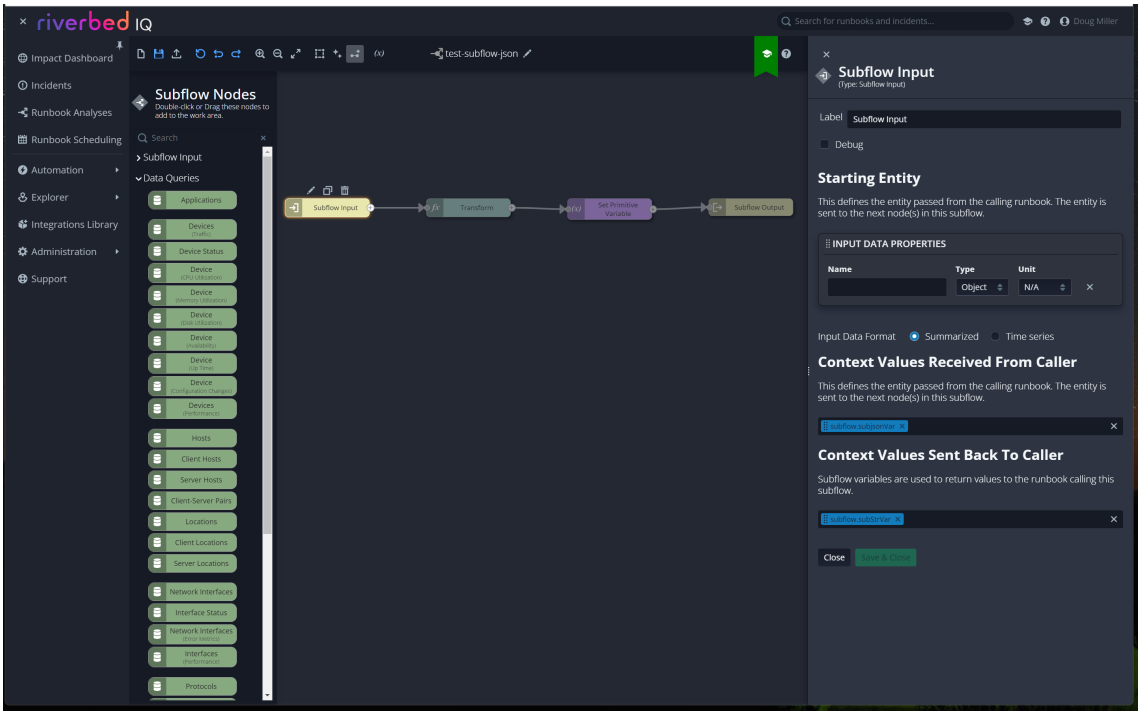


Note the Input Context for the subjsonVar variable. Content from the Webhook trigger is passed to the subflow as JSON variable input. The subflow passes output to the two visualization nodes, Table and Text.

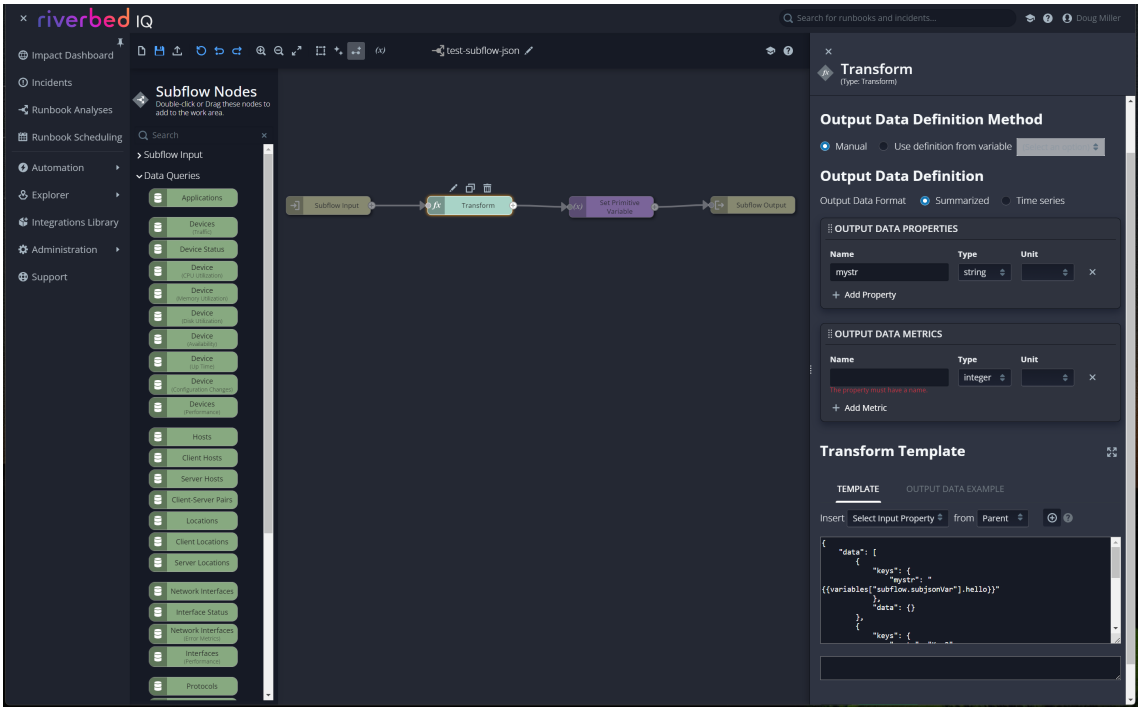
Edit ing the subflow shows its constituent nodes:



Edit the Subflow Input node's properties like so. Note the "subjsonVar" that is received from the parent runbook:



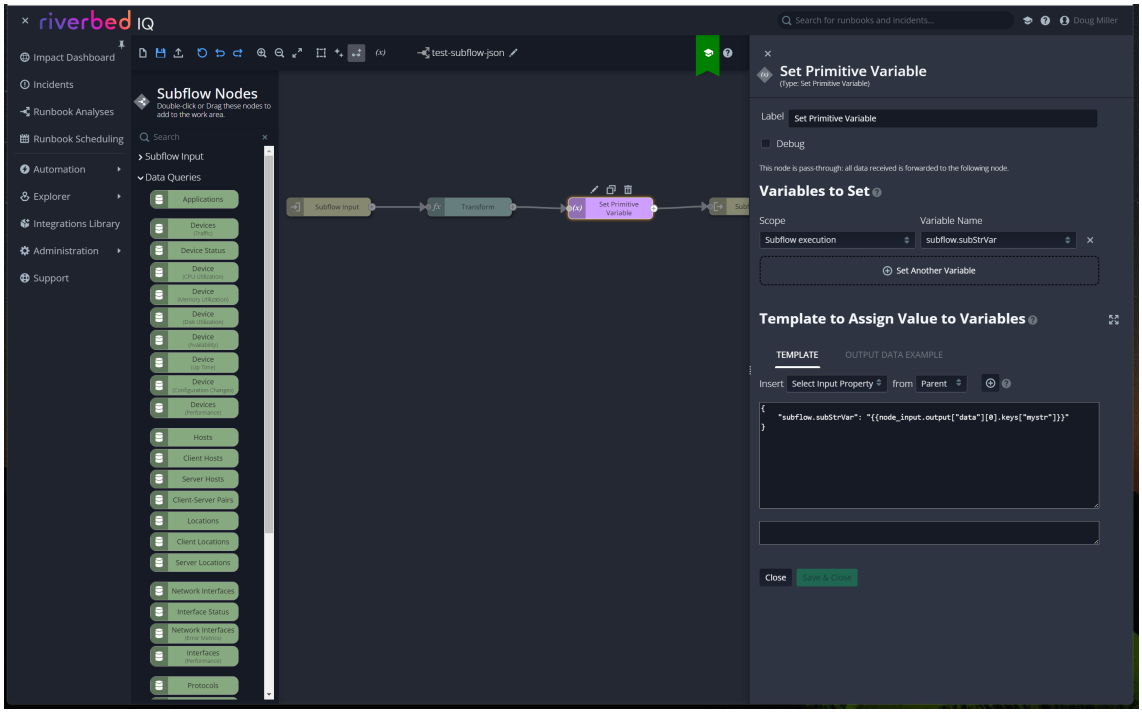
Edit the Transform node's properties and configure them as shown. Note the addition of "mystr" in the Output Data Properties.



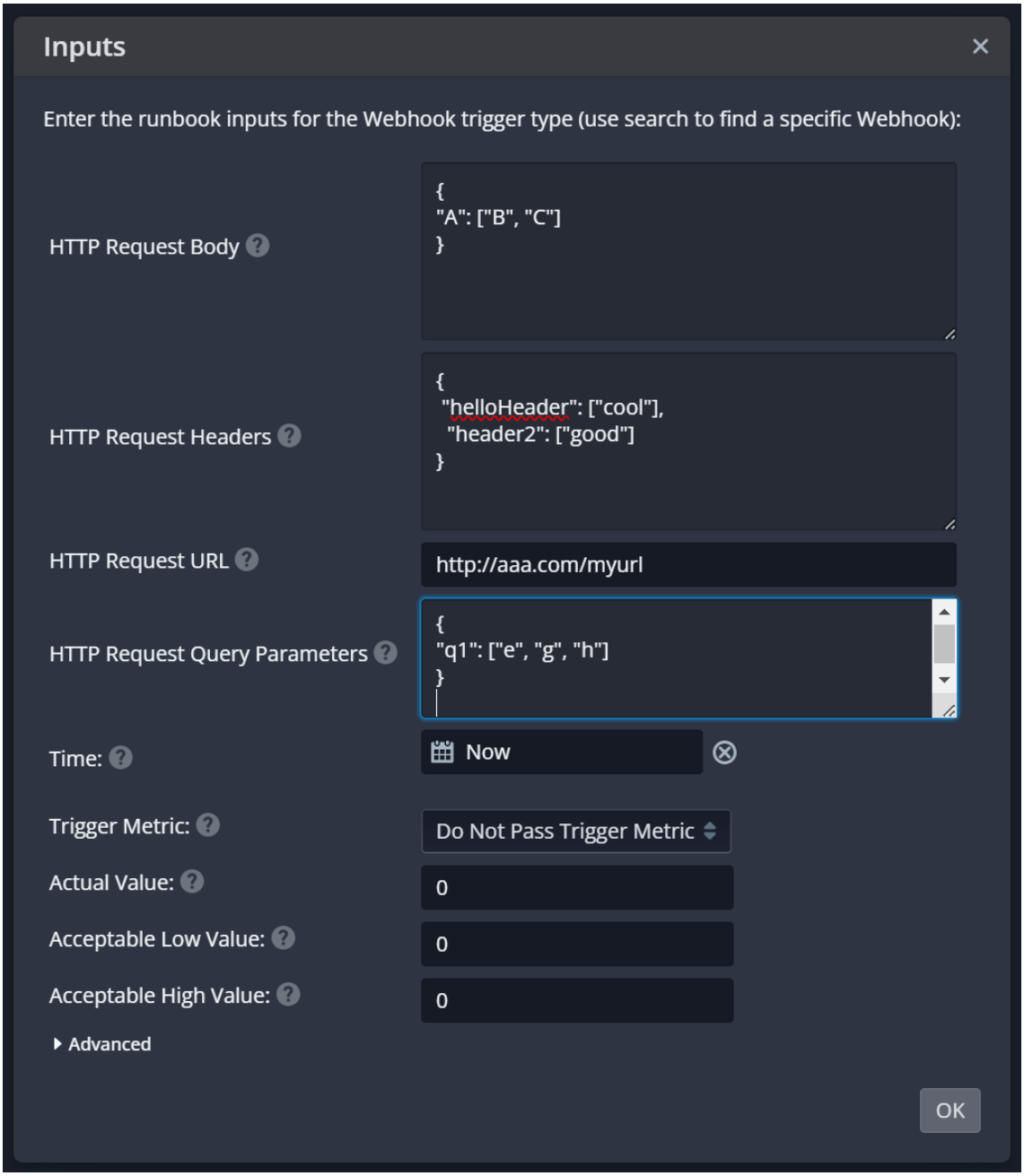
The complete Liquid template comprises the following (this accesses the Hello parameter the same as in the previous example):

```
{
    "data": [
        {
            "keys": {
                "mystr": "{{variables["subflow.subjsonVar"].hello}}"
            },
            "data": {}
        },
        {
            "keys": {
                "mystr": "Key2"
            },
            "data": {}
        }
    ],
    "info": {
        "keys": [
            {
                "id": "mystr",
                "label": "mystr",
                "type": "string",
                "unit": ""
            }
        ],
        "metrics": []
    }
}
```

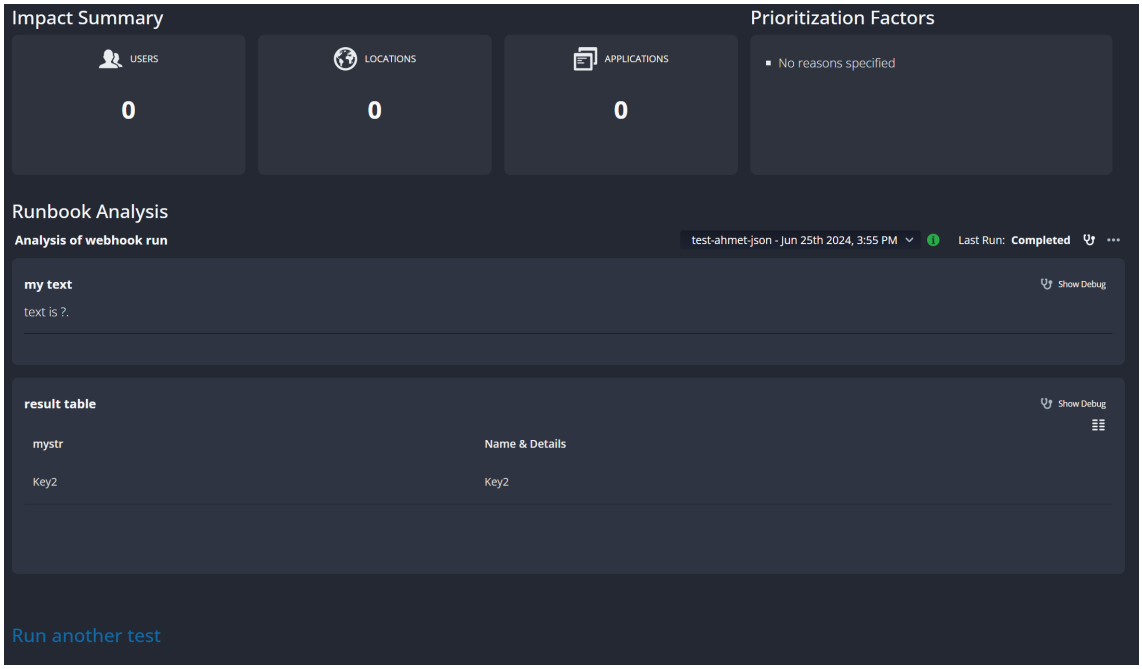
Edit the Set Primitive Variable node. The string variable gets set and is passed from the subflow back to the parent runbook via the Subflow Output node.



Execute a test run of the runbook. Use these test inputs:



The test execution results are:


Impact Summary
USERS
0
LOCATIONS
0
APPLICATIONS
0
Prioritization Factors
No reasons specified
Runbook Analysis
Analysis of webhook run
test-ahmet-json - Jun 25th 2024, 3:55 PM
Last Run: Completed
my text
Show Debug
text is ?.
result table
Show Debug
mystr
Name & Details
Key2
Key2
Run another test

On-Demand Runbooks

An on-demand runbook enables you to execute a runbook manually and immediately, which is distinct from incident runbooks and lifecycle runbooks that execute in response to specific events.

On-demand runbooks are useful for cases such as:

- *Discovery* — List everything known about an entity from various products (Riverbed and third-party).
- *Experimentation/testing* — Test a runbook being defined for remediation to validate it before automating it.
- *Routine IT troubleshooting* — The IT desk received a report of an issue at a specific time; execute a runbook for the reporting user, location, or application at that time.
- *Verification* — An incident occurred yesterday and is now fixed. Execute a runbook to validate the fix and ensure that everything is back to normal.
- *Change validation* — A network configuration was performed; verify that everything is still up and running, based on a checklist of items to validate.
- *Integration* — Integrate with a third party vendor and send it specific network data collected from Riverbed products.

On-demand runbooks are listed on their own On-demand Runbooks page, separate from other runbook types.

Create an on-demand runbook by clicking **New** on the On-demand Runbooks page to open the Runbook Editor. The new runbook is started on the canvas with an Input node added by default.

On-demand runbook definitions are similar to other Runbook types, with these exceptions:

- An on-demand Runbook starts with an Input node, not a Trigger node. The Input node requires you to explicitly supply the details that give the on-demand runbook the context it needs to be able to execute, information that an incident Runbook acquires implicitly from its Trigger node. (This also means that Decision nodes used in on-demand runbooks don't have a Trigger category.) On-demand runbooks can use any of the following entity types as inputs:
    - None (Execute the runbook on all the data without specifying any input.)
    - Interface
    - Device
    - Application
    - Host
    - Client
    - Server
    - Location
    - Client Location
    - Server Location
- Since on-demand runbooks are executed manually at your initiation, they are not managed as automations like triggered runbooks (incident, lifecycle, and external) are.
- On-demand runbooks do not use/support Impact Statements.
- On-demand runbooks do not use/support incident variables.

When you create an on-demand runbook that specifies an entity type, you will be prompted to provide that entity at execution, and that entity then can be used to filter all the data queries within the runbook. For example, for an application input, you will be prompted to provide an application; if you then specify that you want to execute for the Exchange application, you can define the

runbook such that all the runbook's Data Query nodes use Exchange as a filter in order to furnish additional information about Exchange.

Once the on-demand runbook has executed, its output is listed on the Runbook Analyses page.

Executing On-Demand Runbooks

On-demand runbooks can be executed from three places within the UI:

- On-Demand Runbooks page
- Runbook Analyses page
- Global search results for an on-demand runbook

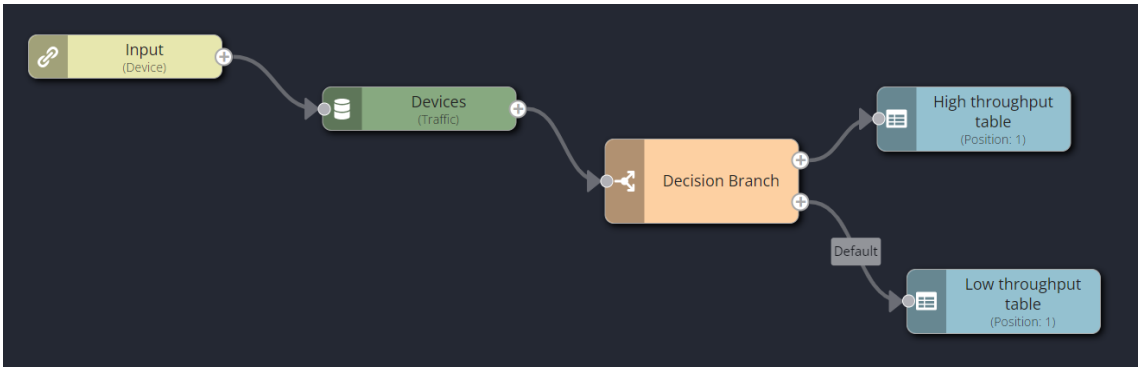
Executing an on-demand runbook uses the Run Runbook dialog:

1. Open the On-Demand runbooks page to see those runbooks listed.
2. For the on-demand runbook you want to execute, click **Run** to display the Run Runbook dialog. This dialog lists the parameters you need to supply in order for the runbook to execute. These correspond to the variables that were added to the runbook's Input node when the runbook was defined. (For comparison, incident runbooks don't use this dialog and its inputs because all the relevant context is supplied to incident runbooks by their triggers.) The parameters include:
    - Primary Entity — The type of network object for which the on-demand runbook executes.
    - Additional Parameters — Specify values to populate the variables included in the runbook's Input node.
3. Specify the Reference Time at which the runbook will examine data for generating output (the default is Now) and click **Run**. The runbook's output will be listed on the Runbook Analyses page.

On-Demand Runbook Example

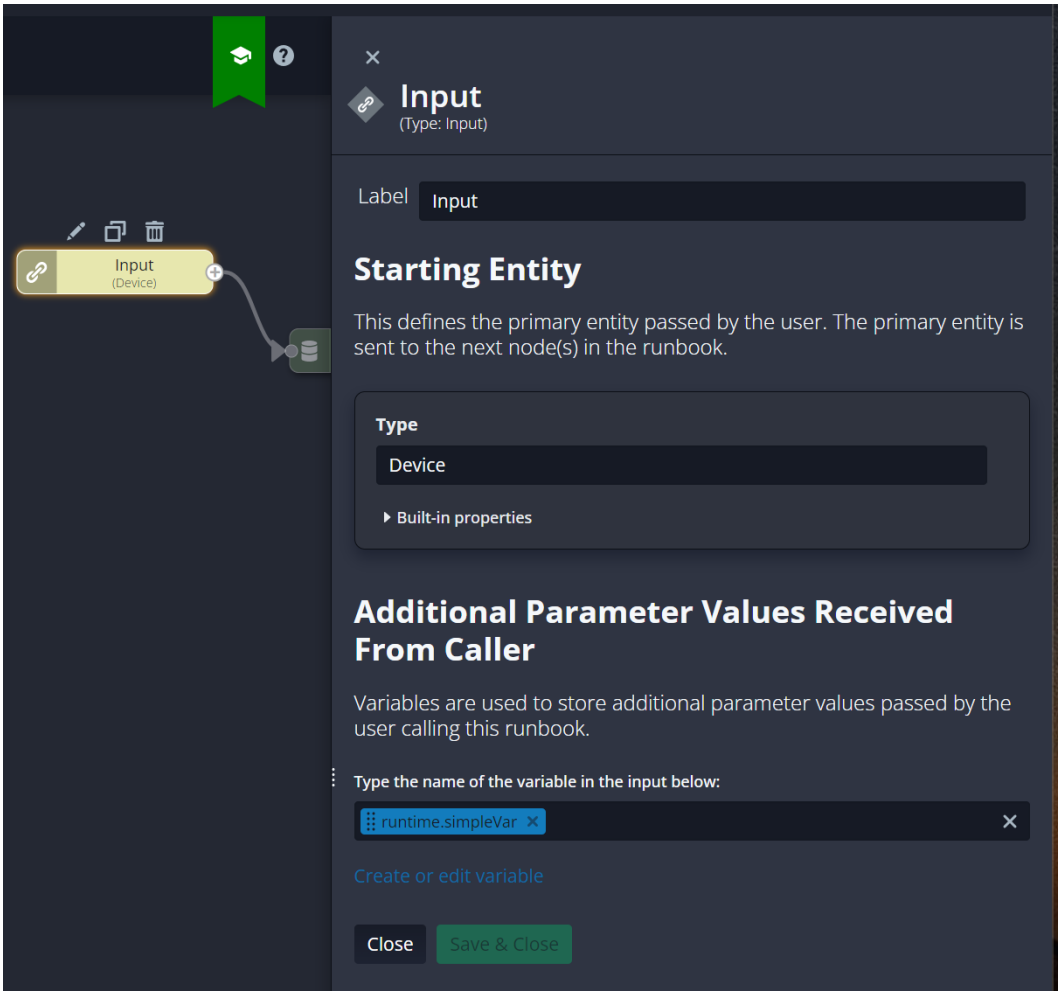
This simple example illustrates the use of on-demand runbooks. It's likely that your purposes will require you to define larger, more sophisticated runbooks than this, but this one provides a starting point to familiarize you with the fundamentals.

This on-demand runbook starts with an Input node that uses a device as its starting entity. The Devices (Traffic) data query node queries that device input for Throughput and Traffic data, which it forwards to a Decision Branch node. The Decision Branch node evaluates the data it receives; if Throughput is greater than 10 Mbps, a table for high throughput values will be shown. If the Decision Branch node receives a runtime variable named "test", a table for low throughput values will be shown instead.

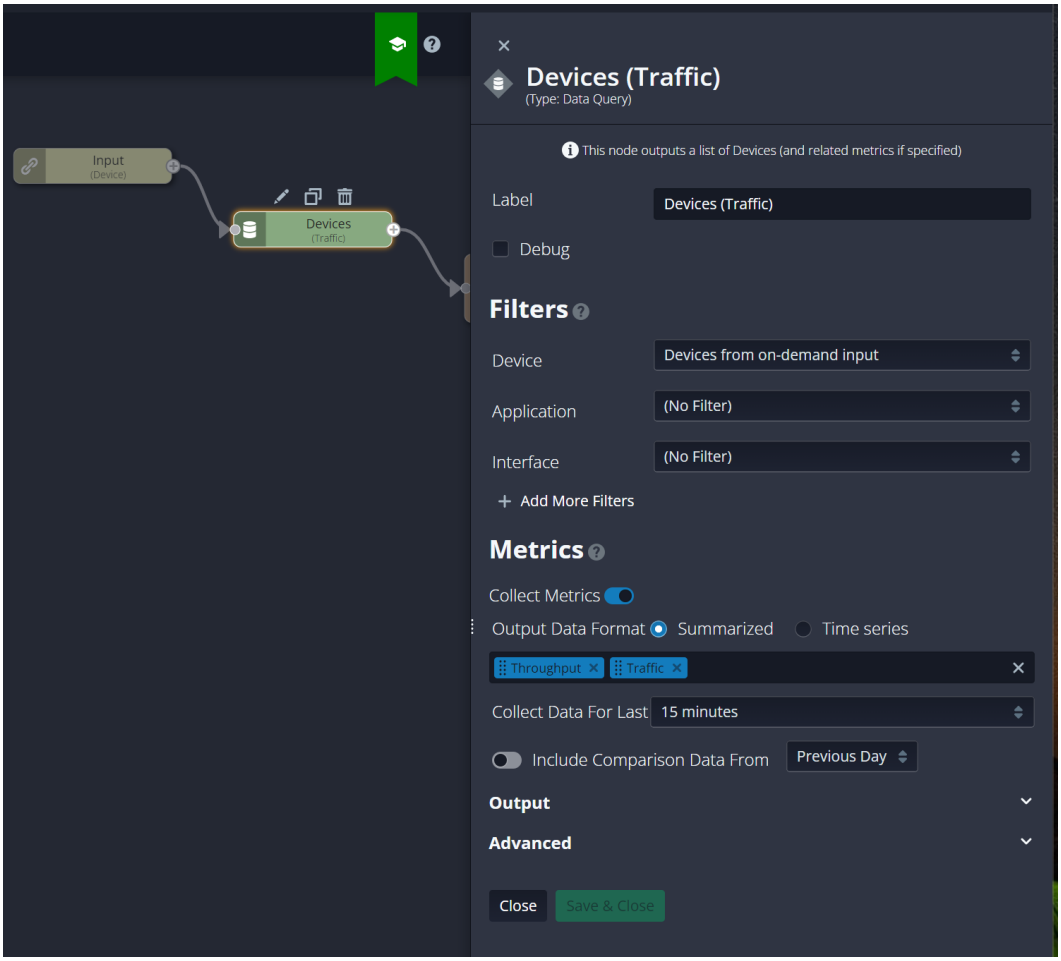


Taking each node in turn and examining its properties:

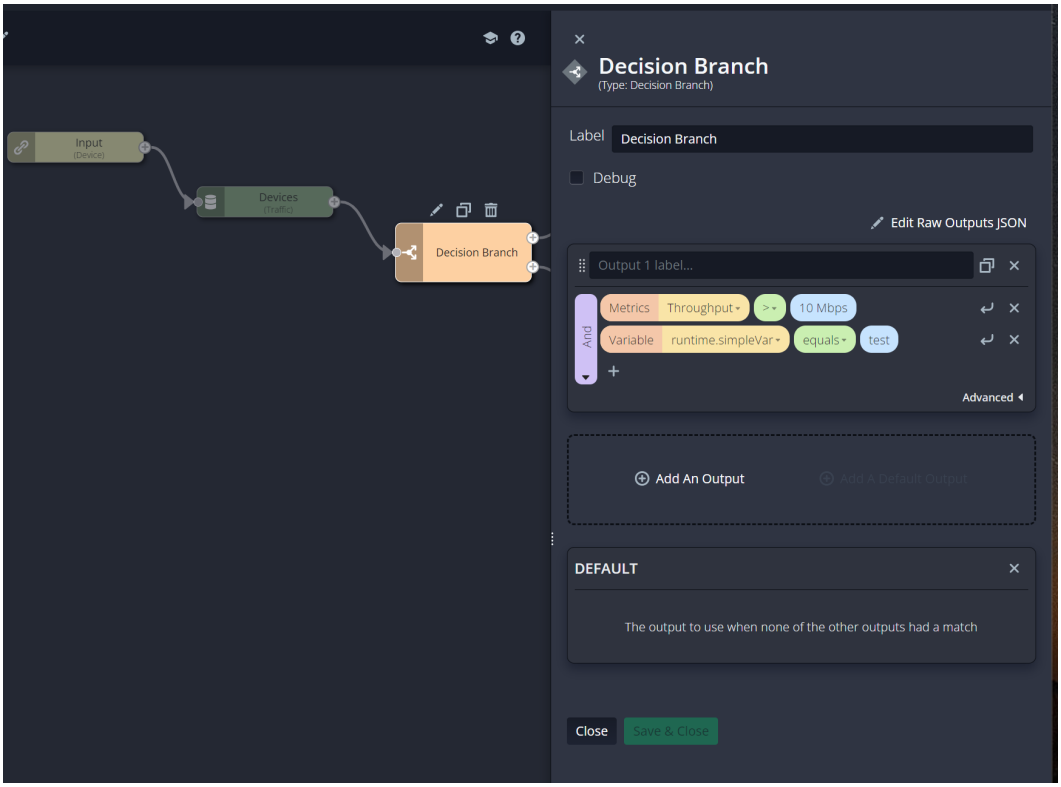
- The Input node specifies "Device" as the primary entity; this is the entity type for which the runbook is executed. Note also that there is a runtime variable named "simpleVar" (defined previously) that is carried with the device data.



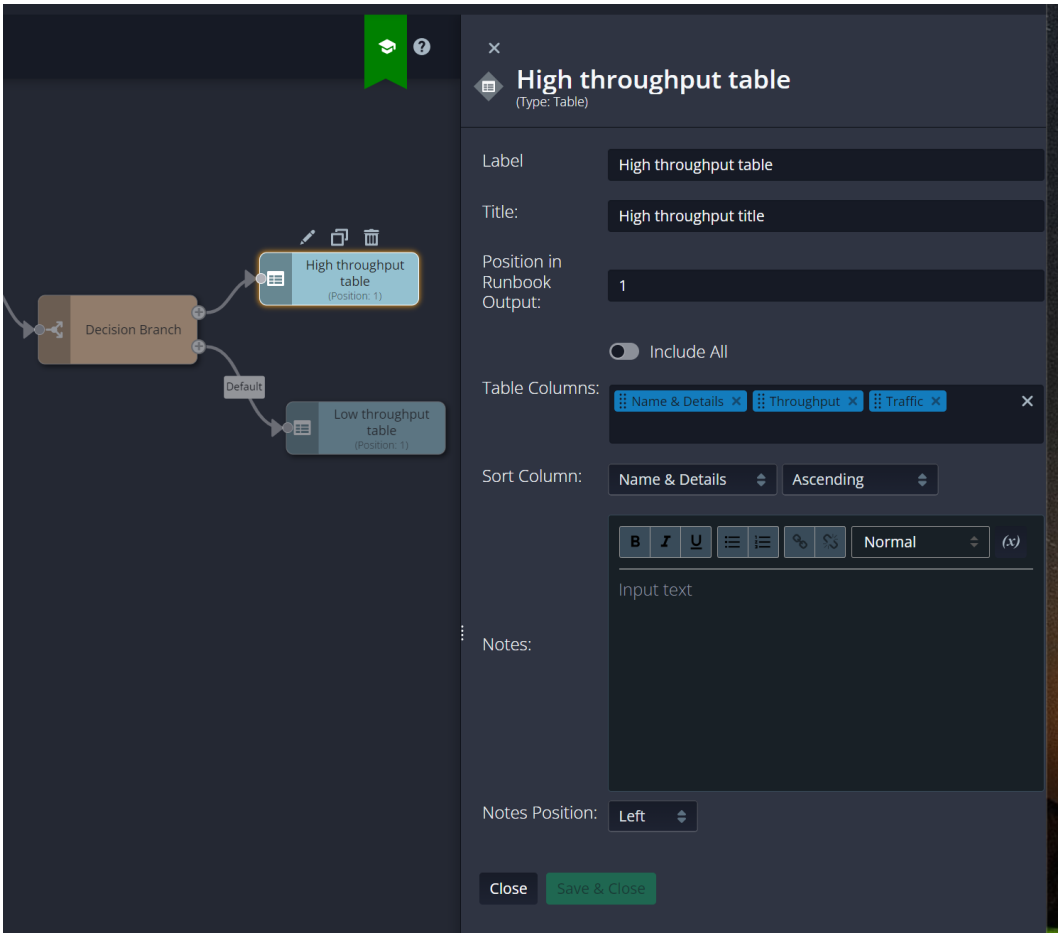
- The Devices (Traffic) data query node queries for device data from on-demand input; that is, from the Input node. Collect Metrics is enabled, and the Throughput and Traffic metrics are specified explicitly for collection.



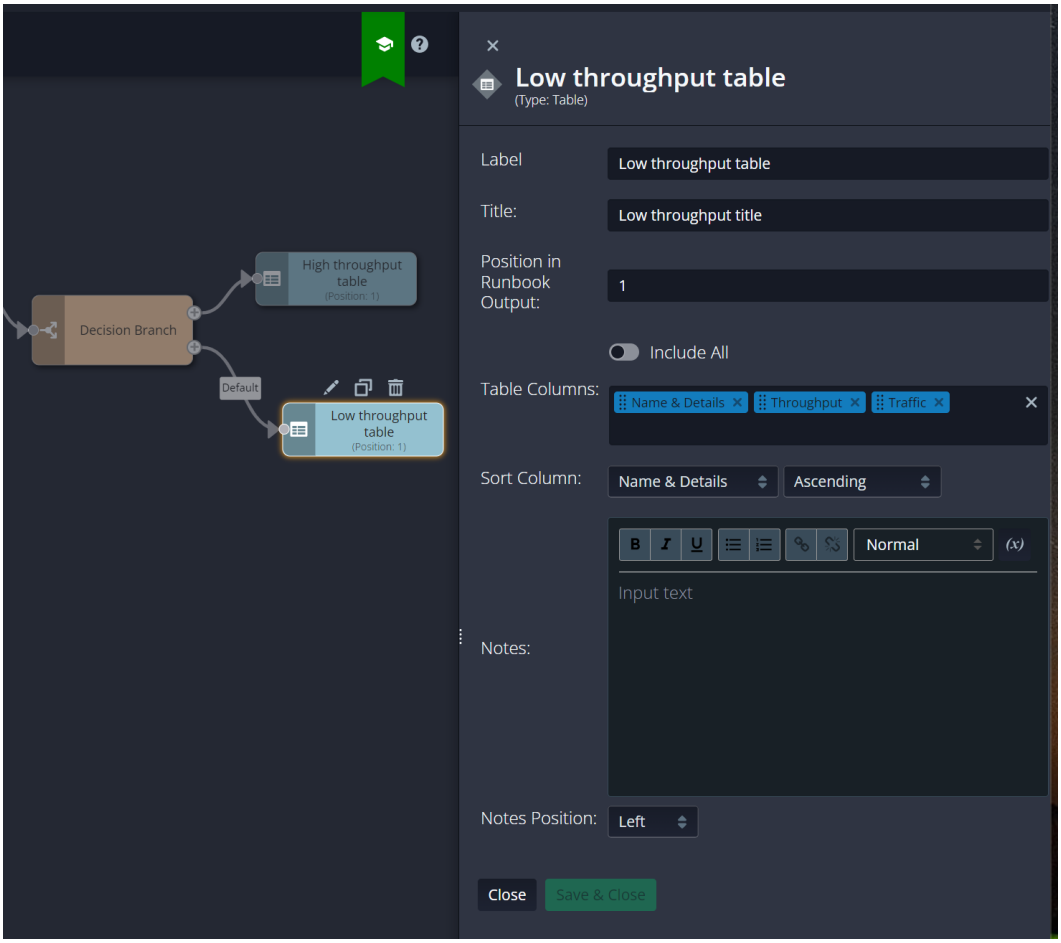
- The Decision Branch node evaluates two conditions to produce output: if the value of the Throughput metric is greater than 10 Mbps *and* if the value of the "simpleVar" runtime variable is "test".



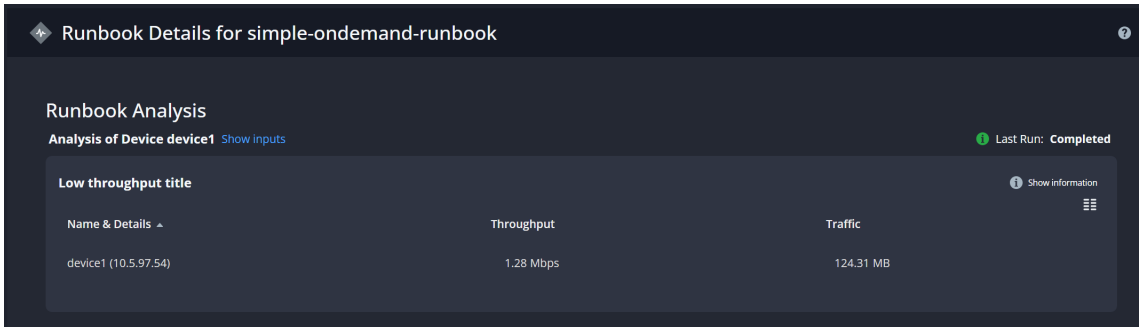
- The High Throughput Table node outputs three columns: Name & Details, Throughput, and Traffic.



- The Low Throughput Table node also outputs three columns: Name & Details, Throughput, and Traffic.



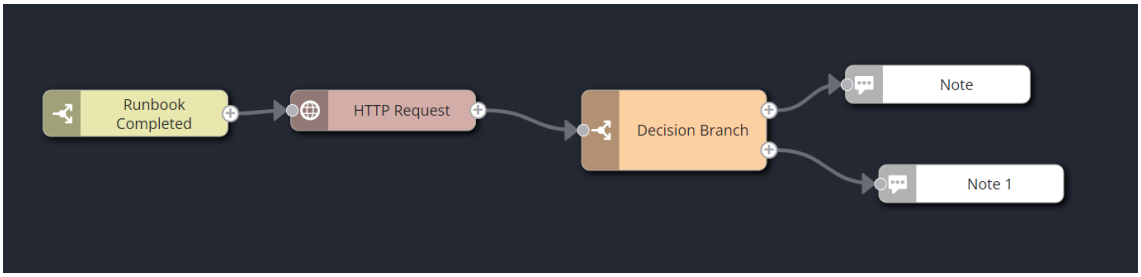
Execute the runbook from the On-Demand runbooks page to see the resulting analysis. For example:



Incident Lifecycle Events and Runbooks

Incident lifecycle *events* indicate the state of ongoing incidents. Incident lifecycle *runbooks* provide a tool for tracking incident lifecycle changes, and making that information available outside of Riverbed IQ, through, for example, messaging applications and service management applications. Unlike incident runbooks, lifecycle runbooks do not perform analysis or output data to visualizations (such as tables and charts) or impact statements. However, you can use notes to record the information that the runbook passes to external recipients.

A simple example of a lifecycle runbook follows. This runbook takes the Runbook Completed trigger information, posts it to an HTTP endpoint, and writes the result in one of two notes: One note for a 200 result code, another note for any other result.



Access Lifecycle Runbooks from the Lifecycle Runbooks page.

The Runbook Editor Node Palette furnishes these incident lifecycle triggers for defining lifecycle runbooks:

- Runbook Completed
- Status Changed
- Note Added
- Note Updated

- Indicators Updated
- Ongoing State Changed

The Automation Management page maps runbooks for these triggering entities:

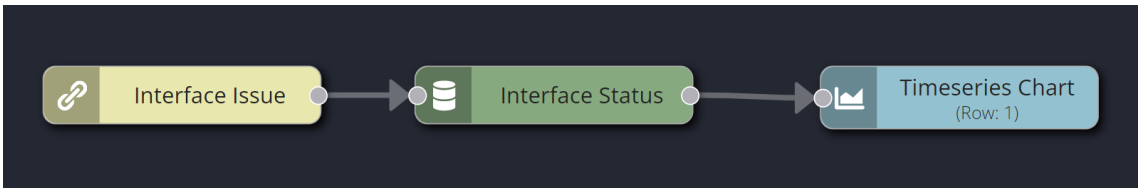
- Impact Analysis Ready
- Incident Indicators Updated
- Incident Note Added
- Incident Status Changed
- Ongoing Incident Changed

Simple Runbook Examples

Runbooks are automated investigations of network incidents. Very simply, a runbook executes in response to a triggering entity and produces one or more outputs (visualizations and impacts) that describe the incident. To produce output from the triggering entity, the runbook queries for data that it can manipulate in a variety of ways. All of these functions are represented by configurable runbook nodes, and every runbook, whether it is built-in or custom (user-created), is defined by a sequence of these nodes that starts with a single triggering entity and ends in one more visualizations or impacts. The nodes are visible in the runbook definition, and the Runbook Editor enables you to create and change runbook definitions by arranging and connecting nodes in sequence.

Note: After you've reviewed the example runbooks that follow, try copying one of the built-in runbooks and experimenting with changing its definition and executing test runs to see the results. This will help you familiarize yourself with runbook definition and configuration.

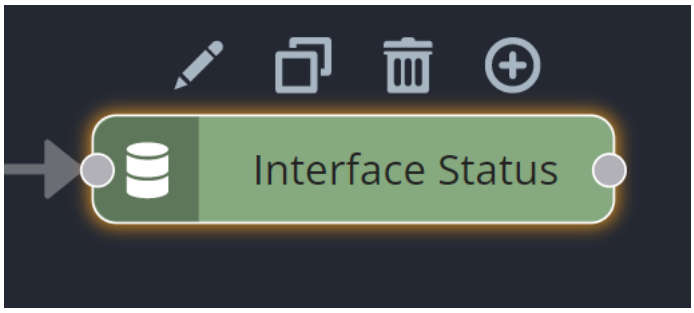
An extremely simple example of a runbook follows. This runbook executes in response to an Interface issue. The runbook queries for interface status information and forwards it to a time series chart, which displays interface status over time. The runbook definition looks like this:

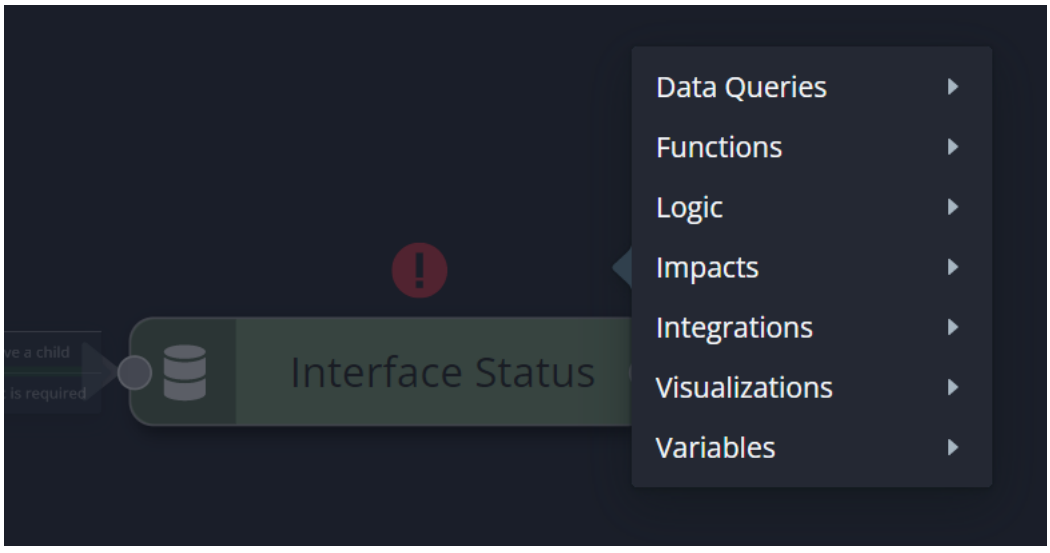


When the Runbook Editor is open, you can add a node to a runbook definition by dragging and dropping it from the Node palette at the left. As shown in the last example, a few node types are required for having a working runbook:

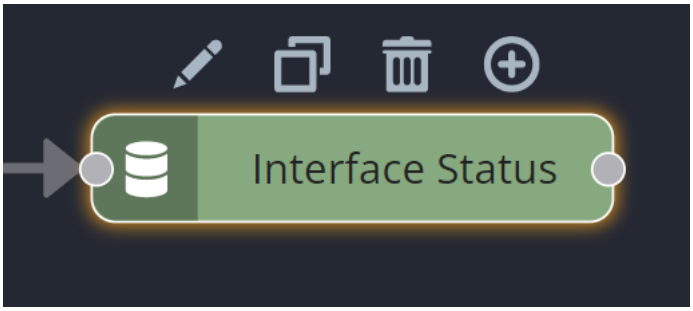
- You must have a triggering entity node (only one per runbook).
- You must have at least one Data Query node. This queries the Data Ocean for data related to the triggering entity and forwards it to nodes that follow it.
- You must have at least one Visualization node to show some results of the runbook's investigation.

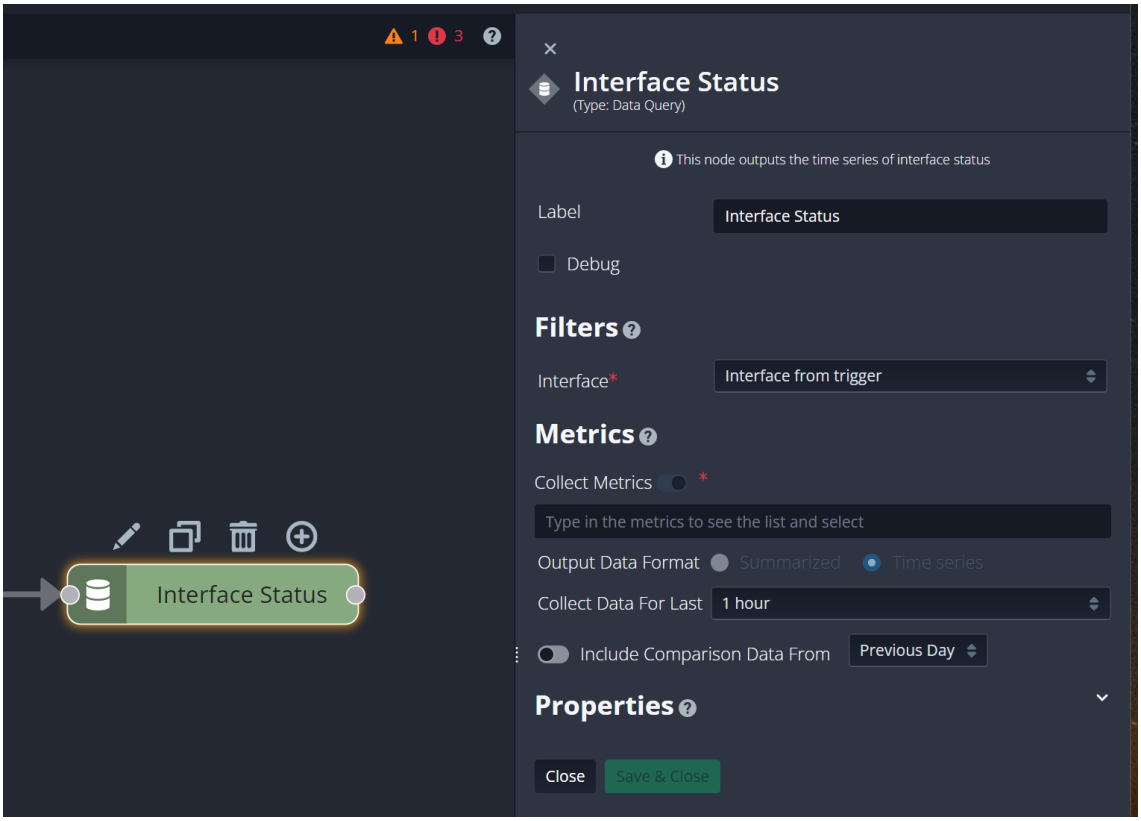
The other runbook node types are optional, but you'll want to use them in various use cases to obtain meaningful results. Note that not every node type is valid in every combination; some nodes require certain preceding node types, and some Visualization nodes require the data they receive to be either summarized or time series. The Runbook Editor will help you see what node types are valid for connecting next in a sequence if you select the last node and click the + icon above it to show a pop-up menu of valid node types you can connect next.





Every runbook node has a set of properties that control how it handles data. Depending on the node type, this can be the data it carries or the data it queries, or how it manipulates or displays data. Access a node's properties in the Runbook Editor by selecting the node on the canvas and clicking the pencil icon above it. The Properties open in a pane at the right.

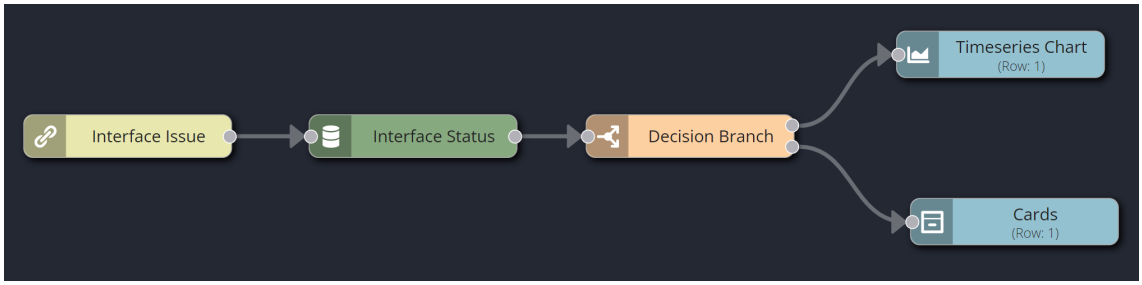




Note that the **Save & Close** button becomes accessible only if you change any of the properties.

As mentioned before, runbooks can include nodes that manipulate the data carried in the investigation. The Decision Branch node is a powerful node that

enables you to create branches in the runbook sequence according to the evaluation of one or more logical conditions. For example, the next example is based on the last one, but has a Decision Branch and a second visualization added to it.



Finally, take a look at the built-in runbook named Application Analysis. This runbook has a lot of nodes, but, if you begin at its triggering entity and follow its sequence along its separate branches, you'll see it's actually quite straightforward. Immediately following the Application Issue triggering entity, there are two Impact nodes, which call out all the applications and locations affected by the incident. Also attached directly to the Application Issue triggering entity is a Decision Branch, which, when you examine its properties, you'll see creates four different branches for the investigation to pursue: one for mean opinion score, one for response time/RTT, one for throughput, and one for the percentage of retrans packets. This one Decision Branch separates those different metrics into their own sequences for investigation. Note that each output of the Decision Branch can have mutliple nodes and sequences connected to it; Output 1, for which the triggering metric is MOS, connects to multiple distinct Data Query nodes, each one filtering for different MOS-related data, each one producing a different visualization or impact. Execute a test run to see the various outputs that result from four different paths of investigation from a single Decision Branch, stemming from a single triggering entity.

Now, try creating a new runbook that could investigate a common use case for your organization.

Creating and Editing Custom Runbooks Using the Runbook Editor

This topic is a high level overview of the workflow for creating and editing a custom runbook. For detailed instructions for choosing specific nodes and assembling them on the runbook canvas, refer to Adding and Assembling Runbook Nodes. Refer to Runbook Examples for walkthroughs of some sample runbooks.

Click **New** on the Runbooks page to open the Runbook Editor and begin defining a new runbook. For an existing runbook, click **Edit**.

The general workflow is to select a runbook node in the palette at the left and move it onto the runbook canvas at right. You can do this by dragging it to the canvas or by double-clicking it on the palette. The palette lists the runbook node categories in approximate order of how they should be assembled in sequence (first a trigger, then a data query, then a logic operation, then an impact or a visualization), but this is broadly advisory, and a complex runbook may have nodes from different categories arranged in myriad ways. Selecting a node on the canvas opens its properties at the right, enabling you to configure that node's behavior in the context of the larger runbook.

Begin defining a new runbook by choosing a trigger on the palette and moving it to the canvas. Each runbook can have a single trigger, which specifies the set of anomalies that are considered for investigation by the runbook.

In most cases, you'll want to add a data query next, to filter the Data Ocean in the context of the trigger. Choose a data query on the palette and move it to the canvas, to the right of the trigger you chose. If you have the **Auto-Connect** option active (it is active by default), an arrow will be drawn from the trigger to the data query automatically, connecting the two nodes in a sequence. If **Auto-Connect** is disabled, you'll need to select the trigger and draw the connection to the data query manually.

Each time you place a new node on the canvas, you can edit its properties, though you can do this in the Runbook Editor at any time, not only at placement. Node properties vary according to the node type, and can be as simple as setting the node name or as sophisticated as specifying metrics and thresholds to control the runbook's behavior.

Make sure to include at least one visualization for each runbook. The runbook will execute and complete its investigation without one, but you won't see any of the output unless there's at least one visualization to show it.

The Comment node (under the Miscellaneous category) enables you to annotate the runbook as you deem appropriate. Comment nodes don't execute or affect the runbook output, and you can use as many of them as you want to document the runbook's behavior.

When you're finished, click the **Save** icon in the tool bar to save the runbook. It will appear in the list of runbooks shown on the Runbooks page.

Adding and Assembling Runbook Nodes

This topic provides detailed instructions for choosing specific nodes and assembling them on the runbook canvas. For a high level overview of the workflow for creating and editing a custom runbook, refer to Creating and Editing Custom Runbooks Using the Runbook Editor. Refer to Runbook Examples for walkthroughs of some sample runbooks.

Click **New** on the Runbooks page to open the Runbook Editor and begin defining a new runbook. For an existing runbook, click **Edit**.

Begin defining a new runbook by choosing a trigger on the palette and moving it to the canvas. You can do this by dragging it to the canvas or by double-clicking it on the palette. Each runbook can have a single trigger, which specifies the set of anomalies that are considered for investigation by the runbook.

When you float the cursor over a node on the runbook canvas, you see a small tool bar pop up just above the node. You can click a button on this tool bar to **Edit** the node's properties, **Duplicate** the node, **Delete** the node, or **Add** a new node after it. If you click **Add**, a menu pops up next to the node, providing a filtered list of nodes that are valid to attach next in the sequence. **Add** is not available for nodes that terminate a runbook, such as visualizations and impacts.

Choose a Data Query node to filter the Data Ocean in the context of the trigger.

Edit the Data Query node's properties: Label, Filter, Limit, and Metrics.

Working With Decision Branches

Decision Branch nodes enable you to create separate paths for a runbook to execute, depending on the evaluation of a logical statement.

Importing and Exporting Runbook Definitions

You can import and export runbook definitions via JSON file.

To import a runbook definition:

1. Click **Import** on the Runbooks page. The Import dialog appears, prompting you to choose a JSON file to import.
2. Click **Save** to execute the import process. The new runbook is listed on the Runbooks page.

To export a runbook definintion:

1. Click **Export** in the Runbook editor. The runbook definition is shown as JSON code in the Export Runbook dialog.
2. Click **Copy to Clipboard** to collect the JSON code.
3. Paste the JSON code into an editor of your choice, and save it with a meaningful filename.

Copying a Runbook

You can copy an existing runbook by selecting it on the Runbooks page, clicking the ... menu at the right of its entry, and choosing **Duplicate** from the pop-up menu. The new runbook will be listed on the Runbooks page as "Copy of <original runbook name>". This is useful for using an existing runbook as a template for creating one or more new runbooks, particularly if the new runbooks are variants of the original.

Testing a Runbook and Previewing Its Output

The **Execute a Test Run** button in the Runbook Editor tool bar enables you execute a runbook and preview the output it will produce.

Deleting a Runbook

Click **...** at the right of a runbook's entry on the Runbooks page and choose **Delete** from the pop-up menu. Riverbed IQ will prompt you to confirm that you really want to discard the selected runbook. If you click the **Delete** button, the runbook will be discarded immediately and permanently. There is no undo once the runbook is deleted.

If you simply need to disable a runbook temporarily, toggle it Off in the Active field on the Runbooks page.

Refer to Creating Custom Runbooks Using the Runbook Editor for instructions for creating one, instead.

Label

Label – Type an informative name for the data query. You can keep the system-provided default if you wish. Labels are not required to be unique within a runbook, but assigning each node its own name is good practice.

Filters

Reduce the set of objects (applications, devices, interfaces) examined by this runbook to investigate the incident.

Application – Reduce the set of applications examined by this runbook to investigate the incident.

- **None** – No applications are excluded from consideration by the runbook.
- **Specific Apps** – Exclude all applications from consideration by the runbook except those that you list here, separated by commas.
- **Apps From Trigger** – Exclude all applications from consideration by the runbook except those that are forwarded from the trigger node.
- **Apps From Previous Node** – Exclude all applications from consideration by the runbook except those that are forwarded from the preceding node in the sequence.

Client – Reduce the set of clients examined by this runbook to investigate the incident.

- **None** – No clients are excluded from consideration by the runbook.
- **Specific Client Hosts** – Exclude all clients from consideration by the runbook except those that you list here, separated by commas.
- **Client Hosts From Previous Node** – Exclude all clients from consideration by the runbook except those that are forwarded from the preceding node in the sequence.

Device – Reduce the set of devices examined by this runbook to investigate the incident.

- **None** – No devices are excluded from consideration by the runbook.
- **Specific Devices** – Exclude all devices from consideration by the runbook except those that you list here, separated by commas.
- **Device From Trigger** – Exclude all devices from consideration by the runbook except those that are forwarded from the trigger node.
- **Device From Previous Node** – Exclude all devices from consideration by the runbook except those that are forwarded from the preceding node in the sequence.

Host – Reduce the set of hosts examined by this runbook to investigate the incident.

- **None** – No hosts are excluded from consideration by the runbook.
- **Specific Hosts** – Exclude all hosts from consideration by the runbook except those that you list here, separated by commas.
- **Hosts From Previous Node** – Exclude all hosts from consideration by the runbook except those that are forwarded from the preceding node in the sequence.

Interface – Reduce the set of device interfaces examined by this runbook to investigate the incident.

- **None** – No interfaces are excluded from consideration by the runbook.
- **Specific Interfaces** – Exclude all interfaces from consideration by the runbook except those that you list here, separated by commas.
- **Interface From Trigger** – Exclude all interfaces from consideration by the runbook except those that are forwarded from the trigger node.
- **Interface From Previous Node** – Exclude all interfaces from consideration by the runbook except those that are forwarded from the preceding node in the sequence.

Location – Reduce the set of locations examined by this runbook to investigate the incident.

- **None** – No locations are excluded from consideration by the runbook.
- **Specific Locations** – Exclude all locations from consideration by the runbook except those that you list here, separated by commas.
- **Locations From Previous Node** – Exclude all locations from consideration by the runbook except those that are forwarded from the preceding node in the sequence.

Server – Reduce the set of servers examined by this runbook to investigate the incident.

- **None** – No servers are excluded from consideration by the runbook.
- **Specific Server Hosts** – Exclude all servers from consideration by the runbook except those that you list here, separated by commas.
- **Server Hosts From Previous Node** – Exclude all servers from consideration by the runbook except those that are forwarded from the preceding node in the sequence.

Limits

Constrain the runbook investigation in terms of a specific metric over a fixed time interval.

Limit To Top *n* objects – Specify a maximum number of objects to consider in the runbook investigation. The default number is 10.

Based On – Constrain the runbook investigation by focusing on a single metric.

Over The Last – Specify a time interval for which to look back at the selected metric.

Metric Collection

Specify the metrics to be collected for consideration by the runbook to investigate the incident.

Collect Metrics – Toggle this to specify which metrics to collect, and how that metric data will be formatted, for what time interval.

Output Data Format – Choose **Summarized** or **Time Series**.

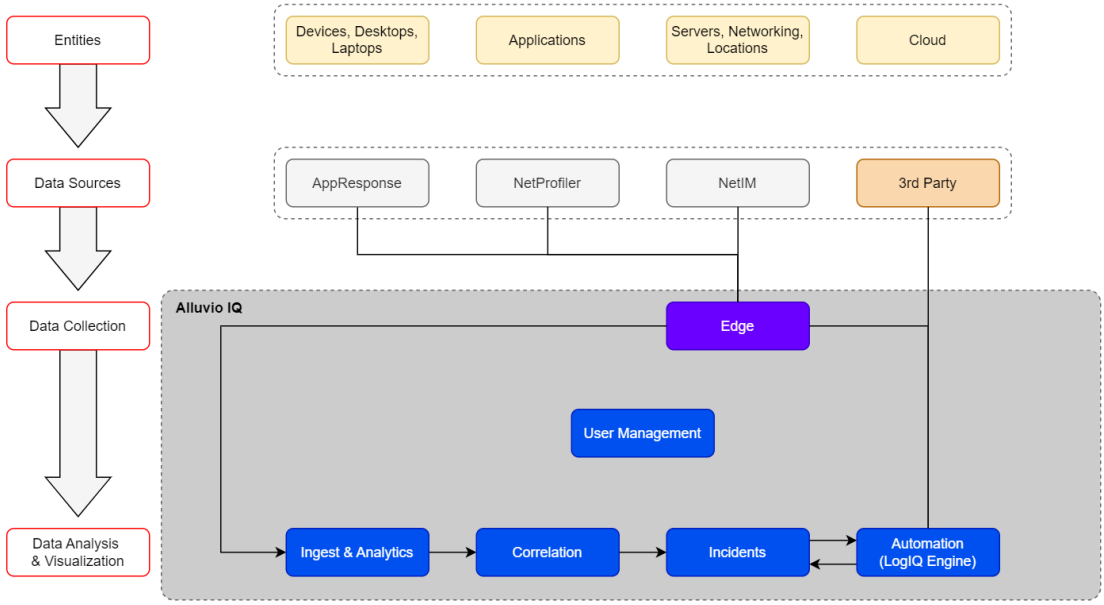
Collect Data For Last – Specify a time interval over which to acquire metric data.

Data Ocean Metrics

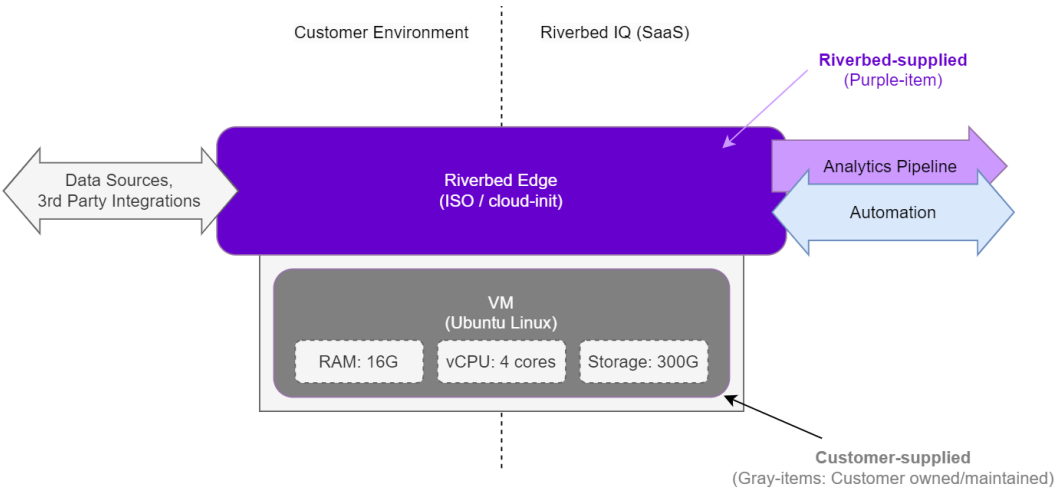
Riverbed IQ tracks the following metrics, and runbooks use them when investigating incidents. Note that not all metrics are relevant or available for use for all incident types and runbook nodes.

Riverbed Edge

Riverbed Edge is a component of Riverbed IQ that you deploy in your network or in the cloud to act as the data broker between Riverbed IQ and other Riverbed Technology products that are deployed in your network (such as NetIM and NetProfiler).



Riverbed employs a shared responsibility model for Riverbed Edge that comprises a Riverbed-supplied Edge ISO image or cloud-init file that is deployed on top of a customer-supplied and managed virtual machine (VM).



This presents a shared-responsibility model in which:

- Riverbed supplies Edge functionality as an ISO/cloud-init file.
- Riverbed Edge runs on a VM that is: owned/maintained by the Customer and configured according to their specific processes and procedures.

The basic Riverbed Edge VM requirements are:

- a host VM supplied by the customer.
    - a host VM running the validated version of Ubuntu Linux.
    - a host VM that provides the following minimum resources:
        - RAM: 16GB,
        - vCPU: 4 cores
        - Storage: 300GB.

Riverbed Edge supports deployment on the following virtual machine environments: AWS (via AMI), Azure (via VHD), and VMware ESXi (via OVA).

Riverbed Edge Requirement Type	**Riverbed Edge Requirement Specifications**	**Notes**
VM image	Ubuntu Linux v22.04	Supports deployment in the following environments: AWS, Azure, and VMware ESXi
VM RAM resources	16G	
VM vCPU	4 cores	
VM storage	300GB	

Data Source

A data source is a Riverbed Technology product in your network that forwards data to Riverbed IQ. This data can be of these types:

- Streaming data that Riverbed IQ uses to detect anomalies and generate incidents.
- Data that can be fetched on demand when runbooks are executed to make decisions and/or display additional useful information about an incident that was detected.

Indicator

An indicator is an observed change in a specific metric stream that is recognized as being outside of an expected model in one or more of these ways:

- Relative to immediate prior history
- Relative to configured thresholds
- Relative to long term, seasonally adjusted prior history
- As a significant change in absolute magnitude
- Due to an explicit notification, such as device down or service unreachable

Indicators are correlated into triggers, and one or more triggers are grouped into incidents. An incident is based on a primary indicator associated with the primary entity used for an associated runbook. Riverbed IQ also uses other criteria to determine if there is any commonality or relationship among other detected anomalies. Indicators that correlate across these criteria are associated with the incident as correlating indicators, providing additional data to the incident.

Incident

An incident comprises one or more related triggers. (Each trigger comprises one or more indicators that have been correlated.) Incidents are listed on the Incidents page, which shows information about the incident, including its basic details, along with runbook output data to assist you in identifying the incident's underlying cause.

Certain relationships cause triggers to be grouped into incidents:

- A trigger by itself
- Application
- Location
- Operating system

Riverbed IQ's default behavior is to group triggers into incidents only if they share two or more types of relationships. For example, two triggers will be grouped into an incident if they share both application and location, but not if they share only an application. However, you can override the system's default behavior and group triggers into incidents manually, or, you can break incidents apart, based on your personal knowledge and judgment.

Runbook

A runbook is an automated workflow that executes a series of steps or tasks in response to a triggered event. Such events can be the detection of anomalous behavior generating an incident, a lifecycle event in response to state changes within an incident, or manually executed runbooks that collect critical data to assist users.

A runbook comprises a set of automated procedures, represented by runbook nodes, which are assembled in sequence to define the automated investigation. See the Runbook page description and the Incident page description for learn how runbooks are used to investigate incidents.

Some runbooks are built in to Riverbed IQ, and it is possible to define your own custom runbooks or import them from a file.

Trigger

Each trigger is a set of one or more indicators that have been correlated based on certain relationships. A trigger can be the result of a manual action, also. Triggers are grouped into incidents. One or more indicators that are correlated constitute a detection which may act as a trigger.

Relationships between indicators that constitute a basis for correlation include:

- Time (indicators occurring at approximately the same time)
- Metric type (e.g.: RTT increase, drop increase, bandwidth exceeds 85%)
- Application affected
- Location
- Network device

For example: “30 indicators are identified for slower-than-expected RTT for application Acme for 10 different endpoints at location: Vancouver”. The trigger contains all 30 indicators because of relationship “application” and relationship “location”.

Support Page

The Support page enables you to contact Riverbed Technology directly to report a problem or request assistance specific to your use of Riverbed IQ.

You can access the general Riverbed Support web site at https://support.riverbed.com/

Frequently Asked Questions

General

- What is Riverbed IQ?
- What is the release schedule? How frequently will Riverbed IQ deliver new capabilities?
- Which Riverbed Network Performance Monitoring (NPM) products are supported with Riverbed IQ?
- Does Riverbed IQ support third-party integration?
- Does Riverbed IQ learn from the customer environment?

Data Ocean

- What are the implications of moving the data from our large on-premise data sources to Riverbed IQ SaaS?
- What data is streamed from available Riverbed Network Performance Monitoring (NPM) products into the Riverbed IQ Analytics Pipeline?
- What data is available to Riverbed IQ for automated investigation?
- Is it possible to customize the data processed by Riverbed IQ?

Automation and Runbooks

- How many runbooks are included, and can customers create their own?
- What skills are needed to build a runbook?

Security

- Does Riverbed IQ allow a user to self-service a password reset?
- What security standards does your SaaS meet?
- What protocol is used for communication to the Edge device and from the Edge to the SaaS? Is it TLS 1.3? Secure Restful API? Other?

General

What is Riverbed IQ?

- Riverbed's new AIOps software as a service (SaaS) is named Riverbed IQ, and it provides unified observability with a data analytics pipeline and a full suite of automation capabilities that enable it to surface critical issues and automate an investigation.
- Additionally, Riverbed offers a suite of observability products in our portfolio (AppResponse, Aternity, NetProfiler, NetIM, and Portal) - and now, with the arrival of Riverbed IQ, Riverbed provides an overarching AIOps SaaS to bring it all together.

What is the release schedule? How frequently will Riverbed IQ deliver new capabilities?

- Riverbed IQ is a software as a service (SaaS) solution.
- As a SaaS solution, Riverbed IQ can grow/evolve quickly, and releases updates on a regular basis in order to facilitate rapid expansion of capabilities.

Which Riverbed Network Performance Monitoring (NPM) products are supported with Riverbed IQ?

- The supported data source types (and recommended versions) are:

  Riverbed IQ supports the three most recent data source release versions.

Data Source Type	Minimum Version Required	Recommended Version
NetIM	2.6.1	2.7 and above
NetProfiler[1]	10.24.2	10.25 and above
AppResponse[2]	11.12	n/a
Aternity	*n/a*	*n/a*

1. NetProfiler will dictate requirement(s) for any subtending AppResponse.
2. These version requirements apply to AppResponse connecting directly to IQ.
   - There are no hardware requirements for Riverbed data sources, just the software version.
   - Customers will also need to install Riverbed Edge in their enterprise environment to interconnect on-premise data sources with the Riverbed IQ SaaS platform.

Does Riverbed IQ support third-party integration?

- Riverbed IQ SaaS provides a generic HTTP-based framework to support two variations of third-party integrations:
  - Riverbed IQ Webhooks enable third-party tools/systems to execute Riverbed IQ runbook automations on Riverbed IQ by calling a Riverbed IQ API.
  - Riverbed IQ runbook HTTP Request nodes enable Riverbed IQ to interact with third-party tools/systems (via third-party API) while executing an automated investigation.
- This generic HTTP-based approach provides a great deal of flexibility and allows customers to not be locked in to specific integrations.
- Riverbed Professional Services can use the APIs to build additional integrations as needed.

Does Riverbed IQ learn from the customer environment?

- Yes, Riverbed IQ learns from the customer environment in multiple ways:
  - Key Measurements — These are metrics that stream into Riverbed IQ from connected data sources that provide full-fidelity observability into the performance of the entities in the customer environment. In this way, Riverbed IQ learns of the various entities in the customer environment.

- Analytics Pipeline — The Riverbed IQ analytics pipeline applies machine learning (ML) algorithms to model the behavior of key measurements in order to continuously learn what is normal for the customer environment, and evolves as the behavior changes to surface unexpected anomalies. In this way, Riverbed IQ learns the behavior/patterns of the various entities in the customer environment.
- Runbook Automations — Automation capabilities enable Riverbed IQ to gather additional data over the course of an automated investigation and build a full 360-degree view of the environment for the time the event occurred. In this way, Riverbed IQ is able to reconstruct the environment at the time of the event, and apply logic to surface actionable insights based on that information.
    - Additionally, Riverbed IQ runbooks can be customized to incorporate organizational knowledge and thereby tune automated investigations to the customer environment.

Data Ocean

What are the implications of moving the data from our large on-premise data sources to Riverbed IQ SaaS?

- Riverbed IQ has access to a wide array of data served up by the Data Ocean. The Data Ocean comprises all data sources connected to Riverbed IQ, including Riverbed Network Performance Monitoring (NPM) products, and any integrated third-party tooling/infrastructure.
- The vast majority of the data available to Riverbed IQ in the Data Ocean is retained in the constituent data sources (i.e., it is not moved to SaaS).
- Only a small subset of the data available in the Data Ocean is moved to SaaS, which amounts to negligible traffic volume:
- Key Measurements — These are the subset of high-value metrics that are streamed from Riverbed Network Performance Monitoring (NPM) data sources to Riverbed IQ SaaS. Key measurement data is

compressed and encrypted, and amounts to a negligible volume of traffic that is streamed to the SaaS.

- Automation/Queries — Riverbed IQ automation is able to tap in to all data sources connected to Riverbed IQ in the course of an automated investigation, including Riverbed Network Performance Monitoring (NPM) products, and any integrated third-party tooling/infrastructure. Riverbed IQ automation has the ability to target retrieval of specific additional supporting data/context from a data source to help execute an investigation (i.e., additional metrics, properties, attributes, other integrated third-party information/capabilities, …). Again, this also amounts to a negligible volume of traffic as it encompasses a very targeted subset of available data.

What data is streamed from available Riverbed Network Performance Monitoring (NPM) products into the Riverbed IQ Analytics Pipeline?

- Riverbed IQ has access to a wide array of data served by the Data Ocean. The Data Ocean comprises all data sources connected to Riverbed IQ, including Riverbed Network Performance Monitoring (NPM) products, and any integrated third-party tooling/infrastructure.
- Riverbed Network Performance Monitoring (NPM) products are currently the only data sources that stream a preconfigured subset of high-value metrics that are leading-indicators of critical issues (i.e. key measurements) into the Riverbed IQ analytics pipeline for processing.
- The following table summarizes the key measurements that can be streamed from the various Riverbed Network Performance Monitoring (NPM) products:

Alluvio Analytics Pipeline - Quick Reference

Data Source Type	Entity	Key Measurement (Streamed Metric)
NetProfiler	Application /	User Response Time[1]

	Client Location	MoS
	Interface	In Utilization
		Out Utilization
AppResponse	Application / Client Location	User Response Time[1]
		Throughput[2]
		% Retrans Packets
		% Failed Connections
NetIM	Device	Device Status
		Device Uptime
	Interface	Interface Status
		In Packet Error Rate
		Out Packet Error Rate
		In Packet Drops Rate
		Out Packet Drops Rate
		In Utilization
		Out Utilization
Aternity	Application / Client Location[3]	Activity Network Time
		Page Load Network Time

Notes:

1 - [Metric: User Response Time] is:

- an approximation of AppResponse [user-response-time] because NetProfiler does not yet account for [connection_setup_time] - while AppResponse does, i.e.

AppResponse User Response Time calculation: [user-response-time]

= ([connection_setup_time] / [connection_setup_time_n])

+ ([request_network_time] / [request_network_n])

+ ([response_network_time] / [response_network_n])

+ ([server_delay] / [server_delay_n])

NetProfiler User Response Time calculation: [user-response-time]

+ ([request_network_time] / [request_network_n])

+ ([response_network_time] / [response_network_n])

+ ([server_delay] / [server_delay_n])

- only processed for “named” applications (e.g. excludes: ICMP, SNMP, TCP_Unknown, UDP_Unknown)

2 - [Metric: Throughput]: only monitored for VoIP-related applications: {VOIP, SIP, RTP}.

3 - [Client Location]: Aternity City/Country/State.

What data is available to Riverbed IQ for automated investigation?

- Riverbed IQ has access to a wide array of data served by the Data Ocean. The Data Ocean comprises all data sources connected to Riverbed IQ, including Riverbed Network Performance Monitoring (NPM) products, and any integrated third-party tooling/infrastructure.
- Riverbed Network Performance Monitoring (NPM) products are currently the only data sources that stream a preconfigured subset of high-value metrics that are leading-indicators of critical issues (i.e., key measurements) into the Riverbed IQ analytics pipeline for processing.
- Riverbed IQ automation is able to tap in to all data sources connected to Riverbed IQ in the course of an automated investigation, including Riverbed Network Performance Monitoring (NPM) products, and any integrated third-party tooling/infrastructure. This is a vast array of information that provides Riverbed IQ the ability to gather additional supporting data/-context to help progress an investigation (i.e. additional metrics, properties, attributes, other integrated third-party information/capabilities, …)

Is it possible to customize the data processed by Riverbed IQ?

- Riverbed IQ has access to a wide array of data served by the Data Ocean. The Data Ocean comprises all data sources connected to

Riverbed IQ, including Riverbed Network Performance Monitoring (NPM) products, and any integrated third-party tooling/infrastructure. Some of this data is preconfigured, and some can be customized.

- Riverbed Network Performance Monitoring (NPM) products are currently the only data sources that stream a preconfigured subset of high-value metrics that are leading-indicators of critical issues (i.e., key measurements) into the Riverbed IQ analytics pipeline for processing. Key measurements are preconfigured and cannot be customized.
- Riverbed IQ automation is able to tap in to all data sources connected to Riverbed IQ in the course of an automated investigation, including Riverbed Network Performance Monitoring (NPM) products, and any integrated third-party tooling/infrastructure. Riverbed IQ automation has the ability to target retrieval of specific additional supporting data/context from a data source to help execute an investigation (i.e., additional metrics, properties, attributes, other integrated third-party information/capabilities, …). This data can be customized to varying degrees:
    - Riverbed Network Performance Monitoring (NPM) products can be queried by Riverbed IQ automation for various combinations/permutations of the data the NPM products collect. The desired combinations/permutations of data needed by an automation is configured in the associated runbook Data Query node. In this way, the NPM data needed for an automated investigation can be customized to need.
    - Third-party tooling/infrastructure can be integrated into Riverbed IQ automation (via a provided third-party API) by configuring the desired API call[s] using runbook Integrations HTTP Request node (or an available subflow, and any other nodes needed to accomplish the desired objective). In this way, a variety of third-party tooling/infrastructure can be leveraged to customize an automated investigation.

Automation and Runbooks

How many runbooks are included, and can customers create their own?

- Since its inception, Riverbed IQ has vastly expanded the automation capabilities provided by runbooks to now include:
    - Incident Runbooks — Runbooks that are executed automatically when a new incident is generated.
    - Incident Lifecycle Runbooks — Runbooks that are executed automatically whenever an incident event occurs (i.e., incident events include: Impact Analysis Ready, Status Changed, Indicators Added, Ongoing State Changed, Note Added, …)
    - Webhook Runbooks — Runbooks that are executed automatically when a Riverbed IQ webhook is called.
    - On-Demand Runbooks — Runbooks that can be executed on-demand (outside the context of an incident or a webhook).
    - Subflows — These enable “containerization” of complex workflows or customized integrations into a single new runbook processing node which provides value when reused in other runbooks. This simplifies the runbook creation process and reduces the amount of touchpoints necessary when editing.
- For certain categories of runbook, Riverbed provides an out-of-the-box generic implementation to help provide benefit from day one:
    - Incident Runbooks — Riverbed packages an out-of-the-box generic implementation for all possible triggering incidents:
        - Interface Incident
        - Device Down Incident
        - Multi-Device Down Incident
        - Application Location Incident

    - Incident Lifecycle Runbooks — No out-of-the-box runbook provided.
    - Webhook Runbooks — No out-of-the-box runbook provided.
    - On-Demand Runbooks — No out-of-the-box runbook provided.
    - Subflows — Riverbed packages a set of common reusable workflows.
- Customers can easily duplicate runbooks, export/import them, edit, and even easily create new runbooks. Customers can also purchase professional services to assist in building runbooks.
- Customers can create multiple runbooks of all types.
    - The expectation is that a customer already has their own processes/procedures for performing the work done by the out-of-the-box runbooks, and that they will customize those out-of-the-box runbooks to automate those processes/procedures.
        - **NOTE:** For automatically-executed runbooks, only one runbook instance can be configured as the active association to execute when a triggering event occurs (e.g., there may be many runbooks created to perform automated investigations of application-triggered incidents, but only one can be configured as the active executable at any time the associated trigger occurs).
- Riverbed IQ does not limit the number of runbooks on a system.
    - **NOTE:** While the number of runbooks on a system is not limited, the number of runbook executions is directly related to the licensing to which a customer is entitled (i.e., a customer may build out a large runbook library at no extra charge, but the customer will be limited in the number of runbooks they can execute in a given period of time). Runbook execution entitlement usage will include those incident runbooks automatically executed on creation of new incidents, those incident lifecycle runbooks automatically executed

on lifecycle events, those webhook runbooks executed in response to an external API call, those on-demand runbooks that are executed as needed, etc..

What skills are needed to build a runbook?

- Runbooks are built using a graphical user-interface to construct a graphical workflow. The Runbook Editor provides a GUI that:
    - Presents a list of available runbook nodes in the palette (there are a variety of node types available to perform various functions, e.g., query network sensors, perform logic tests to make processing decisions, render collected data into a desired visualization, …).
    - Supports drag/drop of runbook nodes from the palette to the canvas, or, alternatively, runbook nodes can be added using the plus-sign icon on a runbook node already present on the canvas.
    - Allows each runbook node to be configured for purpose (e.g., query a network sensor for specific information in a certain time-period).
    - Allows runbook nodes to be interconnected into logical investigative paths that collect data, apply logic, and format output to expedite problem resolution.
- The goal of the runbook is to allow an experienced troubleshooter to build the scripted workflow that they would normally go through to troubleshoot an issue, so that a less experienced resource can view the output of the incident report.
    - The available runbook nodes cover the spectrum from simpler no-code/low-code nodes to the more complex nodes (e.g., HTTP Request node used for third-party integrations). Troubleshooters experienced in gathering debug information from network sensors should easily be able to codify workflows using available no-code/low-code nodes. Troubleshooters experienced in gathering debug information via APIs should be able to codify workflows that

integrate third-party tooling/infrastructure using more complex nodes.

Security

Does Riverbed IQ allow a user to self-service a password reset?

- Riverbed IQ never stores customer user passwords; it relies on the authentication mechanisms outlined below:
    - Option A: Azure Business-to-Business (B2B) — When your organization supports Azure B2B collaboration (e.g., enrolled in Office 365), authentication is handled solely by your Azure Active Directory (AD) and its policies/capabilities.
    - Option B: Microsoft Account — When your organization does not support Azure B2B collaboration, a Personal Microsoft Account needs to be linked to Riverbed IQ. In this case, password reset is provided through the Microsoft User Portal for the account.
    - Option C: One-time Passcode Fallback — Suppose none of the preceding options is chosen. In that case, Riverbed will Email a one-time passcode to the Email address configured for the user account whenever it tries to authenticate against Riverbed IQ.

What security standards does your SaaS meet?

- Riverbed IQ holds the following security-related certifications and attestations:
    - ISO/IEC 27001 — Certification of Riverbed's information security management controls for areas such as data security and business continuity.
    - SOC 2 Type II — Assessment of Riverbed's internal controls and systems related to security, availability, processing integrity, confidentiality, and privacy of data.
- Detailed copies of this material may be viewed or requested by visiting the Riverbed website, The Trust Center at

https://www.riverbed.com/trust-center/.

- In general, direct security questions to the Riverbed IQ SME team and product management.

What protocol is used for communication to the Edge device and from the Edge to the SaaS? Is it TLS 1.3? Secure Restful API? Other?

- Riverbed IQ Edge communication employs a variety of communications channels depending on those entities with which it interconnects. The various channels can use different technologies/protocols. The specific protocols in use are described in Firewall Rules.

What's New In This Version of Riverbed IQ?

Riverbed IQ introduces new and enhanced features frequently. This page announces new functionality added in recent versions.

Note that some features are made available to you before their functionality is built out completely, and, similarly, these features may be available before they're described completely in the user assistance. Feel free to begin using new features as soon as they're available, and check back here for enhancements to them.

October 3, 2024

Application Roll-Up

Riverbed IQ now offers enhanced incident management by correlating multiple Application-Location indicators into a single incident. This means that similar issues across multiple applications at the same location, or the same application across different locations, are grouped together, reducing noise, accelerating root-cause analysis, and allowing for faster, more efficient resolution of problems. The UI features an enhanced Automation Management page with new triggers, ready-to-use runbooks for automated investigations, and new incident types tailored to these events. You'll also have flexible options to configure and manage which applications should be correlated.

August 30, 2024

Subflow Versioning

Riverbed IQ Subflows are now versioned.

July 18, 2024

Scheduled Runbooks

Riverbed IQ now provides the ability to schedule the unattended execution of your On-Demand Runbooks! You can schedule On-Demand Runbooks to execute at specific dates and times, and on recurring hourly, daily, and weekly bases. The new Runbook Scheduling page lists your configured scheduled Runbooks and provides the Schedule Runbook wizard for configuring On-Demand Runbooks for scheduled execution.

Analytics & Threshold Configuration

Riverbed IQ now exposes the configurables employed by the Data Analytics Pipeline to identify anomalous measurements for all Key Measurements. The new Analytics & Threshold Configuration page, provides a listing of all Key Measurements (according to Entity type: Devices, Interfaces, and Applications), with the ability to enable/disable processing on a per Key Measurement basis, and the ability to customize the configuration of Threshold-based Key Measurements in order to tune Riverbed IQ to your environment.

June 27, 2024

Integrations Library

Riverbed IQ now provides an Integrations Library comprising a variety of high-value Integrations, simplifying and accelerating their configuration to bring all of your ecosystem's data together in one place for processing. When you install an Available Integration from the Integrations Library, the associated capability will appear in the Runbook Nodes palette and be available for use in Runbook automations.

April 26, 2024

On-Demand Runbooks

You can now create runbooks that you can execute manually as needed. These are different from incident runbooks that execute automatically for new incidents and life-cycle runbooks that execute automatically for life-cycle events.

Custom Decision Branch Labels

You can now assign customizable labels to a Decision Branch node's outputs, making it easier to annotate each branch on the runbook canvas and track what it does.

Preferred Data Sources

If you use a mix of AppResponse and NetProfiler data sources, you can now specify a preferred data source for streaming key metrics into the Analytics Pipeline. In all cases, all data sources can be queried during Runbook execution.

April 16, 2024

Custom UI Filter Configuration

The Riverbed IQ UI has been enhanced to allow flexible configuration of filter facets. The IQ UI has a number of filterable pages with default filter configurations; Customize Filters can be used, now, to show the set of available and selected facets and configure the filters. This is especially useful for optimizing the Explorer page to show/hide custom property filters.

Runbook Editor Usability

The construction of Liquid templates in runbook nodes has been simplified by an enhanced Template field. The Template field now provides an Insert control

that simplifies the addition of elements from a trigger, parent, or variable, as well as a built-in parser that detects and highlights formatting issues.

Runbook Test Usability

Executing a test run for a runbook now remembers prior inputs (initial conditions for logic processing) so that test runs can be re-executed in rapid succession, or at a later time, without having to re-enter the same inputs each time.

Explorer Detail Enhancements

The Device Explorer and Interface Explorer pages have been enhanced to expose all available properties, making a rich set of searchable, sortable, and filterable information for devices and interfaces available at your fingertips.

OAuth 2.0 Support for Custom Parameters

OAuth 2.0 functionality has been extended to support advanced configuration of custom parameters in the OAuth 2.0 Token Request. This expands the breadth of OAuth 2.0 integrations available to Riverbed IQ.

AppResponse Data Source Vantage Points

Riverbed IQ support for AppResponse as a data source has been enhanced to enable runbook automations to pivot their data-gathering perspective between the various distributed AppResponse instances using the new vantage point property provided in runbook data query nodes.

User Assistance Table of Contents

This user assistance has been enhanced with a table of contents (see the pane at the left) that exposes and surfaces the breadth and depth of available documentation.

February 7, 2024

Custom Properties

Custom properties enable you to define meaningful attributes that you can associate with monitored entities (tag) to enrich the available dataset with custom metadata. This custom data can be used to influence runbook behaviors and improve decision making.

Subflows

Subflows enable “containerization” of complex workflows or customized integrations into a single new runbook processing node which provides value when reused in other Runbooks. This simplifies the runbook creation process and reduces the amount of touchpoints necessary when editing.

October 27, 2023

Custom Triggers

This feature provides a customizable way to configure a trigger for a specific event and map it to a custom investigation (e.g., only execute runbook “R1” when there is a Sharepoint application event). This capability provides a simple way for users to identify and intercept specific events of interest, and perform detailed custom analysis that is informed by their shared practical experience.

Async Support in HTTP Request Node

This is a new capability added to the HTTP Request node in Runbooks that provides the ability to pause execution at the HTTP Request Node in anticipation of an asynchronous response. Simply configure the new *Retry until Ready* section in the HTTP Request node properties to support asynchronous response interactions.

August 24, 2023

Incident Lifecycle Events

Incident lifecycle events provide the ability to execute an associated automation whenever there is activity on an incident. Currently supported events are Status Changed, Indicators Updated, Ongoing State Changed, Runbook Completed, Note Added, and Note Updated. This functionality can be used to integrate with broader ecosystems and to streamline existing workflows (e.g., ITSM systems).

Webhook Trigger

The webhook trigger is a powerful new capability that enables external sources to execute custom automations that leverage the power of the LogIQ Engine.

August 18, 2023

Video tutorials now are available acrossRiverbed IQ -- simply click on the Learning icon to access. Video tutorials are available at all levels with global and contextual quick-links provided for User Assistance Video Tour, Runbook Editor Video Tour, and Incident Runbooks Video Tour.

July 6, 2023

The Incidents page has been enhanced to offer more flexibility than ever before. These enhancements include more granular, intuitive filtering, alongside the ability to perform fuzzy searches, exact matches, and exclusions, everything needed to identify a target set of incidents. Even better, you can now save any filtered search for quick access.

May 24, 2023

Runtime Variables

Variables can be used in runbook definitions, now. A number of built-in variables are provided, and you can create user-defined variables, as well. At this time, all variables are runtime scope variables, meaning that they are persistent over a single execution of a runbook.

Multi-Device Down Analysis Runbook

The Runbooks page provides a new built-in runbook named Multi-Device Down Analysis. Multi-Device Down Incidents are generated when Riverbed IQ detects multiple devices down at a individual location.

Monitoring Health

The Monitoring Health page summarizes the status and performance of your Riverbed IQ instance.